

LAYER: 1_Core

This file is a merged representation of a subset of the codebase, containing files not matching ignore patterns, combined into a single document by Repomix.

<file_summary>

This section contains a summary of this file.

<purpose>

This file contains a packed representation of a subset of the repository's contents that is considered the most important context.

It is designed to be easily consumable by AI systems for analysis, code review, or other automated processes.

</purpose>

<file_format>

The content is organized as follows:

1. This summary section
2. Repository information
3. Directory structure
4. Repository files (if enabled)
5. Multiple file entries, each consisting of:
 - File path as an attribute
 - Full contents of the file

</file_format>

<usage_guidelines>

- This file should be treated as read-only. Any changes should be made to the original repository files, not this packed version.
- When processing this file, use the file path to distinguish between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with the same level of security as you would the original repository.

</usage_guidelines>

<notes>

- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching these patterns are excluded: **/obj/**, **/bin/**, **/*.png, **/*.jpg, **/node_modules/**, **/*.Designer.cs
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Files are sorted by Git change count (files with more changes are at the bottom)

</notes>

</file_summary>

<directory_structure>

Configuration.Web/Components/_Imports.razor
Configuration.Web/Components/Layout/MainLayout.razor
Configuration.Web/Components/Layout/ReconnectModal.razor
Configuration.Web/Components/NavMenu.razor
Configuration.Web/Components/NotFound.razor
Configuration.Web/Components/Routes.razor
Configuration.Web/Configurator/IWebAppConfigurator.cs
Configuration.Web/Configurator/WebAppConfigurator.cs
Configuration.Web/Configurator/WebScanningExtensions.cs
Configuration.Web/Promatis.Net.Configuration.Web.csproj
Configuration.Web/Properties/launchSettings.json
Configuration.Web/Services/TabNavigationService/ComponentRegistry.cs
Configuration.Web/Services/TabNavigationService/TabItem.cs
Configuration.Web/Services/TabNavigationService/TabNavigationService.cs
Configuration.Web/Services/UiModuleService.cs
Configuration.Web/UiCommandBus/IUiCommandBus.cs
Configuration.Web/UiCommandBus/UiCommandBus.cs
Configuration.Web/WebAppBootstrapper.cs
Configuration/AppBootstrapper.cs
Configuration/AppConfigurator.cs
Configuration/AppInfrastructure.cs
Configuration/IAppConfigurator.cs
Configuration/Promatis.Net.Configuration.csproj
Configuration/ServiceScanningExtensions.cs
Data/Configurations/AuditLogConfiguration.cs
Data/Configurations/ModelBuilderConventions.cs
Data/DbContext/ApplicationDbContext.cs

Data/DbContext/ModelBuilderExtensions.cs
Data/Interceptors/ChangeEntryModel.cs
Data/Interceptors/DatabaseTriggerInterceptor.cs
Data/Interceptors/IUserProvider.cs
Data/Promatis.Net.Data.csproj
Data/Triggers/Castom/AuditTrigger.cs
Data/Triggers/Global/FluentValidationTrigger.cs
Data/Triggers/Global/ReferenceTreeParentTrigger.cs
Data/TriggerService/DatabaseTriggerService.cs
Data/TriggerService/EntityStateChangeEnum.cs
Data/TriggerService/IDatabaseTriggerService.cs
Data/TriggerService/PropertyChangeInfo.cs
Data/TriggerService/TriggerEvents/EntityCancelArgsBase.cs
Data/TriggerService/TriggerEvents/EntityCancelEventArgs.cs
Data/TriggerService/TriggerEvents/EntityChangedEventArgsBase.cs
Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs
Data/TriggerService/TriggerEvents/EntityEventArgsBase.cs
Data/TriggerService/TriggerInterfaces/IAfterSaveTrigger.cs
Data/TriggerService/TriggerInterfaces/IBeforeSaveTrigger.cs
Data/Validation/GlobalPolymorphicValidator.cs
Domain.Interface/Enums/EnumExtensions.cs
Domain.Interface/Interfaces/IAudit.cs
Domain.Interface/Interfaces/IDomainObject.cs
Domain.Interface/Interfaces/IDomainObjectHasKey.cs
Domain.Interface/Interfaces/IEntityCloner.cs
Domain.Interface/Interfaces/ISoftDeletable.cs
Domain.Interface/Interfaces/ITreeNode.cs
Domain.Interface/Promatis.Net.Domain.Interface.csproj
Domain/AuditLog/AuditLog.cs
Domain/DomainObject/DomainObject.cs
Domain/DomainObject/DomainObjectBase.cs
Domain/DomainObject/DomainObjectValidator.cs
Domain/EntityCloner/JsonEntityCloner.cs
Domain/Promatis.Net.Domain.csproj
Domain/ReferenceBase/ReferenceBase.cs
Domain/ReferenceBase/ReferenceBaseValidator.cs
Domain/ReferenceTreeBase/ReferenceTreeBase.cs
Domain/ReferenceTreeBase/ReferenceTreeBaseValidator.cs
Promatis.Net.UI/_Imports.razor
Promatis.Net.UI/Components/ActionContext/CrudWorkspaceContext.cs
Promatis.Net.UI/Components/ActionContext/IWorkspaceActionContext.cs
Promatis.Net.UI/Components/ActionContext/WorkspaceActionContext.cs
Promatis.Net.UI/Components/EditDialogs/DialogTab.razor
Promatis.Net.UI/Components/EditDialogs/DialogTab.razor.cs
Promatis.Net.UI/Components/EditDialogs/EditDialog.razor
Promatis.Net.UI/Components/EditDialogs/EditDialog.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/RenderBase.cs
Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor.cs
Promatis.Net.UI/Components/UIControls/BaseUiControl.cs
Promatis.Net.UI/Components/UIControls/IHasOptions.cs
Promatis.Net.UI/Components/UIControls/IHasValue.cs
Promatis.Net.UI/Components/UIControls/IUiControl.cs
Promatis.Net.UI/Components/Workspaces/GridPage.razor
Promatis.Net.UI/Components/Workspaces/GridPage.razor.cs
Promatis.Net.UI/Components/Workspaces/TreePage.razor
Promatis.Net.UI/Components/Workspaces/TreePage.razor.cs
Promatis.Net.UI/Components/Workspaces/UiToolbar.razor
Promatis.Net.UI/Components/Workspaces/UiToolbar.razor.cs
Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor
Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor.cs
Promatis.Net.UI/Controls/CreateEntityButton.cs
Promatis.Net.UI/Controls/DeleteEntityButton.cs
Promatis.Net.UI/Controls/EditEntityButton.cs
Promatis.Net.UI/Controls/ToolbarDivider.cs
Promatis.Net.UI/Data/IUiDataBroker.cs
Promatis.Net.UI/Data/IUiOzuCache.cs
Promatis.Net.UI/Data/UiDataBroker.cs
Promatis.Net.UI/Data/UiOzuCache.cs
Promatis.Net.UI/Extensions/MudColorExtensions.cs

```

Promatis.Net.UI/Interfaces/IHasSelectedData.cs
Promatis.Net.UI/IUiModule.cs
Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor
Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor.cs
Promatis.Net.UI/Pages/AuditLogs/AuditLogWorkspaceContext.cs
Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditEntitySelect.cs
Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditExportButton.cs
Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditPeriodPicker.cs
Promatis.Net.UI/Pages/Dashboard.razor
Promatis.Net.UI/Pages/Dashboard.razor.css
Promatis.Net.UI/Promatis.Net.UI.csproj
Promatis.Net.UI/Theme/PromatisTheme.cs
Promatis.Net.UI/UiModule.cs
Promatis.Net.UI/wwwroot/App.css
Service/Contracts/AuditLog/AuditLogSearchRequest.cs
Service/Contracts/AuditLog/PagedResult.cs
Service/Promatis.Net.Service.csproj
Service/Services/AuditLogService/AuditLogService.cs
Service/Services/AuditLogService/IAuditLogService.cs
Service/Services/BaseService/BaseService.cs
Service/Services/BaseService/IBaseService.cs
Service/Services/ReferenceService/IReferenceService.cs
Service/Services/ReferenceService/ReferenceService.cs
Service/Services/ReferenceTreeService/IReferenceTreeService.cs
Service/Services/ReferenceTreeService/ReferenceTreeService.cs
Service/Services/TreeService/ITreeService.cs
Service/Services/TreeService/TreeService.cs
Tests/AssemblyInfo.cs
Tests/Domain/DomainLogicTests.cs
Tests/Domain/DomainObjectBaseLogicTests.cs
Tests/Domain/DomainObjectBaseTests.cs
Tests/Domain/DomainObjectExtendedTests.cs
Tests/Domain/ReferenceBaseTests.cs
Tests/Domain/ReferenceTreeTests.cs
Tests/Domain/SoftDeletableTests.cs
Tests/Interceptor/DatabaseTriggerInterceptorTests.cs
Tests/Promatis.Net.ApplicationModel.Tests.csproj
Tests/Service/BaseServiceTests_Crud.cs
Tests/Service/BaseServiceTests.cs
Tests/Service/ReferenceServiceTests_Search.cs
Tests/Service/ReferenceTreeServiceTests.cs
Tests/Service/ServiceIntegrationTests.cs
Tests/Trigger/AuditTriggerTests.cs
Tests/Trigger/DatabaseTriggerServiceTests.cs
Tests/Trigger/FluentValidationTriggerTests.cs
Tests/Trigger/FullTriggerChainTests.cs
Tests/Trigger/ReferenceTreeParentTriggerTests.cs
Tests/Trigger/TreeTriggerChainTests.cs
Tests/Validator/DomainObjectValidatorTests.cs
Tests/Validator/ReferenceBaseValidatorTests.cs
Tests/Validator/ReferenceTreeBaseValidatorTests.cs
</directory_structure>

```

<files>

This section contains the contents of the repository's files.

<file path="Configuration.Web/Components/_Imports.razor">

```

@using System.Net.Http
@using System.Net.Http.Json
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using Microsoft.JSInterop
@using MudBlazor
@using Promatis.Net.Ui
@using Promatis.Net.Configuration.Web
@using Promatis.Net.Configuration.Web.Components
@using Promatis.Net.Configuration.Web.Components.Layout
@using static Microsoft.AspNetCore.Components.Web.RenderMode
</file>

```

<file path="Configuration.Web/Components/Layout/MainLayout.razor">

```

@inherits LayoutComponentBase
@using Promatis.Net.Ui.Theme
@
@inject UiModuleService UiService

```

```

@Inject TabNavigationService TabService
@implements IDisposable
{
    @* A,,!Aô A A´ Aô : B 2Dô7D´2C 5CÂ ?D >C$0C"4CT@ D$5CÄK D DC´0C4>CÂ BE <CÔ>C4> D 5Cd8CÄ0 *@
    <MudThemeProvider Theme="PromatisTheme.DefaultTheme" @bind-IsDarkMode="_isDarkMode" />
    <MudPopoverProvider />
    {
        <MudDialogProvider FullWidth="false"
            MaxWidth="MaxWidth.False"
            CloseButton="false"
            BackdropClick="false"
            NoHeader="false"
            Position="DialogPosition.Center"
            CloseOnEscapeKey="true" />
    }
    <MudSnackbarProvider />
    {
        <CascadingValue Value="this">
            <MudLayout>
                @* A$5D ECô0Dò ?C =CT;DÂ ?D 8C´>Cd5C08Dò ¢
                <MudAppBar Color="Color.Default" Elevation="1">
                    @* A,,!Aô A A´ Aô : At0CÄ5C05C0> Color.Primary C00 Color.Default *@
                    <MudIconButton Icon="@Icons.Material.Filled.Menu"
                        Color="Color.Inherit"
                        Edge="Edge.Start"
                        Size="Size.Large"
                        OnClick="ToggleDrawer" />
                    <MudStack Spacing="-1">
                        @* AD5DD>C´BCô>CR =C 7C$0C08CR ¢
                        <MudText Typo="Typo.h6" Class="ml-2">
                            B$5D BCä2D´9 C0@Cä5C¢B Promatis Net
                        </MudText>
                    </MudStack>
                <MudSpacer />
                <MudIconButton Icon="@(_isDarkMode ? Icons.Material.Filled.LightMode :
                    Icons.Material.Filled.DarkMode)"
                    Color="Color.Inherit"
                    OnClick="ToggleDarkMode" />
            </MudAppBar>
            {
                @* Aô0C05C´L CÄ5C0N *@
                <MudDrawer @bind-Open="_drawerOpen" ClipMode="DrawerClipMode.Always"
                    Variant="DrawerVariant.Temporary" Overlay="true" OverlayAutoClose="true" Width="220px"
                    Elevation="1">
                    <NavMenu />
                </MudDrawer>
            }
            <MudMainContent Class="pt-16 px-4 pb-4 d-flex flex-column"
                Style="height: calc(100vh - 8px); min-height: calc(100vh - 8px); margin-top: 10px; display: flex; flex-direction: column; flex-grow: 1; overflow: hidden;">
                {
                    @* A,,!Aô A A´ Aô : A4;Cä1C ;DÄ=D´9 ErrorBoundary D41D 0C0 A C$5D EC05C4> D4@Cä2C00. B BC,,;C, fÄW
                    @if (TabService.OpenTabs.Count == 0)
                    {
                        <div class="flex-grow-1 overflow-auto">
                            <ErrorBoundary>
                                <ChildContent>
                                    @Body
                                </ChildContent>
                                <ErrorContent Context="exception">
                                    <MudAlert Severity="Severity.Error" Class="my-4">
                                        Aô@Cä8Ct>D,,;C :D 8D$8Dt5D :C 0 CäHC,,1C¢0 C,,=D$5D DCT9D 0: @exception.Message
                                    </MudAlert>
                                </ErrorContent>
                            </ErrorBoundary>
                        </div>
                    }
                    else
                    {
                        <MudTabs @bind-ActivePanelIndex="TabService.ActiveTabIndex"
                            Position="Position.Top"
                            Color="Color.Default"
                            SliderColor="Color.Primary"
                            Class="mdi-tabs-container w-100 flex-grow-1 d-flex flex-column"

```

```

                Style="height: 100%; max-height: 100%; min-height: 0; display: flex; flex-
direction: column; flex-grow: 1;"@
                TabPanelsClass="pa-0 flex-grow-1 d-flex flex-column overflow-hidden"@
                ApplyEffectsToContainer="true"@
                TabButtonsClass="rounded-t-lg shadow-sm"@
                Outlined="true">@
@
        @for (int i = 0; i < TabService.OpenTabs.Count; i++)@
        {@
            int localIndex = i;@
            TabItem tab = TabService.OpenTabs[i];@
@
            @* A,,!Aô A A´ Aô : AD>C 0C$;CT=C fÆW,ôAD$@D4:D$CD 0 CD;Dò AC <Cä3Cä BC 1-Cô0C05C´0,
<MudTabPanel @key="tab" ID="@localIndex" Class="h-100 d-flex flex-column">@
                <TabContent>@
                    <div class="d-flex align-center gap-2" style="cursor: pointer;">@
                        @if (!string.IsNullOrEmpty(tab.Icon))@
                        {@
                            <MudIcon Icon="@tab.Icon" Size="Size.Small" />@
                        }@
                        <MudText Typo="Typo.body2" Class="font-weight-
medium">@tab.Title</MudText>@
@
                            <MudIconButton Icon="@Icons.Material.Filled.Close"@
                                Size="Size.Small"@
                                Color="Color.Default"@
                                Class="pa-0 ms-1"@
                                Style="width: 16px; height: 16px; min-width:
16px;"@
                                OnClick="@((e) =>
TabService.CloseTabByIndex(localIndex))" />@
                            </div>@
                        </TabContent>@
                        <ChildContent>@
                            @* BôBCäB Cæ>CôBCT9Cô5D 6CTAD$:Cä 7C =C,,<C 5D" R 2D´ACäBD² 2Cæ;C 4Cæ8 C,
<div class="pa-2 rounded-b-lg w-100 d-flex flex-column flex-grow-1"@
                                Style="height: 100%; max-height: 100%; min-height: 0;
overflow: hidden;">@
@
                                    <ErrorBoundary>@
                                        <ChildContent>@
                                            @* A,,!Aô A A´ Aô : AÄK Cä1Cä@C GC,,2C 5CÄ G-æ Ö-46ö× öæVçB 2 DD8C
                                            @* Cæ>D$>D KC´ AC,,;Cä9 C$KD$0C48C$0CTB D BD 0C08DdC C00D,,5C4> CD5
                                            <div class="d-flex flex-column flex-grow-1 w-100"
style="height: 100%; min-height: 0;">@
                                                <DynamicComponent Type="@tab.ComponentType" />@
                                            </div>@
                                        </ChildContent>@
                                        <ErrorContent Context="exception">@
                                            <MudAlert Severity="Severity.Error" Class="my-4">@
                                                Aô@Cä8Ct>D,,;C :D 8D$8Dt5D :C 0 CäHC,,1Cæ0 C,,=D$5D DCT9D 0 C$:
                                            </MudAlert>@
                                        </ErrorContent>@
                                    </ErrorBoundary>@
@
                                </div>@
                            </ChildContent>@
                        </MudTabPanel>@
                    }@
                </MudTabs>@
            }@
        </MudMainContent>@
    </MudLayout>@
</CascadingValue>@
@
@code {@
    private bool _drawerOpen = true;@
    private bool _isDarkMode; // Aô> D4<Cä;Dt0C08Dâ ?D 8C´>Cd5C08CR 7C ?D4ACæ0CTBD 0 C" AC$5D$;Cä9 D$5CÄ5D
@
    protected override void OnInitialized()@
    {@
        base.OnInitialized();@
        TabService.OnTabsChanged += HandleTabsChanged;@
        _drawerOpen = false;@
    }@
@

```

```

private void ToggleDrawer() => _drawerOpen = !_drawerOpen;
}
/// <summary>
/// A05D 5C;DäGC 5D" DC`0C2 BE <C0>C4> D 5Cd8CÄ0 C, 7C AD$0C$;D05D" xVEF†VÖU &÷f-FW" <C4=Cä2CT=C0> C05D
/// </summary>
private void ToggleDarkMode() => _isDarkMode = !_isDarkMode;
}
private void HandleTabsChanged() => InvokeAsync(StateHasChanged);
}
public void CloseDrawer()
{
    if (_drawerOpen)
    {
        _drawerOpen = false;
        StateHasChanged();
    }
}
public void Dispose() => TabService.OnTabsChanged -= HandleTabsChanged;
}
</file>

<file path="Configuration.Web/Components/Layout/ReconnectModal.razor">
<script type="module" src="@Assets["Components/Layout/ReconnectModal.razor.js"]"></script>
}
<dialog id="components-reconnect-modal" data-nosnippet>
    <div class="components-reconnect-container">
        <div class="components-rejoining-animation" aria-hidden="true">
            <div></div>
            <div></div>
        </div>
        <p class="components-reconnect-first-attempt-visible">
            Rejoining the server...
        </p>
        <p class="components-reconnect-repeated-attempt-visible">
            Rejoin failed... trying again in <span id="components-seconds-to-next-attempt"></span>
seconds.
        </p>
        <p class="components-reconnect-failed-visible">
            Failed to rejoin.<br />Please retry or reload the page.
        </p>
        <button id="components-reconnect-button" class="components-reconnect-failed-visible">
            Retry
        </button>
        <p class="components-pause-visible">
            The session has been paused by the server.
        </p>
        <p class="components-resume-failed-visible">
            Failed to resume the session.<br />Please retry or reload the page.
        </p>
        <button id="components-resume-button" class="components-pause-visible components-resume-
failed-visible">
            Resume
        </button>
    </div>
</dialog>
</file>

<file path="Configuration.Web/Components/NavMenu.razor">
@using System.Reflection
@inject UiModuleService UiService
@inject TabNavigationService TabService
@inject ComponentRegistry Registry
}
<MudNavMenu>
    @{
        List<(string Title, string Href, string Icon, string? Group)> allItems = UiService.Modules
            .SelectMany(m => m.GetMenuItems())
            .ToList();
}
// AäBD 8D >C$KC$0CT< D0;CT<CT=D$K C$5D EC05C4> D4@Cä2C00
IEnumerable<(string Title, string Href, string Icon, string? Group)> topLevelItems =
allItems.Where(i => string.IsNullOrEmpty(i.Group));
foreach ((string Title, string Href, string Icon, string? Group) item in topLevelItems)
{
    <!-- A,,!A0 A A` A0 : A$<CTAD$> Href C,,AC0>C`LCtCCT< OnClick CD;D0 >D$:D KD$8D0 2C;C 4Cä: C 5Cr

```

```

        <MudNavLink Icon="@item.Icon" OnClick="@(() => OpenMenuAsTab(item.Title, item.Href,
item.Icon))">@
            @item.Title@
        </MudNavLink>@
    }@
}
// A4@D4?C08D CCT< D0;CT<CT=D$K C0> C@0D$5C4>D 8D0<@
IOrderedEnumerable<IGrouping<string, (string Title, string Href, string Icon, string?
Group)>> pooledGroups = allItems@
    .Where(i => !string.IsNullOrEmpty(i.Group))@
    .GroupBy(i => i.Group!)@
    .OrderBy(g => g.Key);@
}
foreach (IGrouping<string, (string Title, string Href, string Icon, string? Group)> group
in pooledGroups)@
{
    <MudNavGroup Title="@group.Key" Icon="@Icons.Material.Filled.Folder" Expanded="false">@
        @foreach ((string Title, string Href, string Icon, string? Group) item in group)@
        {
            <!-- A,,!Aô A A' Aô : B41D 0C0 ‡&VbÂ 4Câ1C 2C'5C0 ôâ6Æ-6² 0ô1
            <MudNavLink Icon="@item.Icon" OnClick="@(() => OpenMenuAsTab(item.Title,
item.Href, item.Icon))">@
                @item.Title@
            </MudNavLink>@
        }@
    </MudNavGroup>@
}
}
</MudNavMenu>@
}
@code {
    /// <summary>@
    /// A05D 5DT2C BD'2C 5CÂ AC@2Câ7C0>C' MC@7CT<C0;D0@ D >CD8D$5C'LD :Câ3Câ <C :CTBC
    /// </summary>@
    [CascadingParameter]@
    protected MainLayout? MainLayoutInstance { get; set; }@
}
protected override void OnInitialized()@
{
    base.OnInitialized();@
    IEnumerable<Assembly> assemblies = UiService.Modules.Select(m =>
m.GetType().Assembly).Distinct();@
    Registry.RegisterModules(assemblies);@
}
}
private void OpenMenuAsTab(string title, string href, string icon)@
{
    Type? componentType = Registry.GetComponentTypeByRoute(href);@
}
if (componentType != null)@
{
    // AâBC@D'2C 5CÂ 2C@;C 4C@C@
    TabService.OpenTab(title, href, icon, componentType);@
}
// A 2D$>CÂ0D$8Dt5D :C, 7C :D KC$0CT< C >C@>C$>CR <CT=Dâ ?Câ2CT@DR >C@=C ?CâAC'5 C@;C,,:C
MainLayoutInstance?.CloseDrawer();@
}
else@
{
    System.Diagnostics.Debug.WriteLine($"A0CD$L {href} C05 Ct0D 5C48D BD 8D >C$0C0 2 C@>CÂ?Câ=CT=D$0D
}
}
}
</file>

<file path="Configuration.Web/Components/NotFound.razor">
@page "/not-found"@
@layout MainLayout@
}
<PageTitle>404 - B BD 0C08Dd0 C05 C00C"4CT=C Âö vUF-FÆSı
}
<MudContainer MaxWidth="MaxWidth.Small" Class="d-flex align-center justify-center" Style="height:
70vh;">@
    <MudPaper Elevation="3" Class="pa-8 text-center" Style="width: 100%;">@
        <MudIcon Icon="@Icons.Material.Filled.SearchOff" Color="Color.Error" Size="Size.Large"
Class="mb-4" />@

```

```

<MudText Typo="Typo.h5" Class="mb-2">AäHC,,1C□0 404</MudText>ð
<MudText Typo="Typo.body2" Class="mud-text-secondary mb-6">ð
    A,,7C$8C08D$5, Ct0C0@C HC,,2C 5CÄKC' 0CD@CTA C05 D CD"5D BC$CCTB, C'8C > D >CäBC$5D$AD$2D4ND"8C' <C
</MudText>ð
<MudButton Variant="Variant.Filled" Color="Color.Primary" href="/">ð
    A$5D =D4BDÄAD0 =C 3C'0C$=D4Nð
</MudButton>ð
</MudPaper>ð
</MudContainer>
</file>

```

```

<file path="Configuration.Web/Components/Routes.razor">
@using System.Reflectionð
@inject UiModuleService UiServiceð
ð
<Router AppAssembly="@typeof(Routes).Assembly"ð
    AdditionalAssemblies="@_moduleAssemblies"ð
    NotFoundPage="@typeof(NotFound)">ð
    <Found Context="routeData">ð
        <RouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)" />ð
        <FocusOnNavigate RouteData="@routeData" Selector="h1" />ð
    </Found>ð
</Router>ð
ð
@code {ð
    private Assembly[] _moduleAssemblies = [];ð
    ð
    protected override void OnInitialized()ð
    {ð
        _moduleAssemblies = UiService.Modulesð
            .Select(m => m.GetType().Assembly)ð
            .Distinct()ð
            .ToArray();ð
        ð
        base.OnInitialized();ð
    }ð
}
</file>

```

```

<file path="Configuration.Web/Configurator/IWebAppConfigurator.cs">
namespace Promatis.Net.Configuration.Web;ð
ð
public interface IWebAppConfigurator : IAppConfiguratorð
{ð
    void ConfigureMiddleware(WebApplication app);ð
}
</file>

```

```

<file path="Configuration.Web/Configurator/WebAppConfigurator.cs">
using System.Reflection;ð
using Microsoft.AspNetCore.Components;ð
using MudBlazor.Services;ð
ð
namespace Promatis.Net.Configuration.Web;ð
ð
/// <summary>ð
/// A 0Ct>C$KC' :Cä=DD8C4CD 0D$>D 4C'0 Web-C0@C,,;Cä6CT=C,,9 C00 C 0Ct5 Blazor C, xVD&Æ !÷"í
/// </summary>ð
public class WebAppConfigurator<TRootComponent> : AppConfigurator, IWebAppConfiguratorð
    where TRootComponent : IComponentð
{ð
    public override void ConfigureServices(IServiceCollection services, IConfiguration
configuration)ð
    {ð
        // 1. A 0Ct>C$0D0 8C0DD 0D BD CC=BD4@C ,, ABÂ !CT@C$8D K, A$0C'8CD0D$>D K, B$@C,,3C45D K, B :C =CT@ DL
        base.ConfigureServices(services, configuration);ð
    ð
        // 2. UI C,,=DD@C AD$@D4:D$CD 0 (Aô>C,,AC¢ •V"ÖöGVÆR 2 D 1Cä@C=0DRÄ =C 2C,,3C FC,,0)ð
        services.AddWebInfrastructure(projectPrefix: "Promatis.");ð
    ð
        // B B "AT A A□ A Ää (MDI): B 5C48D BD 8D CCT< C,,=DD@C AD$@D4:D$CD C CÄ=Cä3Cä2C□;C 4CäGC0>C4> C,,
        services.AddSingleton<ComponentRegistry>();ð
        services.AddScoped<TabNavigationService>();ð
    ð
        // 3. B ?CTFC,,DC,,:C &Æ !÷" ,,~çFW& 7F~fR 6W'fW"•
        services.AddRazorComponents()ð
    }
}

```



```

        .AddInteractiveServerComponents();
    }

    // 4. B 5C48D BD 0Dd8Dò 8 C00D BD >C":C xVD&Æ |÷-
    services.AddMudServices(config =>
    {
        config.SnackbarConfiguration.PositionClass =
MudBlazor.Defaults.Classes.Position.BottomRight;
        config.SnackbarConfiguration.PreventDuplicates = false;
        config.SnackbarConfiguration.NewestOnTop = true;
        config.SnackbarConfiguration.ShowCloseIcon = true;
        config.SnackbarConfiguration.VisibleStateDuration = 5000;
    });

    // 5. B,,8C00 UI-D >C KD$8C•
    services.AddScoped<IUiCommandBus, UiCommandBus>();

    // B 5C48D BD 0Dd8Dò AC,,AD$5CÄ=Cä3Cä 0C=ACTAD >D 0 CD;Dò ?Cä4CD5D 6C=8 C´5C08C$>C4> D 0Ct@CTHCT=C,,O S
    services.AddHttpContextAccessor();
}

/// <summary>
/// A00D BD >C":C :Cä=C$5C"5D 0 Cä1D 0C >D$:C, ...EE 07C ?D >D >C" „Ö-FFÆWv &R'í
/// </summary>
public virtual void ConfigureMiddleware(WebApplication app)
{
    // A 0Ct>C$KCR Ö-FFÆWv &W2 4C`O D 0C >D$K Web-Cô@C,,;Cä6CT=C,,0D
    app.UseStaticFiles();
    app.UseAntiforgery();

    // 1. A0>C´CDt0Ct< Ct0D 5C48D BD 8D >C$0C0=D´5 UI-D 1Cä@C=8 C,,7 DI-C= >C0BCT9C05D 0 DT>D BC
    using IServiceScope scope = app.Services.CreateScope();
    var uiService = scope.ServiceProvider.GetRequiredService<UiModuleService>();

    Assembly[] moduleAssemblies = uiService.Modules
        .Select(m => m.GetType().Assembly)
        .Distinct()
        .ToArray();

    // A,, A,,&A,, A´ At Bd Bò A B$+ A$ A´ AD AÆ¢ C ?Cä;C00Ct< D 5CTAD$@ D >D4BCä2 C= >CÄ?Cä=CT=D$0CÄ8 C,,7
    var componentRegistry = scope.ServiceProvider.GetRequiredService<ComponentRegistry>();
    componentRegistry.RegisterModules(moduleAssemblies);

    // 2. B 5C48D BD 8D CCT< C= >D =CT2Cä9 C= >CÄ?Cä=CT=D" 8 D :C =C,,@D45CÄ AD$@C =C,,FD² <Cä4D4;CT9 C00 D 5
    app.MapRazorComponents<TRootComponent>()
        .AddInteractiveServerRenderMode()
        .AddAdditionalAssemblies(moduleAssemblies);
}

public override void ConfigureApp(IHost app)
{
    // A$Kct>C" 1C 7Cä2Cä9 C,,=C,,FC,,0C´8Ct0Dd8C, „0C$BCä@CT3C,,AD$@C FC,,0 D$@C,,3C45D >C" AB GCT@CT7 D :C =
    base.ConfigureApp(app);

    // At4CTADÄ <Cä6C0> CD>C 0C$8D$L C0@Cä2CT@C=C Ct4Cä@Cä2DÄO A C,,;C, =C GC ;DÄ=D´9 Seed CD0C0=D´E CD;
}
}
</file>

<file path="Configuration.Web/Configurator/WebScanningExtensions.cs">
using Promatis.Net.UI;
using System.Reflection;
namespace Promatis.Net.Configuration.Web;
public static class WebInfrastructureExtensions
{
    public static void AddWebInfrastructure(this IServiceCollection services, string projectPrefix
= "Promatis.")
    {
        Console.WriteLine("[SCANNER] At0C0CD : C0@Cä3D 5C$0 D 1Cä@Cä: CÄ>C0>C´8D$0 (B$>C´LC= > UI-CÄ>CDCC´8)..
        string binPath = AppContext.BaseDirectory;

        // A,,!A0 A A´ A0 : A00 D4@Cä2C05 DD0C";Cä2Cä9 D 8D BCT<D² 8D"5CÄ BCä;DÄ:Cä BCR FÆÄÄ :CäBCä@D´5 C00Dt
        string[] assemblyFiles = Directory.GetFiles(binPath, $"{projectPrefix}*.UI.dll",
        SearchOption.TopDirectoryOnly);
    }
}

```

```

    foreach (string file in assemblyFiles)
    {
        try
        {
            // A0C,,=D44C,,BCT;DÄ=Câ 7C 3D CCd0CT< C" 76VÖ&Ç"Æö D6öçFW‡BäFVf VÇM
            Assembly.LoadFrom(file);
        }
        catch (Exception ex)
        {
            Console.WriteLine($"[SCANNER] A05 D44C ;CâADÂ 7C 3D CCT8D$L DD0C"; D 1Câ@C=8 {Path.GetFileName
            });
        }
    }

    // A,,!Aô A A' Aô : BD8C'LD$@D45CÂ FöÖ -âÂ 1CT@Dò AC >D :C, AD$@Cä3Câ A Cò@CTDC,,:D >CÂ 8 Cä:Cä=Dt0
    Assembly[] assemblies = AppDomain.CurrentDomain.GetAssemblies()
        .Where(a => a.FullName != null
            && a.FullName.StartsWith(projectPrefix)
            && a.GetName().Name!.EndsWith(".UI"))
        .Distinct()
        .ToArray();

    Console.WriteLine($"[SCANNER] B :C =C,,@Cä2C =C,,5 Ct0C$5D HCT=Cââ C 9CD5C0> {assemblies.Length} Dd5C'
    Console.WriteLine();

    Console.WriteLine($"[SCANNER] B 5C48D BD 0Dd8Dò T'Ô:Cä<Cò>C05C0BCä2 C, <Cä4D4;CT9 C00C$8C40Dd8C, GCT@C
    // A 2D$>CÄ0D$8Dt5D :Cä5 D :C =C,,@Cä2C =C,,5 C, @CT3C,,AD$@C FC,,O C,,=D$5D DCT9D >C" •V"ÖöGVÆ]
    services.Scan(scan => scan
        .FromAssemblies(assemblies)
        .AddClasses(c => c.AssignableTo<IUiModule>().Where(t => !t.IsAbstract))
        .AsImplementedInterfaces()
        .WithScopedLifetime()
    );

    // B 5C48D BD 0Dd8Dò 2C HCT3Câ 0C4@CT3C BCä@C <Cä4D4;CT9
    services.AddScoped<UiModuleService>();

    // A'>C48D >C$0C08CR >C =C @D46CT=C0KDR ?D4=C=BCä2 CÄ5C0N
    IEnumerable<Type> uiModuleTypes = assemblies
        .SelectMany(a => a.GetTypes())
        .Where(t => typeof(IUiModule).IsAssignableFrom(t) && !t.IsInterface && !t.IsAbstract);

    foreach (Type type in uiModuleTypes)
    {
        try
        {
            if (Activator.CreateInstance(type) is IUiModule module)
            {
                Console.WriteLine($"[SCANNER] UI AÄ>CDCC'L: {module.Name}");
                List<(string Title, string Href, string Icon, string? Group)> menuItems =
module.GetMenuItems()?.ToList() ?? new();

                if (!menuItems.Any())
                {
                    Console.WriteLine("          % % [AôCD BCä5 CÄ5C0N]");
                    continue;
                }

                IEnumerable<(string Title, string Href, string Icon, string? Group)> rootItems
= menuItems.Where(i => string.IsNullOrEmpty(i.Group));
                IEnumerable<IGrouping<string?, (string Title, string Href, string Icon, string?
Group)>> groupedItems = menuItems.Where(i => !string.IsNullOrEmpty(i.Group)).GroupBy(i => i.Group);

                foreach ((string Title, string Href, string Icon, string? Group) item in
rootItems)
                {
                    Console.WriteLine($"          % % {item.Title} -> {item.Href}");
                }
                foreach (IGrouping<string?, (string Title, string Href, string Icon, string?
Group)> group in groupedItems)
                {
                    Console.WriteLine($"          % % [{group.Key}]");
                    List<(string Title, string Href, string Icon, string? Group)> itemsInGroup
= group.ToList();

```

```

        for (int i = 0; i < itemsInGroup.Count; i++)
        {
            string prefix = (i == itemsInGroup.Count - 1) ? "          % % % " :
            Console.WriteLine($"{prefix} {itemsInGroup[i].Title} ->
{itemsInGroup[i].Href}");
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"[SCANNER] AäHC,,1Cð0 DtBCT=C,,0 CÄ5CÔN CD;Dò BC,,?C  ·G— Räæ ÖWÓ¢ ¶W,äÖW76 v
    }
}
Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8Dò T'Ô:Cä<Cô>CÔ5CÔBCä2 D4ACô5D,,=Câ 7C 2CT@D,,5CÔ0");
Console.WriteLine();
}
}
</file>

<file path="Configuration.Web/Promatis.Net.Configuration.Web.csproj">
<Project Sdk="Microsoft.NET.Sdk.Web">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
        "Ä÷WG WEG— SäÆ-'& '"Âô÷WG WEG— Sí
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="MudBlazor" Version="9.4.0" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Configuration\Promatis.Net.Configuration.csproj" />
        <ProjectReference Include="..\Promatis.Net.UI\Promatis.Net.UI.csproj" />
    </ItemGroup>
</Project>
</file>

<file path="Configuration.Web/Properties/launchSettings.json">
{
    "profiles": {
        "Promatis.Net.Configuration.Web": {
            "commandName": "Project",
            "launchBrowser": true,
            "environmentVariables": {
                "ASPNETCORE_ENVIRONMENT": "Development"
            },
            "applicationUrl": "https://localhost:56819;http://localhost:56820"
        }
    }
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/ComponentRegistry.cs">
using System.Reflection;
using Microsoft.AspNetCore.Components;
namespace Promatis.Net.Configuration.Web;
public class ComponentRegistry
{
    private readonly Dictionary<string, Type> _routeMap = new(StringComparer.OrdinalIgnoreCase);
    /// <summary>
    /// B :C =C,,@D45D" AC >D :C, <Cä4D4;CT9 C, AD$@Cä8D" :C @D$C: URL -> B Æ 72 "C,,? Cð>CÄ?Cä=CT=D$0D
    /// </summary>
    public void RegisterModules(IEnumerable<Assembly> assemblies)
    {
        foreach (Assembly assembly in assemblies)
        {

```

```

        IEnumerable<Type> componentTypes = assembly.GetTypes()
            .Where(t => typeof(IComponent).IsAssignableFrom(t) && !t.IsAbstract);

        foreach (Type type in componentTypes)
        {
            // A,,ICT< C BD 8C CD" µ&÷WFT GG&-'WFUÔÂ 2 C¤>D$>D KC' :Că<Cô8C'8D CCTBD 0 CD8D 5C¤BC,,2C
            IEnumerable<RouteAttribute> routeAttributes =
                type.GetCustomAttributes<RouteAttribute>();
            foreach (RouteAttribute attr in routeAttributes)
            {
                _routeMap[attr.Template] = type;
            }
        }
    }

    public Type? GetComponentTypeByRoute(string route)
    {
        return _routeMap.TryGetValue(route, out Type? type) ? type : null;
    }
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/TabItem.cs">
namespace Promatis.Net.Configuration.Web;
{
    public class TabItem
    {
        public string Id { get; init; } = string.Empty; // AÔ0D, ‡&Vm
        public string Title { get; init; } = string.Empty;
        public string Icon { get; init; } = string.Empty;
        public Type ComponentType { get; init; } = null!;
    }
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/TabNavigationService.cs">
namespace Promatis.Net.Configuration.Web;
{
    public class TabNavigationService
    {
        public List<TabItem> OpenTabs { get; } = new();
        public int ActiveTabIndex { get; set; }
        public event Action? OnTabsChanged;

        public void OpenTab(string title, string href, string icon, Type componentType)
        {
            TabItem? existingTab = OpenTabs.FirstOrDefault(t => t.Id.Equals(href,
                StringComparison.OrdinalIgnoreCase));

            if (existingTab != null)
            {
                ActiveTabIndex = OpenTabs.IndexOf(existingTab);
            }
            else
            {
                var newTab = new TabItem
                {
                    Id = href,
                    Title = title,
                    Icon = icon,
                    ComponentType = componentType
                };

                OpenTabs.Add(newTab);
                ActiveTabIndex = OpenTabs.Count - 1;
            }

            NotifyStateChanged();
        }

        /// <summary>
        /// A 5Ct>Cô0D =Că5 Ct0C¤@D'BC,,5 C$:C'0CD:C, ?Că 5E GC,,AC'>C$>CĂC C,,=CD5C¤AD=
        /// </summary>
        public void CloseTabByIndex(int index)
        {
            if (index < 0 || index >= OpenTabs.Count) return;

```

```

    OpenTabs.RemoveAt(index);
}

// A,,=D$5C';CT:D$CC ;DÄ=D'9 D 0D GCTB DD>C=CD 0 CÔ0 CäAD$0C$HC,,5D 0 C$:C'0CD:C•
if (ActiveTabIndex >= OpenTabs.Count)
{
    ActiveTabIndex = OpenTabs.Count - 1;
}

if (ActiveTabIndex < 0 && OpenTabs.Count > 0)
{
    ActiveTabIndex = 0;
}

NotifyStateChanged();
}

private void NotifyStateChanged() => OnTabsChanged?.Invoke();
}
</file>

<file path="Configuration.Web/Services/UiModuleService.cs">
using Promatis.Net.UI;
{
namespace Promatis.Net.Configuration.Web;
{
// B 5D 2C,,A, C= >D$>D KC' ACä1C,,@C 5D" 2D Q C$>CT4C,,=Cí
public class UiModuleService(IEnumerable<IUiModule> modules)
{
    public IEnumerable<IUiModule> Modules => modules;
}
</file>

<file path="Configuration.Web/UiCommandBus/IUiCommandBus.cs">
using MudBlazor;
{
namespace Promatis.Net.Configuration.Web;
{
/// <summary>
/// A,,=D$5D DCT9D HC,,=D² T'ÔACä1D'BC,,9D
/// </summary>
public interface IUiCommandBus
{
    /// <summary>
    /// B 5C48D BD 0Dd8D0 >C @C 1CäBDt8C=0 C$Kct>C$0 DD>D <D² ,,2D'?Cä;CÔ0CTBD 0 Cô@C,,=C,,<C ND"8CÄ <Cä4D4;CT<,
    /// </summary>
    void RegisterHandler(string commandName, Func<Dictionary<string, object>, Task<DialogResult?>>
handler);
{
    /// <summary>
    /// A$7C 8CÄ>CD5C"AD$2C,,5 CÄ5Cd4D2 <Cä4D4;Dô<C, 2D ;CT?D4N (C$KctKC$0CTBD 0 C,,7 MDM)
    /// </summary>
    Task<DialogResult?> ExecuteAsync(string commandName, Dictionary<string, object> parameters);
}
</file>

<file path="Configuration.Web/UiCommandBus/UiCommandBus.cs">
using MudBlazor;
{
namespace Promatis.Net.Configuration.Web;
{
/// <summary>
/// B,,8CÔ0 UI-D >C KD$8C•
/// </summary>
public class UiCommandBus : IUiCommandBus
{
    // Aô>D$>C= >C 5Ct>Cô0D =D'9 D ;Cä2C @DÄ 4C'0 D 5C48D BD 0Dd8C, :Cä<C =CB 2 D 0CÔBC 9CÄ5 SignalR
    private readonly System.Collections.Concurrent.ConcurrentDictionary<string,
Func<Dictionary<string, object>, Task<DialogResult?>>> _handlers = new();
{
    public void RegisterHandler(string commandName, Func<Dictionary<string, object>,
Task<DialogResult?>> handler)
    {
        _handlers[commandName] = handler ?? throw new ArgumentNullException(nameof(handler));
    }
}
}

```

```

        public async Task<DialogResult?> ExecuteAsync(string commandName, Dictionary<string, object>
parameters)ð
        {ð
            if (_handlers.TryGetValue(commandName, out Func<Dictionary<string, object>,
Task<DialogResult?>>? handler))ð
            {ð
                return await handler(parameters);ð
            }ð
        }ð
        // ATAC`8 CÄ>CDCC`L (C00C0@C,,<CT@, DCA) C05 C0>CD:C`NDt5C0 2 C`0D4=Dt5D 5, D 8D BCT<C =CR CC00CD5D"Ä
        return null;ð
    }ð
}
</file>

```

```

<file path="Configuration.Web/WebAppBootstrapper.cs">
using Microsoft.AspNetCore.Components;ð
ð
namespace Promatis.Net.Configuration.Web;ð
ð
// A00D ;CT4C08Cç &ö÷G7G& W"Ä :CäBCä@D`9 D4<CT5D" 7C ?D4ACð0D$L Web-D 5D 2CT@ð
public abstract class WebAppBootstrapper<TRootComponent> : AppBootstrapperð
    where TRootComponent : IComponentð
{ð
    public override void Run(string[] args)ð
    {ð
        // B =Cä2C 7C 3D CCd0CT< DLL (C00 D ;D4GC 9 C0@D0<Cä3Cä 7C ?D4ACð0)ð
        // LoadProjectAssemblies ("Promatis.") D46CR 2D`7C$0C00 C" & 6RÄ`VäÄ
        // C0> Ct4CTADÄ <D² 8D ?Cä;DÄ7D45CÄ vV$ Ä-6 F-öä`V-ÆFW"i
ð
        // Ää B0 A "AT BÄ Ääç CT7 D0BCä3Cä F0Ö -ä =CR CC$8CD8D" <Cä4D4;C, ?D 8 Type.GetTypeð
        LoadProjectAssemblies("Promatis.");ð
ð
        WebApplicationBuilder builder = WebApplicationBuilder.CreateBuilder(args);ð
        List<IAppConfigurator> configs = GetConfigurators(builder.Configuration).ToList();ð
ð
        foreach (IAppConfigurator cfg in configs)ð
            cfg.ConfigureServices(builder.Services, builder.Configuration);ð
ð
        WebApplication app = builder.Build();ð
ð
        // A00D BD >C":C Ö-FFÆWv &R <Cä4D4;CT9ð
        foreach (IWebAppConfigurator cfg in configs.OfType<IWebAppConfigurator>())ð
        {ð
            cfg.ConfigureMiddleware(app);ð
        }ð
ð
        foreach (IAppConfigurator cfg in configs)ð
            cfg.ConfigureApp(app);ð
ð
        app.Run();ð
    }ð
}
</file>

```

```

<file path="Configuration/AppBootstrapper.cs">
using Microsoft.Extensions.Configuration;ð
using Microsoft.Extensions.Hosting;ð
using System.Reflection;ð
using System.Runtime.Loader;ð
ð
namespace Promatis.Net.Configuration;ð
ð
public abstract class AppBootstrapperð
{ð
    public virtual void Run(string[] args)ð
    {ð
        // 1. A0 A,, B4 A,, "AT BÄ A / At A4 B4 Að DLL (DtBCä1D² G- RävWEG- R 8DR 2C,,4CT;)ð
        LoadProjectAssemblies("Promatis.");ð
ð
        HostApplicationBuilder builder = Host.CreateApplicationBuilder(args);ð
ð
        // 2. B4 A0+A' Ää B Að A0$A,, B4 A "Ää Ää A,, JSONð
        List<IAppConfigurator> configurators = GetConfigurators(builder.Configuration).ToList();ð
ð
        foreach (IAppConfigurator config in configurators)ð

```

```

        {
            config.ConfigureServices(builder.Services, builder.Configuration);
        }

        IHost host = builder.Build();

        foreach (IAppConfigurator config in configurators)
        {
            config.ConfigureApp(host);
        }

        host.Run();
    }

    protected virtual IEnumerable<IAppConfigurator> GetConfigurators(IConfiguration configuration)
    {
        string[] typeNames = configuration.GetSection("LauncherSettings:Modules").Get<string[]>()
            ?? [];

        foreach (string name in typeNames)
        {
            // A,,ICT< D$8Cò 2Câ 2D 5DR 7C 3D CCd5CÔ=D´E D 1Câ@C¤0D]
            Type? type = AppDomain.CurrentDomain.GetAssemblies()
                .Select(a => a.GetType(name))
                .FirstOrDefault(t => t != null);

            if (type != null && typeof(IAppConfigurator).IsAssignableFrom(type))
            {
                if (Activator.CreateInstance(type) is IAppConfigurator instance)
                {
                    yield return instance;
                }
            }
            else
            {
                Console.WriteLine($"[BOOTSTRAPPER][WARN] B$8Cò =CR =C 9CD5Cò 8C´8 CÔ5 C$0C´8CD5CÔ¢ ¶æ ÖWò""½
            }
        }
    }

    protected void LoadProjectAssemblies(string prefix)
    {
        string? path = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
        if (path == null) return;

        foreach (string file in Directory.GetFiles(path, $"{prefix}*.dll"))
        {
            try
            {
                AssemblyLoadContext.Default.LoadFromAssemblyPath(file);
            }
            catch { /* A,,3CÔ>D 8D CCT< CâHC,,1C¤8 Ct0C4@D47C¤8 */ }
        }
    }
}
</file>

<file path="Configuration/AppConfigurator.cs">
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using System.Text.Json.Serialization;
namespace Promatis.Net.Configuration;
public class AppConfigurator : IAppConfigurator
{
    public virtual void ConfigureServices(IServiceCollection services, IConfiguration configuration)
    {
        // B :C =C,,@D45CÂ 2D Q (C$:C´NDt0Dò VF-EG&-vvW" 2 Cò@Câ5C¤BCR F F •
        services.AddDomainInfrastructure(projectPrefix: "Promatis.");
    }
}

```

```

        // B 5C48D BD 8D CCT< C 0Ct>C$KCR ACT@C$8D KÐ
        services.AddScoped<DatabaseTriggerService>();Ð
        services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());Ð
        services.AddScoped<DatabaseTriggerInterceptor>();Ð
Ð
        // Aä Bd Af¢ ¥4ôâ =C AD$@Cä9C¤8Ð
        services.AddSingleton(new JsonSerializerOptionsÐ
        {Ð
            PropertyNamingPolicy = JsonNamingPolicy.CamelCase,Ð
            DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull,Ð
            WriteIndented = falseÐ
        });Ð
Ð
        // B A4 B "B Bd Bò A´ B$Aä AÄ AÔ Aä Aâ A´ AÔ B B #B" Aä!B$ A•
        services.AddSingleton<IEntityCloner, JsonEntityCloner>();Ð
    }Ð
Ð
    public virtual void ConfigureApp(IHost app)Ð
    {Ð
        if (app == null) throw new ArgumentNullException(nameof(app));Ð
Ð
        // A,,=C,,FC,,0C´8Ct8D CCT< C4;Cä1C ;DÄ=D´9 Service Locator Cò@C, AD$0D BCR ?D 8C´>Cd5C08Dý
        AppInfrastructure.Initialize(app.Services);Ð
Ð
        using IServiceScope scope = app.Services.CreateScope();Ð
        scope.ServiceProvider.AutoRegisterTriggers();Ð
    }Ð
}
</file>

<file path="Configuration/AppInfrastructure.cs">
namespace Promatis.Net.Configuration;Ð
Ð
/// <summary>Ð
/// A4;Cä1C ;DÄ=C O D 8D BCT<C00Dò BCäGC¤0 CD>D BD4?C : C,,=DD@C AD$@D4:D$CD =D´< D 5D 2C,,AC < Cò;C BDD>D <D²
/// A,,=C,,FC,,0C´8Ct8D CCTBD O Cä4C0>C¤@C BC0> Cò@C, AD$0D BCR ?D 8C´>Cd5C08Dò 8 C,,AC¤;DäGC 5D" @C 7CDCC$0C08CR
/// </summary>Ð
public static class AppInfrastructureÐ
{Ð
    private static IServiceProvider? _provider;Ð
    private static readonly object _lock = new();Ð
Ð
    /// <summary>Ð
    /// Read-only CD>D BD4? C¢ 3C´>C 0C´LC0>CÄC C¤>C0BCT9C05D C Ct0C$8D 8CÄ>D BCT9.D
    /// </summary>Ð
    public static IServiceProvider ProviderÐ
    {Ð
        getÐ
        {Ð
            if (_provider == null)Ð
            {Ð
                throw new InvalidOperationException(Ð
                    "A¤@C,,BC,,GCTAC¤0Dò >D,,8C :C ¢ -æg& 7G'V7GW&R =CR 8C08Dd8C ;C,,7C,,@Cä2C =. " +Ð
                    "B41CT4C,,BCTADÄÄ GD$> CÄ5D$>CB -æg& 7G'V7GW&Rä-æ-F- Æ-¡R,' 2D´7C$0C0 2 Program.cs C,,;C
                )Ð
            }Ð
            return _provider;Ð
        }Ð
    }Ð
Ð
    /// <summary>Ð
    /// Aô>D$>C¤>C 5Ct>Cò0D =C O C,,=C,,FC,,0C´8Ct0Dd8Dò ;Cä:C BCä@C â D´7D´2C 5D$ADò AD$@Cä3Câ >CD8C0 @C 7 Cò@
    /// </summary>Ð
    public static void Initialize(IServiceProvider provider)Ð
    {Ð
        if (provider == null) throw new ArgumentNullException(nameof(provider));Ð
        if (_provider != null) return;Ð
Ð
        lock (_lock)Ð
        {Ð
            _provider ??= provider;Ð
        }Ð
    }Ð
}
</file>

```



```

<file path="Configuration/IAppConfigurator.cs">
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;

namespace Promatis.Net.Configuration;

public interface IAppConfigurator
{
    void ConfigureServices(IServiceCollection services, IConfiguration configuration);
    void ConfigureApp(IHost app);
}
</file>

```

```

<file path="Configuration/Promatis.Net.Configuration.csproj">
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  .
</PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.Configuration.Abstractions" Version="10.0.0" />
    <PackageReference Include="Microsoft.Extensions.DependencyInjection.Abstractions" Version="10.0.0" />
    <PackageReference Include="Microsoft.Extensions.Hosting.Abstractions" Version="10.0.0" />
    <PackageReference Include="Promatis.Net.Data" Version="1.0.0" />
    <PackageReference Include="Promatis.Net.Domain" Version="1.0.0" />
    <PackageReference Include="Promatis.Net.Service" Version="1.0.0" />
    <PackageReference Include="System.Reflection" Version="4.7.0" />
    <PackageReference Include="System.Runtime.Loader" Version="4.7.0" />
  </ItemGroup>
</Project>
</file>

```

```

<file path="Configuration/ServiceScanningExtensions.cs">
using FluentValidation;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using System.Reflection;
using System.Runtime.Loader;

namespace Promatis.Net.Configuration;

public static class ServiceScanningExtensions
{
    public static void AddDomainInfrastructure(this IServiceCollection services, string projectPrefix = "Promatis.")
    {
        Console.WriteLine();
        Console.WriteLine("[SCANNER] At0C0CD : C 2D$>CÄ0D$8Dt5D :Cä9 D 5C48D BD 0Dd8C, 2 DI...");

        // 1. A0 A,, B4 A,, "AT BÄ A / At A4 B4 A B Aä Aä
        string? path = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
        if (path != null)
        {
            foreach (string file in Directory.GetFiles(path, $"{projectPrefix}*.dll"))
            {
                try
                {
                    AssemblyLoadContext.Default.LoadFromAssemblyPath(file);
                }
                catch (Exception ex)
                {
                }
            }
        }
    }
}

```

```

        string fileName = Path.GetFileName(file);
        Console.ForegroundColor = ConsoleColor.Red;
        Console.WriteLine($"[SCANNER][ERROR] A05 D44C ;CäADÂ 7C 3D Cct8D$L D 1Cä@C C {fileName}");
        Console.ResetColor();
    }
}

// A 5D 5CÂ 2D 5 Ct0C4@D46CT=C0KCR AC >D :C, A C$0D,,8CÂ AC,,AD$5CÄ=D´< C0@CTDC,,:D >Cí
Assembly[] assemblies = AppDomain.CurrentDomain.GetAssemblies()
    .Where(a => a.FullName != null && a.FullName.StartsWith(projectPrefix))
    .Distinct()
    .ToArray();

int countBefore = services.Count;

// 2. A B$ AÄ B$ Bt B Aä B A A,, Aä A A,, A, AT A,,!B$ A &A,,/ Bt B Ar 45%UDö-
services.Scan(scan =>
{
    // A´>C=0C´LC00D0 DD4=C=FC,,0 CD;D0 @CT:D4@D 8C$=Cä9 C0@Cä2CT@C=8 C$ACT9 Dd5C0>Dt:C, =C AC´5CD>C$0
    bool IsSubclassOfRawGeneric(Type generic, Type? toCheck)
    {
        while (toCheck != null && toCheck != typeof(object))
        {
            Type cur = toCheck.IsGenericType ? toCheck.GetGenericTypeDefinition() : toCheck;
            if (generic == cur) return true;
            toCheck = toCheck.BaseType;
        }
        return false;
    }

    scan.FromAssemblies(assemblies)
        // A,,!A0 A A´ A0 : B 5C48D BD 0Dd8D0 !AT A$ B A" A C0>CD4CT@Cd:Cä9 C4;D41Cä:Cä3Cä =C AC´5CD
        .AddClasses(c => classes.Where(type =>
            !type.IsAbstract &&
            !type.IsGenericTypeDefinition &&
            IsSubclassOfRawGeneric(typeof(BaseService<,,>), type)))
        .AsSelf()
        .AsImplementedInterfaces()
        .WithScopedLifetime()

        // B 5C48D BD 0Dd8D0 A A,, A "Aä Aä (FluentValidation)
        .AddClasses(c => c.AssignableTo(typeof(IValidator<>))
            .Where(t => !t.IsAbstract && !t.IsGenericTypeDefinition))
        .AsImplementedInterfaces()
        .WithTransientLifetime()

        // B 5C48D BD 0Dd8D0 "B A4 AT Aä (Before/After Save)
        .AddClasses(c => c.AssignableToAny(typeof(IBeforeSaveTrigger<>),
            typeof(IAfterSaveTrigger<>)).Where(t => !t.IsAbstract))
        .AsSelfWithInterfaces()
        .WithScopedLifetime();

});

// --- B A4 B "B Bd B0 A´ A A´,A0 A4 A0 A´ AÄ B $A0 A4 A$ A´ AD B$ B B0 B ---
// B 5C48D BD 8D CCT< C=0Cf >D$:D KD$K´ 4Cd5C05D 8Cfä "CT?CT@DÄ ?D 8 Ct0C0@CäACR •f Æ-F F÷#Ä Dä1Cä9A
// DI-C= C0BCT9C05D ääUB 3C @C =D$8D >C$0C0=Cä >D$4C AD" =C H C0>C´8CÄ>D DC0KC´ 4C,,AC05D$GCT@D
services.AddScoped(typeof(IValidator<>), typeof(GlobalPolymorphicValidator<>));

// 3. A,,!A0 A A´ A0 Aä A, A AT A0 AR Aä A,, Aä A A,, B At#A´,B$ B$ A-
List<ServiceDescriptor> newRegistrations = services.Skip(countBefore).ToList();
var loggedTypes = new HashSet<string>();

foreach (ServiceDescriptor reg in newRegistrations)
{
    // A 5D 5CÂ @CT0C´LC0KC´ BC,,? D 5C ;C,,7C FC,,8 (C" ?D 8Cä@C,,BCTBCR 8Cr -x ÆVÖVçF F-öâG- RÄ 8C00Dt5
    Type? implType = reg.ImplementationType ?? reg.ImplementationInstance?.GetType();

    if (implType == null) continue;

    // --- A A A,, A$ A´ AD B$ B A" 00Ý
    bool isValidator = reg.ServiceType.IsGenericType &&
        reg.ServiceType.GetGenericTypeDefinition() == typeof(IValidator<>);

    if (isValidator && loggedTypes.Add($"Val:{implType.FullName}"))
    {

```

```

        var inheritanceChain = new List<string>();
        Type? currentType = implType;

        while (currentType != null && currentType != typeof(object))
        {
            string typeName = currentType.Name.Contains('.') ? currentType.Name.Split('.')[0] : currentType.Name;
            inheritanceChain.Add(typeName);
            currentType = currentType.BaseType;
            if (typeName == "AbstractValidator") break;
        }

        string chainDisplay = string.Join(" -> ", inheritanceChain);
        Console.WriteLine($"[SCANNER] A$0C'8CD0D$>D $ ¶6† -äF-7 Æ -Ö $ •f Æ-F F÷""½
        continue; // A'>C2 4C'0 D0BCä9 Ct0C08D 8 C$Kc$5CD5C0Â 8CD5CÂ : D ;CT4D4ND"5C•
    }

    // --- A A A,, B B A,,!Aä (A00D ;CT4C08C>C" & 6U6W'f-6R' 00Ý
    // A,,!A0 A A' A0 : B41D 0C0> Cd5D BC>CR CD ;Cä2C,,5 reg.ServiceType == reg.ImplementationType,
    // D$0C$ :C : D 5D 2C,,AD² BCT?CT@DÂ @CT3C,,AD$@C,,@D4ND$AD0 ?C @C <C, C'0D A + A,,=D$5D DCT9D
    bool isService = false;
    var serviceChain = new List<string>();
    Type? currentServiceType = implType;

    while (currentServiceType != null && currentServiceType != typeof(object))
    {
        string typeName = currentServiceType.Name.Contains('.') ?
        currentServiceType.Name.Split('.')[0] : currentServiceType.Name;
        serviceChain.Add(typeName);

        if (currentServiceType.IsGenericType &&
        currentServiceType.GetGenericTypeDefinition() == typeof(BaseService<,,>))
        {
            isService = true;
            break;
        }
        currentServiceType = currentServiceType.BaseType;
    }

    if (isService && loggedTypes.Add($"Srv:{implType.FullName}"))
    {
        string chainDisplay = string.Join(" -> ", serviceChain);
        Console.WriteLine($"[SCANNER] B 5D 2C,,A: {chainDisplay}");
        continue;
    }

    // --- A A A,, B$ A,, A4 B A" 00Ý
    bool isTrigger = implType.GetInterfaces().Any(i => i.IsGenericType &&
        (i.GetGenericTypeDefinition() == typeof(IBeforeSaveTrigger<>) ||
        i.GetGenericTypeDefinition() == typeof(IAfterSaveTrigger<>)));

    if (isTrigger && loggedTypes.Add($"Tri:{implType.FullName}"))
    {
        Console.WriteLine($"[SCANNER] B$@C,,3C45D $ ¶-x ÄG- Räæ ÖW0""½
    }
}

Console.WriteLine("[SCANNER] A 2D$>CÄ0D$8Dt5D :C 0 D 5C48D BD 0Dd8D0 CD ?CTHC0> Ct0C$5D HCT=C ä""½
Console.WriteLine();
}

public static void AutoRegisterTriggers(this IServiceProvider serviceProvider)
{
    Console.WriteLine("[AUTOREG] At0C0CD : C 2D$>CÄ0D$8Dt5D :Cä9 D 5C48D BD 0Dd8C, BD 8C43CT@Cä2 C" F F &

    var triggerService = serviceProvider.GetRequiredService<IDatabaseTriggerService>();

    // 1. A0>C,,AC$ 2D 5DR :C'0D ACä2 D$@C,,3C45D >C" 2 D 1Cä@C0DR &öÖ F-2ä-
    IEnumerable<Type> triggerTypes = AppDomain.CurrentDomain.GetAssemblies()
        .Where(a => a.FullName?.StartsWith("Promatis.") == true)
        .SelectMany(s => s.GetTypes())
        .Where(t => t is { IsClass: true, IsAbstract: false } &&
            t.GetInterfaces().Any(i => i.IsGenericType &&
                (i.GetGenericTypeDefinition() ==
                typeof(IBeforeSaveTrigger<>) ||
                typeof(IAfterSaveTrigger<>)))));

```

```

        .ToList();
    }

    MethodInfo? registerMethod =
    typeof(IDatabaseTriggerService).GetMethod(nameof(IDatabaseTriggerService.Register));

    int totalBindings = 0;

    foreach (Type triggerType in triggerTypes)
    {
        // A00DT>CD8CÂ 2D 5 C,,=D$5D DCT9D K D$@C,,3C45D >C"Â :CäBCä@D´5 D 5C ;C,,7D45D" 4C =C0KC´ :C´0D AB
        IEnumerable<Type> interfaces = triggerType.GetInterfaces()
            .Where(i => i.IsGenericType &&
                (i.GetGenericTypeDefinition() == typeof(IBeforeSaveTrigger<>) ||
                 i.GetGenericTypeDefinition() == typeof(IAfterSaveTrigger<>)));

        foreach (Type @interface in interfaces)
        {
            // A0>C´CDt0CT< D$8C0 AD4IC0>D BC, ...B 8Cr "&Vf÷&U6 fUG&-vvW#ÂCâ•
            Type entityType = @interface.GetGenericArguments()[0];

            // Aä?D 5CD5C´OCT< D$8C0 BD 8C43CT@C 4C´O C@C AC,,2Cä3Cä ;Cä3C
            string triggerKind = @interface.GetGenericTypeDefinition() ==
            typeof(IBeforeSaveTrigger<>)
                ? "Before"
                : "After ";

            try
            {
                // A$KCTKC$0CT< Register<TEntity, TTrigger>()
                MethodInfo genericRegister = registerMethod!.MakeGenericMethod(entityType,
                triggerType);

                genericRegister.Invoke(triggerService, null);

                // A´ A4 B A$ A0 AR #B AT(A0 A´ B A$/At A•
                Console.WriteLine($"[AUTOREG] [{triggerKind}] {entityType.Name} <---
                {triggerType.Name}");
                totalBindings++;
            }
            catch (Exception ex)
            {
                Console.ForegroundColor = ConsoleColor.Red;
                Console.WriteLine($"[AUTOREG][ERROR] AäHC,,1C@0 C0@C,,2D07C@8 {triggerType.Name} C¢ ¶VçF—G•
                {ex.InnerException?.Message ?? ex.Message}");
                Console.ResetColor();
            }
        }
    }

    Console.WriteLine($"[AUTOREG] At0C$5D HCT=Cââ !Cä7CD0C0> C0@C,,2D07Cä:: {totalBindings}");
    Console.WriteLine();
}
</file>

<file path="Data/Configurations/AuditLogConfiguration.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.Domain;

namespace Promatis.Net.Data;

public class AuditLogConfiguration : IEntityTypeConfiguration<AuditLog>
{
    public void Configure(EntityTypeBuilder<AuditLog> builder)
    {
        builder.Property(e => e.EntityName).HasMaxLength(100);
        builder.Property(e => e.Action).HasMaxLength(50);
        builder.Property(e => e.ChangedBy).HasMaxLength(100);

        // ChangesJson CäAD$0C$;D05CÂ 1CT7 C´8CÄ8D$0 (text/jsonb), D$0C¢ :C : D ?C,,ACä: C,,7CÄ5C05C08C´ <Cä6CT
        builder.Property(e => e.ChangesJson).HasColumnType("jsonb");
    }
}
</file>

<file path="Data/Configurations/ModelBuilderConventions.cs">

```

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata;
using System.Linq.Expressions;
using Promatis.Net.Domain.Interface;

namespace Promatis.Net.Data;

public static class ModelBuilderConventions
{
    public static void ApplyGlobalConventions(this ModelBuilder modelBuilder)
    {
        foreach (ImmutableEntityType entityType in modelBuilder.Model.GetEntityTypes())
        {
            Type type = entityType.ClrType;

            // A`8CÄ8D$K CD;Dò AD$@Cä: (255 Cõ> D4<Cä;Dt0C08Dâ•
            foreach (ImmutableProperty property in entityType.GetProperties())
            {
                if (property.ClrType == typeof(string))
                {
                    if (property.GetMaxLength() == null)
                        property.SetMaxLength(255);
                }
            }

            // A 2D$>-Cä;DäGC, 4C`O Guid (ValueGeneratedNever, D$0C¢ :C : Id D >Ct4C 5D$ADò 2 DomainObject)
            if (typeof(IDomainObjectHasKey<Guid>).IsAssignableFrom(type))
            {
                modelBuilder.Entity(type).Property("Id").ValueGeneratedNever();
            }

            // A B$ -BD A´,B$ C, A$"Aâ Ô Â AT B 4C`O SoftDelete Cä1D=5C=BCä2
            if (typeof(ISoftDeletable).IsAssignableFrom(type))
            {
                ImmutableEntityType? baseType = entityType.BaseType;

                // B BC 2C„< DD8C´LD$@ D$>C´LCª> C00 Cª>D 5C0L C„5D 0D EC„8 (C$:C´NDt0Dò 0C AD$@C :D$=D´9 Uni
                if (baseType == null || !typeof(ISoftDeletable).IsAssignableFrom(baseType.ClrType))
                {
                    // B4AD$0C0>C$:C VW'f-ÇFW-
                    modelBuilder.SetSoftDeleteFilter(type);
                }

                // B4AD$0C0>C$:C 8C04CT:D 0 C00 DeletedAt CD;Dò CD :Cä@CT=C„0 Ct0C0@CäACä2
                modelBuilder.Entity(type).HasIndex("DeletedAt");
            }
        }
    }

    private static void SetSoftDeleteFilter(this ModelBuilder modelBuilder, Type entityType)
    {
        // AD;Dò E BõE , 2C 6C0> D BC 2C„BDÂ DC„;DÄBD =C :Cä@CT=DÂÂ 4C 6CR 5D ;C, >C0 0C AD$@C :D$=D´9.D
        // B4AC´>C$8CR -46Æ 72 4CäAD$0D$>Dt=Câí
        if (entityType.IsClass)
        {
            modelBuilder.Entity(entityType).HasQueryFilter(GenerateFilter(entityType));
        }
    }

    private static LambdaExpression GenerateFilter(Type type)
    {
        ParameterExpression parameter = Expression.Parameter(type, "e");
        // A„AC0>C´LCtCCT< Property CD;Dò 4CäAD$CC00 C¢ FVÆWFVD B GCT@CT7 C„=D$5D DCT9D 8C´8 D 2Cä9D BC$>D
        MemberExpression property = Expression.Property(parameter,
            nameof(ISoftDeletable.DeletedAt));
        ConstantExpression nullConstant = Expression.Constant(null, typeof(DateTime?));
        BinaryExpression body = Expression.Equal(property, nullConstant);

        return Expression.Lambda(body, parameter);
    }
}
</file>

<file path="Data/DBContext/ApplicationDbContext.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;

```

```

using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain;
namespace Promatis.Net.Data;
public class ApplicationDbContext(
    DbContextOptions options,
    IConfiguration configuration,
    IServiceProvider? serviceProvider = null)
    : DbContext(options)
{
    protected virtual string Schema => "public";
    public DbSet<AuditLog> AuditLogs => Set<AuditLog>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.HasDefaultSchema(Schema);
        modelBuilder.ApplyModuleConfigurations(this);
        modelBuilder.ApplyGlobalConventions();
        base.OnModelCreating(modelBuilder);
    }

    // A,,!A A A' A B' A, A AT A+A' A A,, A"
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        base.OnConfiguring(optionsBuilder);

        // 1. A@Cä2CT@Dô5CÂÂ GD$> CÄK C" @C =D$0C"<CRÂ 0 CÔ5 C" ?D >Dd5D ACR <C,,3D 0Dd8C, 4Ä•
        if (!EF.IsDesignTime && serviceProvider != null)
        {
            DatabaseTriggerInterceptor? interceptor = null;

            try
            {
                // Aô>CôKD$:C  D >C CCT< D 0Ct@CTHC,,BDÂ =C ?D 0CÄCDâ „5D ;C, ?D >C$0C"4CT@ Cä:C 7C ;D O S
                interceptor = serviceProvider.GetService(typeof(DatabaseTriggerInterceptor)) as
                DatabaseTriggerInterceptor;
            }
            catch (InvalidOperationException)
            {
                // Aô>CôKD$:C #D D ;C, ?D >C$0C"4CT@ C>D =CT2Cä9 (Root), D >Ct4C 5CÂ 66÷ R 2D Cdt=D4N!D
                // BÔBCâ 3C @C =D$8D CCTB, DtBCâ 66÷ VBÔ8CÔBCT@Dd5CôBCä@ D 0Ct@CTHC,,BD O C 5Cr >D,,8C >C  > Ro
                var scopeFactory = serviceProvider.GetService(typeof(IServiceScopeFactory)) as
                IServiceScopeFactory;

                if (scopeFactory != null)
                {
                    // B >Ct4C 5CÂ 2D 5CÄ5CÔ=D4N Cä1C'0D BDÂ 2C,,4C,,<CäAD$8D
                    using IServiceScope scope = scopeFactory.CreateScope();
                    interceptor =
                    scope.ServiceProvider.GetService(typeof(DatabaseTriggerInterceptor)) as DatabaseTriggerInterceptor;
                }
            }

            // ATAC'8 C,,=D$5D FCT?D$>D CD ?CTHCÔ> CÔ0C"4CT= Cä4CÔ8CÂ 8Cr ACô>D >C >C" r ?Cä4C;DäGC 5CÂ 5C4>
            if (interceptor != null)
            {
                optionsBuilder.AddInterceptors(interceptor);
            }
        }
    }
}
</file>

<file path="Data/DbContext/ModelBuilderExtensions.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain.Interface;
using System.Reflection;
using Microsoft.EntityFrameworkCore.Metadata;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
namespace Promatis.Net.Data;
public static class ModelBuilderExtensions
{
    public static void ApplyModuleConfigurations(this ModelBuilder modelBuilder, DbContext context)

```

```

{
    // 1. B :C =C,,@D45CÂ BCä;DÄ:Câ AC >D :C, 4C =CÔKDR ?D >CT:D$0 Promatis.*.Data
    List<Assembly> dataAssemblies = AppDomain.CurrentDomain.GetAssemblies()
        .Where(a => {
            string? name = a.GetName().Name;
            return name != null &&
                name.StartsWith("Promatis.") &&
                name.EndsWith(".Data");
        })
        .ToList();

    // 2. A0>C`CDt0CT< D$8C0K D CD"=CäAD$5C'Â :CäBCä@D`5 D02C0> Cä1D0C$;CT=D² 2 D$5C0CD"5CÂ :Cä=D$5C0AD$
    HashSet<Type> contextDbSetTypes = context.GetType()
        .GetProperties(BindingFlags.Public | BindingFlags.Instance)
        .Where(p => p.PropertyType.IsGenericType && p.PropertyType.GetGenericTypeDefinition()
            == typeof(DbSet<>))
        .Select(p => p.PropertyType.GetGenericArguments()[0])
        .ToHashSet();

    foreach (Assembly assembly in dataAssemblies)
    {
        // A0@C,,<CT=D05CÂ :Cä=DD8C4CD 0Dd8C, BCä;DÄ:Câ 4C`O D$5DR BC,,?Cä2, C0>D$>D KCR <C BCT@C,,0C`LC0K (
        modelBuilder.ApplyConfigurationsFromAssembly(assembly, type =>
        {
            if (type is not { IsClass: true, IsAbstract: false, IsGenericTypeDefinition:
false })
                return false;

            // A00DT>CD8CÂ 8C0BCT@DD5C"A C0>C0DC,,3D4@C FC,,8 C, BC,,? D CD"=CäAD$8, C0>D$>D CDä >C00 C00D B
            Type? configInterface = type.GetInterfaces()
                .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(IEntityTypeConfiguration<>));

            if (configInterface == null) return false;

            Type entityType = configInterface.GetGenericArguments()[0];

            // A0 A,, "A,, 'AT!A0 A' $A,, BÄ"B 0 D ;C, =C AD$@C 8C$0CT<C 0 D CD"=CäAD$L C 1D BD 0C0BC00,
            // C0@Cä2CT@D05CÂÂ 5D BDÄ ;C, C C05E ECäBDÄ >CD8C0 6C,,2Cä9 C00D ;CT4C08C0 2 D$5C0CD"5CÂ F%6W
            if (entityType.IsAbstract)
            {
                bool hasConcreteDerivedInDbSet = contextDbSetTypes.Any(t =>
t.IsSubclassOf(entityType));
                if (!hasConcreteDerivedInDbSet) return false; // A0@Cä?D4AC00CT< C0>C0DC,,3D4@C FC,,N C 1D

                return type.GetConstructor(Type.EmptyTypes) != null;
            }
        });
    }

    // 3. A B$ -A,, A0 B B A$ A0 AR ` B B B$ A$(A,,%» A B "B A0&A,,
    IEnumerable<Type> allAbstractEntities = dataAssemblies
        .SelectMany(a => a.GetTypes())
        .Where(t => t is { IsClass: true, IsAbstract: true } && !t.IsGenericTypeDefinition);

    foreach (Type abstractType in allAbstractEntities)
    {
        bool isUsedInCurrentContext = contextDbSetTypes.Any(t => t == abstractType ||
t.IsSubclassOf(abstractType));

        if (!isUsedInCurrentContext)
        {
            modelBuilder.Ignore(abstractType);
        }
    }

    // 4. A B$ AÄ B$ Bt B A / A0 B "B A" A At A` B A$ A0 B`% AD B A$,AT A, A0 AT B A-
    foreach (IMutableEntityType entityType in modelBuilder.Model.GetEntityTypes())
    {
        Type? clrType = entityType.ClrType;
        if (clrType == null) continue;

        // A,,ICT< D 5C ;C,,7C FC,,N C,,=D$5D DCT9D 0 ITreeNode<>
        Type? treeInterface = clrType.GetInterfaces()
            .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(ITreeNode<>));
    }
}

```

```

        if (treeInterface != null){
            {
                Type targetTreeType = treeInterface.GetGenericArguments()[0];

                // A00D BD 0C,,2C 5CÂ BCä;DÄ:Cä =C 1C 7Cä2Cä< D4@Cä2C05 Cä1D=0C$;CT=C,,0 D 2Cä9D BC" „GD$>C K
                if (clrType == targetTreeType){
                    {
                        EntityTypeBuilder builder = modelBuilder.Entity(clrType);

                        // 1. A00D BD >C":C AC$0Ct8 B >CD8D$5C`L-A0>D$>CÄ>C-
                        builder.HasOne("Parent")
                            .WithMany("Children")
                            .HasForeignKey("ParentId")
                            .OnDelete(DeleteBehavior.Restrict);

                        // 2. A B$ -A,, AD A=: B >Ct4C 5CÂ 8C04CT:D 4C`O C$=CTHC05C4> C;DäGC &VçD-M
                        // B0BCä CD :Cä@C,,B C0>C,,AC C MC`5CÄ5C0BCä2 C0> ParentId C0@C, @C 1CäBCR "Æöö·W 2 D 5D 2C
                        builder.HasIndex("ParentId")
                            .HasDatabaseName($"IX_{clrType.Name}_ParentId");
                    }
                }
            }
        }
    }
</file>

```

```

<file path="Data/Interceptors/ChangeEntryModel.cs">
namespace Promatis.Net.Data;
{
    /// <summary>
    /// AÄ>CD5C`L DD8C=AC FC,,8 C,,7CÄ5C05C08C' „BD 0C0AC0>D BC0KC' >C JCT:D"'Ä ?D 5CDAD$0C$;D0ND"0D0 AC08CÄ>C C ACä
    /// CD>CÄ5C0=Cä9 D CD"=CäAD$8 C" <Cä<CT=D" 5E 4Cä1C 2C`5C08D0Ä <Cä4C,,DC,,C FC,,8 C,,;C, CCD0C`5C08D0i
    /// </summary>
    internal class ChangeEntryModel
    {
        /// <summary>
        /// A,,7CÄ5C00CT<D`9 D0:Ct5CÄ?C`OD 4Cä<CT=C0>C4> Cä1D=5C=BC „AD4IC0>D BC,'i
        /// A0@C,,2Cä4C,,BD 0 C C :Cä=C@CTBC0>CÄC D$8C0C C$=D4BD 8 D$@C,,3C45D >C"i
        /// </summary>
        public object Entity { get; init; } = null!;

        /// <summary>
        /// B$5C=CD"8C' AD$0D$CD 8Ct<CT=CT=C,,0 D CD"=CäAD$8 C" AC,,AD$5CÄ5 (Added, Modified, Deleted, SoftDeleted
        /// </summary>
        public EntityStateChangeEnum State { get; init; }

        /// <summary>
        /// A>C`CT:Dd8D0 BCäGCTGC0KDR 8Ct<CT=CT=C,,9 D 2Cä9D BC" „?Cä;CT9) D CD"=CäAD$8.D
        /// At0C0>C`=D05D$AD0 4CT;DÄBCä9 Ct=C GCT=C,,9 "B BC @Cä5 -> A0>C$>CR"ä C`O C0>C$KDR AD4IC0>D BCT9 D >CD5
        /// </summary>
        public List<PropertyChangeInfo> Changes { get; init; } = [];

        /// <summary>
        /// A,,4CT=D$8DD8C=0D$>D „8CÄ0 C,,;C, ;Cä3C,,) C0>C`Lct>C$0D$5C`O, D >C$5D HC,,2D,,5C4> CD0C0=Cä5 CD5C"AD$2C
        /// </summary>
        public string? ChangedBy { get; init; }

        /// <summary>
        /// AD0D$0 C, 2D 5CÄ0 DD8C=AC FC,,8 C,,7CÄ5C05C08D0 2 DD>D <C BCR UD2i
        /// </summary>
        public DateTime ChangedAt { get; init; }
    }
}
</file>

```

```

<file path="Data/Interceptors/DatabaseTriggerInterceptor.cs">
using System.Collections.Concurrent;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.ChangeTracking;
using Microsoft.EntityFrameworkCore.Diagnostics;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain.Interface;
{
    namespace Promatis.Net.Data;
    {
        /// <summary>
        /// A05D 5DT2C Bdt8C C05D 0Dd8C' F$6öçFW+BÄ >C 5D ?CTGC,,2C ND"8C' @C 1CäBD2 $<D03C= >C4> D44C ;CT=C,,0" (Soft

```



```

/// C0@CT4C$0D 8D$5C'LC0CDâ 2C ;C,,4C FC,,N C,,7CÄ5C05C08C' 8 D :C$>Ct=Cä5 D$@C,,3C45D =Cä5 D42CT4Cä<C'5C08CR AC,,
/// </summary>D
/// <param name="scopeFactory">BD0C @C,,:C 4C'0 D >Ct4C =C,,0 C,,7Cä;C,,@Cä2C =C0KDR >C ;C AD$5C' 2C,,4C,,<CäAD$8
public class DatabaseTriggerInterceptor(IServiceScopeFactory scopeFactory) : SaveChangesInterceptorD
{D
    /// <summary>D
    /// A0>D$>C>C 5Ct>C00D =Cä5 DT@C =C,,;C,,ICR 7C DC,,:D 8D >C$0C0=D'E C,,7CÄ5C05C08C' 2 D 0CÄ:C E D$5C=C'D"5C'
    /// A;DäG - D4=C,,:C ;DÄ=D'9 C,,4CT=D$8DD8C=0D$>D MC=7CT<C0;D0@C F$60çFW†B1
    /// </summary>D
    private readonly ConcurrentDictionary<Guid, List<ChangeEntryModel>> _capturedChanges = new();D
D
    /// <summary>D
    /// A AC,,=DT@Cä=C0KC' ?CT@CTEC$0D$GC,,:, C$KC0>C'=D05CÄKC' Aä DC :D$8Dt5D :Cä3Cä ACäED 0C05C08D0 8Ct<CT=C
    /// AäBC$5Dt0CTB Ct0 C'>C48C=C Soft Delete, DD8C=AC FC,,N D =C,,<C=0 C,,7CÄ5C05C08C' 8 Ct0C0CD : D$@C,,3C45D
    /// </summary>D
    public override async ValueTask<InterceptionResult<int>> SavingChangesAsync(D
        DbContextEventData eventData, InterceptionResult<int> result, CancellationTokens ct =
default)D
    {D
        if (eventData.Context == null) return result;D
D
        // B >Ct4C 5CÄ ;Cä:C ;DÄ=D'9 C40D 0C0BC,,@Cä2C =C0> Cd8C$>C' 66÷ R 4C'0 C0@CT4CäBC$@C ICT=C,,0 C0@Cä1C'
        using IServiceScope scope = scopeFactory.CreateScope();D
        var triggerService = scope.ServiceProvider.GetRequiredService<IDatabaseTriggerService>();D
D
        string? userName = await GetUserNameInternalAsync(scope.ServiceProvider);D
D
        // 1. A0@CT4C$0D 8D$5C'LC00D0 >C @C 1CäBC=0 Soft Delete (AÄ0C4:Cä5 D44C ;CT=C,,5)D
        // A00DT>CD8CÄ 2D 5 D CD"=CäAD$8 D 8C0BCT@DD5C"ACä< ISoftDeletable, D2 :CäBCä@D'E D BC BD4A "B44C ;C
        IEnumerable<EntityEntry<ISoftDeletable>> entries =
eventData.Context.ChangeTracker.Entries<ISoftDeletable>()D
        .Where(e => e.State == EntityState.Deleted);D
D
        foreach (EntityEntry<ISoftDeletable> entry in entries)D
        {D
            // A0>CD<CT=D05CÄ DC,,7C,,GCTAC=CR CCD0C'5C08CR =C >C =Cä2C'5C08CR 4C =C0KDR 2 A D
            entry.State = EntityState.Modified;D
            entry.Entity.DeletedAt = DateTime.UtcNow;D
            entry.Entity.DeletedBy = userName;D
        }D
D
        // A0@C,,=D44C,,BCT;DÄ=Cä 7C AD$0C$;D05CÄ Tb 6÷&R ?CT@CTADt8D$0D$L CD5D 5C$> C,,7CÄ5C05C08C' ?CäAC'5 C0>
        eventData.Context.ChangeTracker.DetectChanges();D
D
        // 2. At0DT2C B C,,7CÄ5C05C08C' „ACäED 0C00CTB C00D$8C$=D'5 D$8C0K CD0C0=D'E object? CD;D0 BCTAD$>C".
        List<ChangeEntryModel> captured = CaptureChanges(eventData.Context, userName);D
D
        // A0@C,,2D07D'2C 5CÄ :Cä;C'5C=FC,,N C,,7CÄ5C05C08C' : C=0C0:D 5D$=Cä<D2 MC=7CT<C0;D0@D2 :Cä=D$5C=AD$0 C
        _capturedChanges[eventData.Context.ContextId.InstanceId] = captured;D
D
        // 3. A$0C'8CD0dd8D0 „&Vf÷&U6 fr' GCT@CT7 Cd8C$>C' G&-vvW%6W'f-6]
        // At0C0CD :C 5D" 4Cä<CT=C0KCR ?D 0C$8C'0 C, 2C ;C,,4C BCä@D² ?CT@CT4 D$5CÄÄ :C : CD0C0=D'5 C0>C00CDCD
        foreach (ChangeEntryModel item in captured)D
            await triggerService.ValidateAsync(item.Entity, item.State, item.Changes,
eventData.Context);D
D
        return result;D
    }D
D
    /// <summary>D
    /// A AC,,=DT@Cä=C0KC' ?CT@CTEC$0D$GC,,:, C$KC0>C'=D05CÄKC' Aä!A' D4AC05D,,=Cä3Cä ACäED 0C05C08D0 8Ct<CT=C
    /// AäBC$5Dt0CTB Ct0 D42CT4Cä<C'5C08CR ?Cä4C08D GC,,:Cä2 C, BD 8C43CT@Cä2 C0>D B-Cä1D 0C >D$:C, „DC 7C 6
    /// </summary>D
    public override async ValueTask<int> SavedChangesAsync(D
        SaveChangesCompletedEventData eventData, int result, CancellationTokens ct = default)D
    {D
        // A,,7C$;CT:C 5CÄ 7C EC$0Dt5C0=D'5 C,,7CÄ5C05C08D0 8 D @C 7D2 0D$>CÄ0D =Cä CCD0C'0CT< C,,E C,,7 D ;Cä2C
        if (eventData.Context != null &&
        _capturedChanges.TryRemove(eventData.Context.ContextId.InstanceId, out List<ChangeEntryModel>?
entries))D
        {D
            // B >Ct4C 5CÄ ;Cä:C ;DÄ=D'9 C40D 0C0BC,,@Cä2C =C0> Cd8C$>C' 66÷ R 4C'0 DD0Ctk Saved (C$KC0>C'=D05
            using IServiceScope scope = scopeFactory.CreateScope();D
            var triggerService =
scope.ServiceProvider.GetRequiredService<IDatabaseTriggerService>();D
D

```

```

        // A AC,,=DT@Cä=CÔ> D42CT4Cä<C´OCT< D 8D BCT<D2 >C 8Ct<CT=CT=C,,ODR ,,=C ?D 8CÄ5D Ä 4C´O Cä1CÔ>C$;C
        foreach (ChangeEntryModel item in entries)ð
            await triggerService.NotifyAsync(item.Entity, item.State, item.Changes,
item.ChangedBy, item.ChangedAt);ð
        }ð
        return result;ð
    }ð
ð
    /// <summary>ð
    /// A05D 5DT2C BDt8C¢ AC >D0 ?D 8 D >DT@C =CT=C,,8 C,,7CÄ5C05C08C´â C @C =D$8D CCTB CäGC,,AD$:D2 :D0HC 7C
    /// C0@CT4CäBC$@C IC 0 D4BCTGC¢8 C00CÄOD$8 C0@C, ?C 4CT=C,,8 D$@C =Ct0C¢FC,,9.ð
    /// </summary>ð
    public override Task SaveChangesFailedAsync(DbContextErrorEventData eventData,
CancellationToken ct = default)ð
    {ð
        if (eventData.Context != null)ð
            _capturedChanges.TryRemove(eventData.Context.ContextId.InstanceId, out _);ð
ð
        return base.SaveChangesFailedAsync(eventData, ct);ð
    }ð
ð
    /// <summary>ð
    /// A$=D4BD 5C0=C,,9 CÄ5D$>CB 0C00C´8Ct0 D$@CT:CT@C 8Ct<CT=CT=C,,9 EF Core. ð
    /// A$KDt8D ;D05D" 4CT;DÄBD2 ,,AD$0D KCR0=Cä2D´5 Ct=C GCT=C,,0 C0>C´5C´´ 8 DD>D <C,,@D45D" <Cä4CT;C, 8Ct<CT=
    /// </summary>ð
    /// <param name="context">B$5C¢CD"8C´ :Cä=D$5C¢AD" 1C 7D² 4C =C0KDRäÄ÷ & Óí
    /// <param name="userName">A,,<D0 ?Cä;DÄ7Cä2C BCT;D0Ä ACä2CT@D,,8C$HCT3Cä >C05D 0Dd8DäãÄ÷ & Óí
    /// <returns>B ?C,,ACä: D BD CC¢BD4@C,,@Cä2C =C0KDR <Cä4CT;CT9 C,,7CÄ5C05C08C´ Ç6VR 7&Vc0$6† ævTVÇG'"ÖöFVÄ"ó
    private List<ChangeEntryModel> CaptureChanges(DbContext context, string? userName)ð
    {ð
        // BD8C´LD$@D45CÄ BCä;DÄ:Cä AD4IC0>D BC, 2 D >D BCä0C08D0E AD>C 0C$;CT=C,,0, A,,7CÄ5C05C08D0 8C´8 B44C
        List<EntityEntry> entries = context.ChangeTracker.Entries()ð
            .Where(e => e.State is EntityState.Added or EntityState.Modified or EntityState.Deleted)ð
            .Where(e => !e.Entity.GetType().Name.Contains("AuditLog"))ð
            .ToList();ð
ð
        var result = new List<ChangeEntryModel>();ð
ð
        foreach (EntityEntry e in entries)ð
        {ð
            EntityStateChangeEnum state = MapState(e.State);ð
ð
            // B ?CTFC,,DC,,GC0KC´ <C ?C08C03 D >D BCä0C08D0 4C´O "CÄ0C4:Cä CCD0C´5C0=D´E" Ct0C08D 5C•
            if (e.Entity is ISoftDeletable soft)ð
            {ð
                PropertyEntry prop = e.Property(nameof(ISoftDeletable.DeletedAt));ð
                if (prop.IsModified && soft.DeletedAt != null) state =
EntityStateChangeEnum.SoftDeleted;ð
            }ð
ð
            List<PropertyChangeInfo> changes = new();ð
ð
            // B FCT=C @C,,9 1: B CD"=CäAD$L CD>C 0C$;CT=C â D 5 D$5C¢CD"8CR 7C00Dt5C08D0 AC$>C"AD$2 D GC,,BC
            if (state == EntityStateChangeEnum.Added)ð
            {ð
                changes = e.Propertiesð
                    .Select(p => new PropertyChangeInfoð
                    {ð
                        PropertyName = p.Metadata.Name,ð
                        OriginalValue = null,ð
                        CurrentValue = p.CurrentValueð
                    }).ToList();ð
            }ð
            // B FCT=C @C,,9 2: B CD"=CäAD$L C,,7CÄ5C05C00 C,,;C, <D03C¢> D44C ;CT=C â D´GC,,AC´OCT< D 0Ct=C,,FD2
            else if (state is EntityStateChangeEnum.Modified or EntityStateChangeEnum.SoftDeleted)ð
            {ð
                PropertyValues originalValues = e.OriginalValues;ð
ð
                foreach (PropertyEntry p in e.Properties.Where(p => p.IsModified))ð
                {ð
                    string propertyName = p.Metadata.Name;ð
                    object? originalValue = originalValues[propertyName];ð
                    object? currentValue = p.CurrentValue;ð
ð
                    // BD8C¢AC,,@D45CÄ 8Ct<CT=CT=C,,5 D$>C´LC¢> CTAC´8 Ct=C GCT=C,,0 CD5C"AD$2C,,BCT;DÄ=Cä @C 7C´
                    if (!Equals(originalValue, currentValue))ð

```

```

        {
            changes.Add(new PropertyChangedEventArgs
            {
                PropertyName = propertyName,
                OriginalValue = originalValue,
                CurrentValue = currentValue
            });
        }
    }

    // AD0Cd5 CTAC'8 Cα>C';CT:Dd8Dò BCäGCTGCÔKDR 8Ct<CT=CT=C,,9 Cò>C'5C' „6† ævW2' ?D4AD$0
    // C,,7-Ct0 D 0C0BC 9CÂ01C,,=CD8C03C &Æ |÷"Â <D² B B A$ Aâ 4Cä1C 2C'0CT< D CD"=CäAD$L C" AC08D
    // CTAC'8 DD0C @C,,:C 7C DC,,:D 8D >C$0C'0 D >D BCä0C08CR 0öF-f-VB -D$> C40D 0C0BC,,@D45D" R 4C
    if (state == EntityStateChangeEnum.Modified && changes.Count == 0)
    {
        // B >Ct4C 5CÂ DC,,:D$8C$=D4N Ct0C08D L C,,7CÄ5C05C08Dò 4C'0 CD>CÄ5C0=D'E D$@C,,3C45D >C"Â
        // DtBCä1D² =CR ;Cä<C BDÂ 2C ;C,,4C FC,,N, C0> D >DT@C =C,,BDÂ 8CÄ?D4;DÄA CD;Dò T•
        changes.Add(new PropertyChangedEventArgs
        {
            PropertyName = "Id",
            OriginalValue = e.Property("Id").OriginalValue,
            CurrentValue = e.Property("Id").CurrentValue
        });
    }

    result.Add(new ChangeEntryModel
    {
        Entity = e.Entity,
        State = state,
        ChangedBy = userName,
        ChangedAt = DateTime.UtcNow,
        Changes = changes
    });
}

return result;
}

/// <summary>
/// AÄ0C0?C,,=C2 2C0CD$@CT=C08DR ACäAD$>Dò=C,,9 EntityFramework State C$> C$=D4BD 5C0=C,,9 CD>CÄ5C0=D'9 C05D
/// </summary>
private EntityStateChangeEnum MapState(EntityState state) => state switch
{
    EntityState.Added => EntityStateChangeEnum.Added,
    EntityState.Deleted => EntityStateChangeEnum.Deleted,
    _ => EntityStateChangeEnum.Modified
};

/// <summary>
/// A 5Ct>C00D =Cä5 Cò>C'CDt5C08CR 8CÄ5C08 D$5CαCD"5C4> Cò>C'LCt>C$0D$5C'0 C,,7 Cα>C0BCT:D BC 0C$BCä@C,,7C
/// </summary>
/// <param name="provider">A'>Cα0C'LC0KC' ?D >C$0C"4CT@ D ;D46C „D' 6öçF -æW"'ãÂ÷ & Óí
private async Task<string?> GetUserNameInternalAsync(IServiceProvider provider)
{
    try
    {
        var userProvider = provider.GetService<IUserProvider>();
        if (userProvider != null)
            return await userProvider.GetCurrentUserNameAsync();
    }
    catch
    {
        // ignored
    }

    return "System";
}
}
</file>

<file path="Data/Interceptors/IUserProvider.cs">
namespace Promatis.Net.Data;

public interface IUserProvider
{

```

```
// AÄ5D$>CB <Cä6CTB C KD$ L C AC,,=DT@Cä=CÔKCÄÄ 5D ;C, ?Cä;D4GCT=C,,5 C,,<CT=C, BD 5C CCTB Ct0C0@CäAC
Task<string?> GetCurrentUserAsync();
}
</file>

<file path="Data/Promatis.Net.Data.csproj">
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.8" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />
    <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.1" />
  </ItemGroup>
  <ItemGroup>
    <ProjectReference Include="..\Domain\Promatis.Net.Domain.csproj" />
  </ItemGroup>
  <AssemblyAttribute Include="System.Runtime.CompilerServices.InternalsVisibleTo">
    <Attribute Include="Promatis.Net.Data" />
  </AssemblyAttribute>
</Project>
</file>

<file path="Data/Triggers/Custom/AuditTrigger.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;

namespace Promatis.Net.Data;

public class AuditTrigger(
    IServiceScopeFactory scopeFactory,
    JsonSerializerOptions jsonOptions) : IAfterSaveTrigger<DomainObject>
{
    public async Task HandleAsync(EntityChangedEventArgs<DomainObject> args)
    {
        // AÄGC,,IC 5CÄ @C =D$0C"<-D$8C0 >D" 2Cä7CÄ>Cd=Cä9 CD8C00CÄ8Dt5D :Cä9 C0@Cä:D 8-Cä1Cä;CäGC=8 EF Core (
        Type entityType = args.Entity.GetType();
        if (entityType.Namespace == "Castle.Proxies" && entityType.BaseType != null)
        {
            entityType = entityType.BaseType;
        }

        // 1. BD8C`LD$@: C`>C48D CCT< D$>C`LC= D$5 D CD`=CäAD$8, C=D$>D KCR ?Cä<CTGCT=D² " VF-M
        if (entityType.GetInterfaces().All(i => i.Name != nameof(IAudit)))
            return;

        // 2. BD>D <C,,@D45CÄ 4C =CÔKCR 4C`O JSON
        object changesToSerialize = args.State switch
        {
            EntityStateChangeEnum.Added => args.Changes.Select(c => new
            {
                c.PropertyName,
                NewValue = c.CurrentValue
            }),
            EntityStateChangeEnum.Modified or EntityStateChangeEnum.SoftDeleted =>
                args.Changes.Select(c => new
            {
                c.PropertyName,
                OldValue = c.OriginalValue,
                NewValue = c.CurrentValue
            })
        }
    }
}
</file>
```

```

    }},
    EntityStateChangeEnum.Deleted => new { Message = "Aä1D▯5C▯B D44C ;CT= Cö>C´=CäAD$LDâ" òí
    _ => args.Changes
};

// 3. B >Ct4C 5CÂ 7C ?C,,ADÂ ;Cä3C
var auditLog = new AuditLog
{
    Id = Guid.NewGuid(),
    EntityName = entityType.Name, // A,,!Aö A A´ Aö : Aö8D,,5CÂ GC,,AD$>CR 8CÄO C▯;C AD 0 Cä@C4AD$@D4:D
    EntityId = args.Entity.Id,
    Action = args.State.ToString(),
    ChangedAt = args.ChangedAt,
    ChangedBy = args.ChangedBy,
    ChangesJson = JsonSerializer.Serialize(changesToSerialize, jsonOptions)
};

// 4. A,,!Aö A A´ Aö : A 5Ct>Cö0D =Cä5 D >DT@C =CT=C,,5 C" AB GCT@CT7 C$KCD5C´5Cö=D´9, C40D 0CöBC,,@Cä
using IServiceScope scope = scopeFactory.CreateScope();

// A,,ICT< DD0C @C,,:D2 1C 7Cä2Cä3Cä :Cä=D$5C▯AD$0 Dt5D 5Cr ?D >C$0C"4CT@ C´>C▯0C´LCö>C4> Scope
var factory = scope.ServiceProvider.GetService<IdbContextFactory<ApplicationDbContext>>();

if (factory != null)
{
    await using ApplicationDbContext context = await factory.CreateDbContextAsync();
    context.Set<AuditLog>().Add(auditLog);
    await context.SaveChangesAsync();
}
else
{
    Console.WriteLine("[AuditTrigger][Error] Aö5 D44C ;CäADÂ =C 9D$8 IdbContextFactory<ApplicationDbC
}
}
</file>

<file path="Data/Triggers/Global/FluentValidationTrigger.cs">
using FluentValidation;
using FluentValidation.Results;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Data;
public class FluentValidationTrigger(IServiceScopeFactory scopeFactory) // A,,!Aö A A´ Aö : A,,=Cd5C▯BC,,@D45CÄ
IServiceProvider
{
    : IBeforeSaveTrigger<IDomainObjectHasKey<Guid>>
{
    public async Task HandleAsync(EntityCancelEventArgs<IDomainObjectHasKey<Guid>> args)
    {
        Type entityType = args.Entity.GetType();
        Type validatorType = typeof(IValidator<>).MakeGenericType(entityType);

        // A,,!Aö A A´ Aö : B >Ct4C 5CÂ 8Ct>C´8D >C$0Cö=D4N, C 5Ct>Cö0D =D4N Cä1C´0D BDÂ 2C,,4C,,<CäAD$8 (Scope
        // BöBCä 3C @C =D$8D CCTB, DtBCä •6W'f-6U &÷f-FW" 1D44CTB "Cd8C$KCÄ" 8 D4ACö5D,,=Cä @C 7D 5D,,8D" =C H
        using IServiceScope scope = scopeFactory.CreateScope();

        // At0Cö@C HC,,2C 5CÂ 2C ;C,,4C BCä@ C,,7 D 2CT6CT3CäÂ 3C @C =D$8D >C$0Cö=Cä 6C,,2Cä3Cä 66÷ Rô?D >C$0C"4C
        if (scope.ServiceProvider.GetService(validatorType) is IValidator validator)
        {
            var context = new ValidationContext<object>(args.Entity);
            ValidationResult result = await validator.ValidateAsync(context);

            if (!result.IsValid)
            {
                args.Cancel = true;
                // BD>D <C,,@D45CÄ :D 0D 8C$>CRÂ GC,,AD$>CR ACä>C ICT=C,,5 Cä1 CäHC,,1C▯0DR 4C´O UIö
                args.ErrorMessage = string.Join(" | ", result.Errors.Select(e => e.ErrorMessage));
            }
        }
    }
}
</file>

```

```

<file path="Data/Triggers/Global/ReferenceTreeParentTrigger.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata;
using Promatis.Net.Domain.Interface;
using System.Reflection;
Ð
namespace Promatis.Net.Data;
Ð
/// <summary>Ð
/// B4=C,,2CT@D 0C'LCÔKC' 8CÔDD 0D BD CC=BD4@CÔKC' BD 8C43CT@ CD;Dò ?D >C$5D :C, FCT;CäAD$=CäAD$8 C,,5D 0D EC,,G
/// C, AC=2Cä7C0>C' 7C IC,,BD² >D" FC,,:C'8Dt5D :C,,E Ct0C$8D 8CÄ>D BCT9 C05D 5CB ACäED 0C05C08CT< C" !B4 ABÍ
/// </summary>Ð
public class ReferenceTreeParentTrigger : IBeforeSaveTrigger<IDomainObjectHasKey<Guid>>Ð
{
    private static readonly MethodInfo DbContextSetMethod = typeof(DbContext)Ð
        .GetMethod(BindingFlags.Public | BindingFlags.Instance)Ð
        .First(m => m.Name == nameof(DbContext.Set) && m.IsGenericMethod &&
m.GetParameters().Length == 0);
    Ð
    private static readonly MethodInfo AsNoTrackingMethod =
typeof(EntityFrameworkQueryableExtensions)Ð
        .GetMethod(BindingFlags.Public | BindingFlags.Static)Ð
        .First(m => m.Name == nameof(EntityFrameworkQueryableExtensions.AsNoTracking) &&
m.IsGenericMethod);
    Ð
    private static readonly MethodInfo IgnoreQueryFiltersMethod =
typeof(EntityFrameworkQueryableExtensions)Ð
        .GetMethod(BindingFlags.Public | BindingFlags.Static)Ð
        .First(m => m.Name == nameof(EntityFrameworkQueryableExtensions.IgnoreQueryFilters) &&
m.IsGenericMethod);
    Ð
    public async Task HandleAsync(EntityCancelEventArgs<IDomainObjectHasKey<Guid>> args)Ð
    {
        // A0 Aä!B$ AR AT(AT A,, : Bt5D 5Cr @CTDC'5C=AC,,N C0@Cä2CT@D05CÄÄ 5D BDÄ ;C, C Cä1D=5C=BC AC$>C"AD$2
        // ATAC'8 D 2Cä9D BC$0 C05D" r MD$>D" >C JCT:D" =CR 4CT@CT2CäÄ <D² ?D >D BCä 2D'ECä4C,,<!Ð
        PropertyInfo? parentIdProp = args.Entity.GetType().GetProperty("ParentId");
        if (parentIdProp == null) return;
    Ð
        // Bt8D$0CT< Ct=C GCT=C,,0 D 2Cä9D BC" 4C,,=C <C,,GCTAC=8D
        Guid entityId = args.Entity.Id;
        Guid? parentId = (Guid?)parentIdProp.GetValue(args.Entity);
    Ð
        PropertyInfo? deletedAtProp = args.Entity.GetType().GetProperty("DeletedAt");
    Ð
        // 1. At0D'8D$0 CäB C0@D0<Cä3Cä AC <CäFC,,BC,,@Cä2C =C,,0Ð
        if (parentId.HasValue && parentId == entityId)Ð
        {
            args.Cancel = true;
            args.ErrorMessage = "Aä1D=5C=B C05 CÄ>Cd5D" 1D'BDÄ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.";
            return;
        }
    Ð
        if (parentId.HasValue && parentId.Value != Guid.Empty)Ð
        {
            // A,,ICT< D >CD8D$5C'0 C" 6† ævUG& 6¶W" AD 5CD8 C'NC KDR >C JCT:D$>C"Â C C= >D$>D KDR ACä2C00CD0CT
            object? localParent = args.Context.ChangeTracker.Entries()Ð
                .FirstOrDefault(e => ((IDomainObjectHasKey<Guid>)e.Entity).Id ==
parentId.Value)?.Entity;
            Ð
            bool exists;
            bool isDeleted;
    Ð
            if (localParent != null)Ð
            {
                exists = true;
                var localDeletedAt = (DateTime?)deletedAtProp?.GetValue(localParent);
                isDeleted = localDeletedAt != null;
            }
            elseÐ
            {
                // A$0D,,0 Cä@C,,3C,,=C ;DÄ=C 0 D 5DD;CT:D 8Dò 4C'0 C$KDt8D ;CT=C,,0 C= >D =Dò 8 Query C" Tb 6÷&]
                IEntityType? entityTypeInModel =
args.Context.Model.FindEntityType(args.Entity.GetType());
                Type rootClrType = entityTypeInModel?.GetRootType()?.ClrType ??
args.Entity.GetType();
            }
        }
    }
}

```

```

    object? rawSet =
DbContextSetMethod.MakeGenericMethod(rootClrType).Invoke(args.Context, null);
    object? noTrackingQuery =
AsNoTrackingMethod.MakeGenericMethod(rootClrType).Invoke(null, [rawSet]);
    object? ignoreFiltersQuery =
IgnoreQueryFiltersMethod.MakeGenericMethod(rootClrType).Invoke(null, [noTrackingQuery]);
    IQueryable<object> query = ((IQueryable)ignoreFiltersQuery!).Cast<object>();
    // A„ICT< Că1D≈5C≈B Cô> CD8C00CĂ8Dt5D :Că<D2 AC$>C"AD$2D2 $-B" !B4 AM
    object? dbParent = await query.FirstOrDefaultAsync(x => EF.Property<Guid>(x, "Id")
== parentId.Value);
    exists = dbParent != null;
    var dbDeletedAt = dbParent != null ? (DateTime?)deletedAtProp?.GetValue(dbParent) :
null;
    isDeleted = dbDeletedAt != null;
}
// 2. A0@Că2CT@C≈0 DD8Ct8Dt5D :Că3Că AD4ICTAD$2Că2C =C„0 D >CD8D$5C'LD :Că3Că CCT;C
if (!exists)
{
    args.Cancel = true;
    args.ErrorMessage = $"B4:C 7C =C0KC' @Că4C„BCT;DĂAC≈8C' >C JCT:D" „"C¢ . &VÇD-G0' =CR =C 9CD
    return;
}
// 3. A0@Că2CT@C≈0 D BC BD4AC <D03C≈>C4> D44C ;CT=C„0
if (isDeleted)
{
    args.Cancel = true;
    args.ErrorMessage = "A05C'LCt0 C00Ct=C GC„BDÂ @Că4C„BCT;CT< D44C ;CT=C0KC' >C JCT:D"â#½
    return;
}
// 4. A4 B4 Aă A / At B" B$ Aă" Bd A≈ Aă (A -> B -> C -> A)
Guid? currentCheckId = parentId;
var visitedIds = new HashSet<Guid>();
while (currentCheckId.HasValue && currentCheckId.Value != Guid.Empty)
{
    if (currentCheckId == entityId)
    {
        args.Cancel = true;
        args.ErrorMessage = "Bd8C≈;C„GCTAC≈0D0 7C 2C„AC„<CăAD$L: C05C'LCt0 C05D 5Că5D BC„BDÂ @Că4
        return;
    }
    if (!visitedIds.Add(currentCheckId.Value))
        break;
    // A„ICT< D0;CT<CT=D" FCT?CăGC≈8 D =C GC ;C 2 ChangeTracker
    object? nextLocalElement = args.Context.ChangeTracker.Entries()
.FirstOrDefault(e => ((IDomainObjectHasKey<Guid>)e.Entity).Id ==
currentCheckId.Value)?.Entity;
    if (nextLocalElement != null)
    {
        currentCheckId = (Guid?)parentIdProp.GetValue(nextLocalElement);
    }
    else
    {
        Guid? id = currentCheckId;
        IEntityType? loopEntityTypeInModel =
args.Context.Model.FindEntityType(args.Entity.GetType());
        Type loopRootClrType = loopEntityTypeInModel?.GetRootType()?.ClrType ??
args.Entity.GetType();
        object? loopRawSet =
DbContextSetMethod.MakeGenericMethod(loopRootClrType).Invoke(args.Context, null);
        object? loopNoTrackingQuery =
AsNoTrackingMethod.MakeGenericMethod(loopRootClrType).Invoke(null, [loopRawSet]);
        object? loopIgnoreFiltersQuery =
IgnoreQueryFiltersMethod.MakeGenericMethod(loopRootClrType).Invoke(null, [loopNoTrackingQuery]);

```

```

    IQueryable<object> loopQuery =
    ((IQueryable)loopIgnoreFiltersQuery!).Cast<object>();
    object? parentNodeObj = await loopQuery.FirstOrDefaultAsync(x =>
    EF.Property<Guid>(x, "Id") == id.Value);
    currentCheckId = parentNodeObj != null ?
    (Guid?)parentIdProp.GetValue(parentNodeObj) : null;
    }
    }
}
</file>

<file path="Data/TriggerService/DatabaseTriggerService.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using System.Collections.Concurrent;
namespace Promatis.Net.Data;
/// <summary>
/// B 5C ;C,,7C FC,,0 Dd5C0BD 0C'LC0>C4> CD8D ?CTBDt5D 0 D$@C,,3C45D >C"â #C0@C 2C'0CTB D 5C48D BD 0Dd8CT9 CD>CÄ
/// C, :Cä>D 4C,,=C,,@D45D" DC 7D² 2C ;C,,4C FC,,8 (CD> D >DT@C =CT=C,,0) C, CC$5CD>CÄ;CT=C,,0 (C0>D ;CR ACäED 0C05
/// </summary>
/// <param name="serviceProvider">A :D$CC ;DÄ=D'9 Cä>C0BCT:D B Ct0C$8D 8CÄ>D BCT9 (DI Container) D$5CäCD"5C4>
public class DatabaseTriggerService(IServiceProvider serviceProvider) : IDatabaseTriggerService
{
    /// <summary>
    /// A4;Cä1C ;DÄ=Cä5 D >C KD$8CRâ CC$5CD>CÄ;D0ND"5CR AC,,AD$5CÄC Cä1 D4AC05D,,=Cä< Cä>CÄ<C,,BCR AD4IC0>D BC,
    /// A,,AC0>C'LCtCCTBD 0 CD;D0 @CT0CäBC,,2C0>C4> Cä1C0>C$;CT=C,,0 C,,=D$5D DCT9D 0 (MudBlazor) C,,;C, ACä2Cä7C0
    /// </summary>
    public static event Action<EntityStateChangeEnum, object>? OnEntityCommitted;
    /// <summary>
    /// B 5CTAD$@ C0>CD?C,,Adt8Cä>C"Ä 2D' ?Cä;C00DäIC,,ED 0 AD D >DT@C =CT=C,,0 C,,7CÄ5C05C08C' ,, $C 7C C ;C,,4C
    /// AD5C'5C40D" ?D 8C08CÄ0CTB C @C4CCÄ5C0BD² ACä1D'BC,,0 C, 0CäBD40C'LC0KC' 66÷ VB 6W'f-6U &÷f-FW"i
    /// </summary>
    private static readonly Dictionary<Type, List<Func<EntityCancelArgsBase, IServiceProvider,
Task>>> BeforeSubscribers = new();
    /// <summary>
    /// B 5CTAD$@ C0>CD?C,,Adt8Cä>C"Ä 2D' ?Cä;C00DäIC,,ED 0 A0 B AR CD ?CTHC0>C4> D >DT@C =CT=C,,0 C,,7CÄ5C05C08C
    /// </summary>
    private static readonly Dictionary<Type, List<Func<EntityChangedArgsBase, IServiceProvider,
Task>>> AfterSubscribers = new();
    /// <summary>
    /// A0>D$>Cä>C 5Ct>C00D =D'9 CäMD, 8CT@C @DT8C, BC,,?Cä2. A,,ACä;DäGC 5D" ?Cä2D$>D =Cä5 C,,AC0>C'LCt>C$0C08C
    /// C0@C, >C0@CT4CT;CT=C,,8 C 0Ct>C$KDR :C'0D ACä2 C, 8C0BCT@DD5C"ACä2 Cä1D 0C 0D$KC$0CT<D'E D CD"=CäAD$5C
    /// </summary>
    private static readonly ConcurrentDictionary<Type, List<Type>> HierarchyCache = new();
    /// <summary>
    /// B 5C48D BD 8D CCTB CD>CÄ5C0=D'9 D$@C,,3C45D 4C'0 Cä>C0:D 5D$=Cä3Cä BC,,?C AD4IC0>D BC,i
    /// A$KctKC$0CTBD 0 C 2D$>CÄ0D$8Dt5D :C, ?D 8 D :C =C,,@Cä2C =C,,8 D 1Cä@Cä: C$> C$@CT<D0 8C08Dd8C ;C,,7C FC
    /// </summary>
    /// <typeparam name="TEntity">B$8C0 4Cä<CT=C0>C' AD4IC0>D BC,äÄ÷G- W & Ói
    /// <typeparam name="TTrigger">B$8C0 :C'0D AC 0BD 8C43CT@C Ä @CT0C'8CtCDäICT3Cä ;Cä3C,,:D2 >C @C 1CäBCä8.<
    public void Register<TEntity, TTrigger>()
    where TEntity : class
    where TTrigger : class
    {
        Type entityType = typeof(TEntity);
        Type triggerType = typeof(TTrigger);
        // A0@Cä2CT@D05CÄÄ :C :C,,5 C,,=D$5D DCT9D K Cd8Ct=CT=C0>C4> Dd8Cä;C @CT0C'8CtCCTB CD0C0=D'9 D$@C,,3C45
        bool isBefore = typeof(IBeforeSaveTrigger<TEntity>).IsAssignableFrom(triggerType);
        bool isAfter = typeof(IAfterSaveTrigger<TEntity>).IsAssignableFrom(triggerType);
        if (isBefore)
        {
            // BD>D <C,,@D45CÄ >D$;Cä6CT=C0CDA ;D0<C 4D2ä Cä=D$5C"=CT@ 'sp' C05D 5CD0CTBD 0 C" <Cä<CT=D" 2D'7
            // DtBCä ?D 5CD>D$2D 0D"0CTB D4BCTGCäC CD>C'3Cä6C,,2D4IC,,E Cä1Dä5CäBCä2 (Captive Dependency).

```



```

AddSubscriber(BeforeSubscribers, entityType, async (argsBase, sp) =>
{
    // B 0Ct@CTHC 5CÂ MC7CT<C0;D0@ D$@C,,3C45D 0 C,,7 C :D$CC ;DÄ=Cä3Câ 66÷ R BCT:D4ICT3Câ 7C ?D >
    if (sp.GetRequiredService<TTrigger>() is IBeforeSaveTrigger<TEntity> handler)
    {
        // Aä1Cä@C GC,,2C 5Câ 1C 7Cä2D´5 C @C4CCÄ5C0BD² 2 D BD >C4> D$8C08Ct8D >C$0C0=D´9 C7>C0BCT
        var typedArgs = new EntityCancelEventArgs<TEntity>(
            (TEntity)argsBase.Entity,
            argsBase.State,
            argsBase.Changes,
            argsBase.Context);

        await handler.HandleAsync(typedArgs);

        // A$>Ct2D 0D"0CT< D 5CtCC´LD$0D$K C$KC0>C´=CT=C,,0 D$@C,,3C45D 0 Cä1D 0D$=Câ 2 C 0Ct>C$KC´
        argsBase.Cancel = typedArgs.Cancel;
        argsBase.ErrorMessage = typedArgs.ErrorMessage;
        argsBase.Handled = typedArgs.Handled;
    }
});

if (isAfter)
{
    AddSubscriber(AfterSubscribers, entityType, async (argsBase, sp) =>
    {
        if (sp.GetRequiredService<TTrigger>() is IAfterSaveTrigger<TEntity> handler)
        {
            var typedArgs = new EntityChangedEventArgs<TEntity>(
                (TEntity)argsBase.Entity,
                argsBase.State,
                argsBase.Changes,
                argsBase.ChangedBy,
                argsBase.ChangedAt);

            await handler.HandleAsync(typedArgs);
            argsBase.Handled = typedArgs.Handled;
        }
    });
}

// ATAC´8 C7;C AD =CR @CT0C´8CtCCTB C08 Cä4C,,= C7>C0BD 0C7B – D0BCâ :D 8D$8Dt5D :C 0 CäHC,,1C70 C7>C0
if (!isBefore && !isAfter)
{
    throw new InvalidOperationException(
        $"A7;C AD .G&-vvw%G– Rææ ÖWÖ =CR @CT0C´8CtCCTB IBeforeSaveTrigger C,,;C, " gFW%6 fUG&-vvw" 4C
    );
}

/// <summary>
/// A$KC0>C´=D05D" AC7Cä7C0CDâ 2C ;C,,4C FC,,N C,,7CÄ5C00CT<Cä9 D CD"=CäAD$8 C05D 5CB 5E >D$?D 0C$:Cä9 C"
/// Aä?D 0D,,8C$0CTB C$ADâ FCT?CäGC7C C00D ;CT4Cä2C =C,,0 D CD"=CäAD$8. A0@CT@D´2C 5D" >C05D 0Dd8Dâ ?D 8 C0
/// </summary>
public async Task ValidateAsync(object entity, EntityStateChangeEnum state,
List<PropertyChangeInfo> changes, DbContext context)
{
    var args = new EntityCancelArgsBase(entity, state, changes, context);

    // Aä1DT>CD8CÂ 8CT@C @DT8Dâ BC,,?Cä2 (D 0CÄ>C´ AD4IC0>D BC,Â 1C 7Cä2D´E C7;C AD >C" 8 C,,=D$5D DCT9D >C
    foreach (Type type in GetTypesHierarchy(entity.GetType()))
    {
        if (BeforeSubscribers.TryGetValue(type, out List<Func<EntityCancelArgsBase,
IServiceProvider, Task>>> actions))
        {
            foreach (Func<EntityCancelArgsBase, IServiceProvider, Task> action in actions)
            {
                // A05D 5CD0CT< C´>C70C´LC0KC´ 6W'f-6U &÷f-FW"Â A C7>D$>D KCÂ 1D´; D >Ct4C = CD0C0=D´9 D0
                await action(args, serviceProvider);

                // ATAC´8 D$@C,,3C45D 2D´AD$0C$8C² DC´0C2 6 æ6VÂ r =CT<CT4C´5C0=Câ ?D 5D KC$0CT< D$@C =Ct
                if (args.Cancel) throw new OperationCanceledException(args.ErrorMessage);
            }
        }
    }
}

```

```

    /// <summary>ð
    /// B 0D AD';C 5D" CC$5CD>CÄ;CT=C,,0 Cä1 D4AC05D,,=Cä< D >DT@C =CT=C,,8 C,,7CÄ5C05C08C' 2 C 0CtC CD0C0=D'E.ð
    /// Að>CD4CT@Cd8C$0CTB C=0D :C 4C0KC' ?CT@CTEC$0D" ,,DC'0C2 † æFÆVB' 8 C0CC ;C,,:D45D" ACä1D'BC,,5 C" 3C'>C
    /// </summary>ð
    public async Task NotifyAsync(object entity, EntityStateChangeEnum state,
List<PropertyChangeInfo> changes, string? user, DateTime at)ð
    {ð
        var args = new EntityChangedEventArgsBase(entity, state, changes, user, at);ð
    ð
        foreach (Type type in GetTypesHierarchy(entity.GetType()))ð
        {ð
            if (AfterSubscribers.TryGetValue(type, out List<Func<EntityChangedEventArgsBase,
IServiceProvider, Task>>? actions))ð
            {ð
                foreach (Func<EntityChangedEventArgsBase, IServiceProvider, Task> action in actions)ð
                {ð
                    await action(args, serviceProvider);ð
                ð
                // ATAC'8 Cä4C,,= C,,7 D$@C,,3C45D >C" 2D'AD$0C$8C² † æFÆVB Ò G'VRÂ >C @C 1CäBC=0 Dd5C0>Dt:C
                if (args.Handled) return;ð
            }ð
        }ð
    ð
        // A0CC ;C,,:C FC,,0 C" AD$0D$8Dt5D :C,,9 D02CT=D" 4C'0 D 5C :D$8C$=Cä3Cä >C =Cä2C'5C08D0 :Cä<C0>C05C0BC
        OnEntityCommitted?.Invoke(state, entity);ð
    }ð
    ð
    /// <summary>ð
    /// A$AC0>CÄ>C40D$5C'LC0KC' <CTBCä4 CD;D0 1CT7Cä?C AC0>C4> CD>C 0C$;CT=C,,0 C0>CD?C,,Adt8C= >C" 2 D ;Cä2C @C
    /// </summary>ð
    private void AddSubscriber<TArgs>(Dictionary<Type, List<Func<TArgs, IServiceProvider, Task>>>
dict, Type type, Func<TArgs, IServiceProvider, Task> action)ð
    {ð
        if (!dict.TryGetValue(type, out List<Func<TArgs, IServiceProvider, Task>>? list))ð
        {ð
            list = new List<Func<TArgs, IServiceProvider, Task>>();ð
            dict[type] = list;ð
        }ð
        list.Add(action);ð
    }ð
    ð
    /// <summary>ð
    /// A,,7C$;CT:C 5D" ?Cä;C0CDä 8CT@C @DT8Dä BC,,?Cä2 CD;D0 CC=0Ct0C0=Cä3Cä >C JCT:D$0 (C$:C'NDt0D0 1C 7Cä2D'
    /// B 5CtCC'LD$0D$K C=MD,,8D CDäBD 0 CD;D0 >C 5D ?CTGCT=C,,0 C$KD >C= >C' ?D >C,,7C$>CD8D$5C'LC0>D BC, ?Cä4 C
    /// </summary>ð
    private IEnumerable<Type> GetTypesHierarchy(Type type) =>ð
    {
        HierarchyCache.GetOrAdd(type, t => {ð
            var types = new List<Type>();ð
            // B 5C=CD AC,,2C0> C0>CD=C,,<C 5CÄAd0 2C$5D E C0> CD@CT2D2 =C AC'5CD>C$0C08D0 :C'0D ACä2, C,,AC=;Dä
            for (Type? c = t; c != null && c != typeof(object); c = c.BaseType) types.Add(c);ð
            // AD>C 0C$;D05CÄ 2D 5 D 5C ;C,,7Cä2C =C0KCR 8C0BCT@DD5C"AD² 8 D41C,,@C 5CÄ 4D41C'8C=0D$KB
            return types.Concat(t.GetInterfaces()).Distinct().ToList();ð
        });ð
    }ð
    ð
    /// <summary>ð
    /// B @CäGC0KC' AC @CäA D BC BC,,GCTAC=8DR @CT3C,,AD$@C FC,,9. ð
    /// A,,AC0>C'LCtCCTBD 0 C0@CT8CÄCD"5D BC$5C0=Cä 2 Unit-D$5D BC E CD;D0 8Ct>C'ODd8C, ?D >C4>C0>C" BCTAD$>C"
    /// </summary>ð
    internal static void ClearInternalRegistrations()ð
    {ð
        BeforeSubscribers.Clear();ð
        AfterSubscribers.Clear();ð
        HierarchyCache.Clear();ð
    }ð
}
</file>

<file path="Data/TriggerService/EntityStateChangeEnum.cs">
namespace Promatis.Net.Data;ð
ð
public enum EntityStateChangeEnumð
{ð
    Added,ð
    Modified,ð
    Deleted,ð

```

```

        SoftDeleted
    }
</file>

<file path="Data/TriggerService/IDatabaseTriggerService.cs">
using Microsoft.EntityFrameworkCore;

namespace Promatis.Net.Data;

public interface IDatabaseTriggerService
{
    // AÄ5D$>CB 4C´O C00D BD >C":C, AC$0Ct5C' $!D4ICÔ>D BDÂÔ"D 8C43CT@
    void Register<TEntity, TTrigger>()
        where TEntity : class
        where TTrigger : class;
}

    // AÄ5D$>CDK C$KCô>C´=CT=C,,O (C$KctKC$0DäBD O C,,=D$5D FCT?D$>D >CÂ•
    Task ValidateAsync(object entity, EntityStateChangeEnum state, List<PropertyChangeInfo>
changes, DbContext context);
    Task NotifyAsync(object entity, EntityStateChangeEnum state, List<PropertyChangeInfo> changes,
string? user, DateTime at);
}
</file>

<file path="Data/TriggerService/PropertyChangeInfo.cs">
namespace Promatis.Net.Data;

public class PropertyChangeInfo
{
    public string PropertyName { get; set; } = string.Empty;
    public object? OriginalValue { get; set; }
    public object? CurrentValue { get; set; }
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityCancelArgsBase.cs">
using Microsoft.EntityFrameworkCore;

namespace Promatis.Net.Data;

public class EntityCancelArgsBase(
    object entity,
    EntityStateChangeEnum state,
    List<PropertyChangeInfo> changes,
    DbContext context) : EntityEventArgsBase(entity, state, changes)
{
    public DbContext Context { get; } = context;
    public bool Cancel { get; set; }
    public string? ErrorMessage { get; set; }
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityCancelEventArgs.cs">
using Microsoft.EntityFrameworkCore;

namespace Promatis.Net.Data;

public class EntityCancelEventArgs<T>(
    T entity,
    EntityStateChangeEnum state,
    List<PropertyChangeInfo> changes,
    DbContext context)
    : EntityCancelArgsBase(entity!, state, changes, context) where T : class
{
    public new T Entity => (T)base.Entity;
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityChangedEventArgsBase.cs">
namespace Promatis.Net.Data;

// A 0Ct>C$KC' :C´0D A CD;Dò 2C0CD$@CT=C05C4> C,,ACô>C´Lct>C$0C08Dò 2 D 5D 2C,,AC]
public class EntityChangedEventArgsBase(
    object entity,
    EntityStateChangeEnum state,
    List<PropertyChangeInfo> changes,

```

```

        string? changedBy, D
        DateTime changedAt) : EntityEventArgsBase(entity, state, changes) // <-- A00D ;CT4D45CÂ 7CD5D L0
    {D
        public string? ChangedBy { get; } = changedBy;D
        public DateTime ChangedAt { get; } = changedAt;D
    }
</file>

```

```

<file path="Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs">
namespace Promatis.Net.Data;D
D
// A$0D, BC,,?C,,7C,,@Cä2C =C0KC' :C'0D A CD;D0 BD 8C43CT@Cä2D
public class EntityChangedEventArgs<T>(D
    T entity, EntityStateChangeEnum state, D
    List<PropertyChangeInfo> changes, D
    string? changedBy, D
    DateTime changedAt)D
    : EntityChangedEventArgsBase(entity!, state, changes, changedBy, changedAt) where T : classD
{D
    public new T Entity => (T)base.Entity;D
}
</file>

```

```

<file path="Data/TriggerService/TriggerEvents/EntityEventArgsBase.cs">
namespace Promatis.Net.Data;D
D
public abstract class EntityEventArgsBase(D
    object entity,D
    EntityStateChangeEnum state,D
    List<PropertyChangeInfo> changes) : EventArgsD
{D
    public object Entity { get; } = entity;D
    public EntityStateChangeEnum State { get; } = state;D
    public List<PropertyChangeInfo> Changes { get; } = changes;D
    public bool Handled { get; set; }D
}
</file>

```

```

<file path="Data/TriggerService/TriggerInterfaces/IAfterSaveTrigger.cs">
namespace Promatis.Net.Data;D
D
// AD;D0 ?D 0C$8C² $ Aä!A' " D >DT@C =CT=C,,0 (D42CT4Cä<C'5C08D00HC,,=C •
public interface IAfterSaveTrigger<T> where T : classD
{D
    Task HandleAsync(EntityChangedEventArgs<T> args);D
}
</file>

```

```

<file path="Data/TriggerService/TriggerInterfaces/IBeforeSaveTrigger.cs">
namespace Promatis.Net.Data;D
D
// AD;D0 ?D 0C$8C² $ Aä" ACäED 0C05C08D0 „2C ;C,,4C FC,,0)D
public interface IBeforeSaveTrigger<T> where T : classD
{D
    Task HandleAsync(EntityCancelEventArgs<T> args);D
}
</file>

```

```

<file path="Data/Validation/GlobalPolymorphicValidator.cs">
using FluentValidation;D
using FluentValidation.Results;D
using Microsoft.Extensions.DependencyInjection;D
D
namespace Promatis.Net.Domain;D
D
public class GlobalPolymorphicValidator<T> : AbstractValidator<T> where T : classD
{D
    private readonly IServiceProvider _serviceProvider;D
D
    public GlobalPolymorphicValidator(IServiceProvider serviceProvider)D
    {D
        _serviceProvider = serviceProvider ?? throw new
ArgumentNullException(nameof(serviceProvider));D
    }D
D
    protected override bool PreValidate(ValidationContext<T> context, ValidationResult result)D

```

```

    {
        if (context.InstanceToValidate == null) return true;

        // 1. A0>C'CDt0CT< D 5C ;DÄ=D'9 D$8C0 >C JCT:D$0 C" @C =D$0C"<CR „=C ?D 8CÄ5D Ä FW 'FÖVÇEVæ-B 8C'8 A
        Type realType = context.InstanceToValidate.GetType();

        // ATAC'8 D$8C0 ACä2C00CD0CTB D B „B.CRâ MD$> Cä>C05Dt=D'9 Cä;C AD Ä 0 C05 C 0Ct>C$KC' 0C AD$@C :D$=
        // D$> C„ACä0D$L CD>Dt5D =C„9 C$0C'8CD0D$>D =CR =D46C0>, DtBCä1D² =CR CC"BC, 2 C 5D :Cä=CTGC0CDâ @CT
        if (realType == typeof(T)) return true;

        // 2. B BD >C„< D$8C0 8C0BCT@DD5C"AC 4C'O D$>Dt5Dt=Cä3Cä 2C ;C„4C BCä@C „=C ?D 8CÄ5D Ä •f Æ-F F÷#ÄF
        Type specificValidatorType = typeof(IValidator<>).MakeGenericType(realType);

        // 3. At0C0@C HC„2C 5CÄ 8Cr D' B C$0C'8CD0D$>D K CD;D0 MD$>C4> Cä>C0:D 5D$=Cä3Cä BC„?C
        // B0BCä 2C 6C0>, D$0C4 :C : CD;D0 >CD=Cä3Cä BC„?C <Cä6CTB C KD$L Ct0D 5C48D BD 8D >C$0C0> C05D :Cä;
        List<IValidator> specificValidators =
        _serviceProvider.GetServices(specificValidatorType).Cast<IValidator>().ToList();

        if (specificValidators.Any())
        {
            // B >Ct4C 5CÄ =CR04Cd5C05D 8C4 :Cä=D$5CäAD" 2C ;C„4C FC„8 FluentValidation CD;D0 ?CT@CT4C GC, 4C
            ValidationContext<object>? newContext =
            ValidationContext<object>.CreateWithOptions(context.InstanceToValidate, options =>
            {
                // A„!A0 A A' A0 : Aä0C0>C08Dt=D'9 CÄ5D$>CB fÇVVÇEf Æ-F F-öâ 4C'O C0@D0<Cä9 C0@Cä1D >D :C, A
                if (context.Selector != null)
                {
                    options.UseCustomSelector(context.Selector);
                }
            });

            // At0C0CD :C 5CÄ :C 6CDKC' =C 9CD5C0=D'9 D$>Dt5Dt=D'9 C$0C'8CD0D$>D =C AC'5CD=C„:C
            foreach (IValidator validator in specificValidators)
            {
                // BtBCä1D² 8Ct1CT6C BDÄ 7C FC„:C'8C$0C08D0Ä ?D >C$5D OCT<, DtBCä =C 9CD5C0=D'9 C$0C'8CD0D$>D
                if (validator.GetType().IsGenericType &&
                validator.GetType().GetGenericTypeDefinition() == typeof(GlobalPolymorphicValidator<>))
                continue;

                ValidationResult specificResult = validator.Validate(newContext);

                // A05D 5C0>D 8CÄ 2D 5 Cä1C00D CCd5C0=D'5 CäHC„1Cä8 C" >C IC„9 D 5CtCC'LD$0D" 2C ;C„4C FC„8 D
                foreach (ValidationFailure error in specificResult.Errors)
                {
                    result.Errors.Add(error);
                }
            }
        }

        return true;
    }
}
</file>

```

```

<file path="Domain.Interface/Enums/EnumExtensions.cs">
using System.ComponentModel;
using System.Reflection;

namespace Promatis.Net.Domain.Interface
{
    public static class EnumExtensions
    {
        /// <summary>
        /// A$>Ct2D 0D"0CTB Ct=C GCT=C„5 C BD 8C CD$0 [Description] CD;D0 ;Dä1Cä3Cä MC'5CÄ5C0BC ?CT@CTGC„AC'5C08
        /// ATAC'8 C BD 8C CD" >D$AD4BD BC$CCTB, C$>Ct2D 0D"0CTB D BC =CD0D BC0>CR 8CÄ0 .ToString().
        /// </summary>
        public static string GetDescription(this Enum value)
        {
            FieldInfo? field = value.GetType().GetField(value.ToString());
            if (field == null) return value.ToString();

            var attribute = field.GetCustomAttribute<DescriptionAttribute>();
            return attribute != null ? attribute.Description : value.ToString();
        }
    }
}
</file>

```

```

<file path="Domain.Interface/Interfaces/IAudit.cs">
namespace Promatis.Net.Domain.Interface;
{
    /// <summary>
    /// B CD'=CäAD$8, D 5C ;C,,7D4ND"8CR MD$>D" 8CÔBCT@DD5C"A, C CCDCD" 7C ?C,,AD'2C BDÄADò 2 AuditLogð
    /// </summary>
    public interface IAudit
    {
        ð
    }
}
</file>

```

```

<file path="Domain.Interface/Interfaces/IDomainObject.cs">
namespace Promatis.Net.Domain.Interface;
{
    public interface IDomainObject
    {
        DateTime CreatedAt { get; init; }
    }
}
</file>

```

```

<file path="Domain.Interface/Interfaces/IDomainObjectHasKey.cs">
namespace Promatis.Net.Domain.Interface;
{
    public interface IDomainObjectHasKey<TKey> : IDomainObject
    {
        TKey Id { get; set; }
    }
}
</file>

```

```

<file path="Domain.Interface/Interfaces/IEntityCloner.cs">
namespace Promatis.Net.Domain.Interface;
{
    /// <summary>
    /// A,,=D$5D DCT9D ?C'0D$DCä@CÄ5CÔ=Cä3Cä 4C$8Cd:C 8Ct>C'8D >C$0CÔ=Cä3Cä :C'>CÔ8D >C$0CÔ8Dò 4Cä<CT=CÔKDR AD4I
    /// </summary>
    public interface IEntityCloner
    {
        T CloneEntity<T>(T entity) where T : class;
    }
}
</file>

```

```

<file path="Domain.Interface/Interfaces/ISoftDeletable.cs">
namespace Promatis.Net.Domain.Interface;
{
    /// <summary>
    /// A,,=D$5D DCT9D 4C'0 CÄ0C4:Cä3Cä CCD0C'5CÔ8Dò >C JCT:D$>C-
    /// </summary>
    public interface ISoftDeletable
    {
        DateTime? DeletedAt { get; set; }
        string? DeletedBy { get; set; }
    }
}
</file>

```

```

<file path="Domain.Interface/Interfaces/ITreeNode.cs">
namespace Promatis.Net.Domain.Interface;
{
    /// <summary>
    /// A4;Cä1C ;DÄ=D'9 Cò;C BDD>D <CT=CÔKC' :Cä=D$@C :D" 4C'0 C$ACTE C,,5D 0D EC,,GCTAC=8DR ,,4D 5C$>C$8CD=D'E) D C
    /// A,,!Aô A A' Aô : AD>C 0C$;CT= set CD;Dò 2Cä7CÄ>Cd=CäAD$8 C,,7CÄ5CÔ5CÔ8Dò AC$0Ct5C' ,,?CT@CT<CTICT=C,,O C
    /// <typeparam name="T">B$8Cò 4Cä<CT=CÔ>C' <Cä4CT;C,äÄ÷G- W & Óí
    public interface ITreeNode<T> : IDomainObjectHasKey<Guid> where T : class
    {
        /// <summary>
        /// A,,4CT=D$8DD8C=0D$>D @Cä4C,,BCT;DÄAC=8C' >C JCT:D"1
        /// A,,!Aô A A' Aô : AD>C 0C$;CT= set CD;Dò 2Cä7CÄ>Cd=CäAD$8 C,,7CÄ5CÔ5CÔ8Dò AC$0Ct5C' ,,?CT@CT<CTICT=C,,O C
        /// </summary>
        Guid? ParentId { get; set; }
    }
}
    /// <summary>
    /// A00C$8C40Dd8Cä=CÔ>CR AC$>C"AD$2Cä AD KC':C, =C @Cä4C,,BCT;DÄAC=8C' >C JCT:D"1
    /// </summary>

```

```

    T? Parent { get; set; }
}
/// <summary>
/// A<C';CT:Dd8D 4CäGCT@C08DR MC'5CÄ5C0BCä2 (C0>CDGC,,=CT=C0KDR Cct;Cä2).D
/// </summary>
ICollection<T> Children { get; }
}
</file>

```

```

<file path="Domain.Interface/Promatis.Net.Domain.Interface.csproj">
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
</file>

```

```

<file path="Domain/AuditLog/AuditLog.cs">
namespace Promatis.Net.Domain;
{
  /// <summary>
  /// A CCD8D" AC,,AD$5CÄK
  /// </summary>
  public class AuditLog : DomainObject
  {
    /// <summary>
    /// A00C,,<CT=Cä2C =C,,5 D >C KD$8Dý
    /// </summary>
    public string EntityName { get; init; } = string.Empty;

    /// <summary>
    /// A,,4CT=D$8DD8C00D$>D ACä1D'BC,,0
    /// </summary>
    public Guid EntityId { get; init; }

    /// <summary>
    /// AD5C"AD$2C,,5
    /// </summary>
    public string Action { get; init; } = string.Empty;

    /// <summary>
    /// AD0D$0 C,,7CÄ5C05C08Dý
    /// </summary>
    public DateTime ChangedAt { get; init; }

    /// <summary>
    /// A,,7CÄ5C05C0>
    /// </summary>
    public string? ChangedBy { get; init; }

    /// <summary>
    /// B BD >C00 C,,7CÄ5C05C08C•
    /// </summary>
    public string ChangesJson { get; init; } = string.Empty;
  }
</file>

```

```

<file path="Domain/DomainObject/DomainObject.cs">
namespace Promatis.Net.Domain;
{
  /// <summary>
  /// AäAC0>C$=Cä9 C0;C AD „uT"B ?Cä CCÄ>C'GC =C,,N)
  /// </summary>
  public abstract class DomainObject : DomainObjectBase<Guid>
  {
    protected DomainObject()
    {
      Id = Guid.NewGuid();
    }
  }
</file>

```

```

<file path="Domain/DomainObject/DomainObjectBase.cs">
using Promatis.Net.Domain.Interface;
{
namespace Promatis.Net.Domain;
{
/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 1C 7Cä2D'9 C;C AD A C0>CD4CT@Cd:Cä9 C'NC >C4> D$8C00 C;DäGC
/// </summary>
/// <typeparam name="TKey">B$8C0 :C'NDt0</typeparam>
public abstract class DomainObjectBase<TKey> : IDomainObjectHasKey<TKey>
{
    public TKey Id { get; set; } = default!;
}
    public DateTime CreatedAt { get; init; } = DateTime.UtcNow;
}
</file>

<file path="Domain/DomainObject/DomainObjectValidator.cs">
using FluentValidation;
{
namespace Promatis.Net.Domain;
{
/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 2C ;C,,4C BCä@ CD;D0 2D 5DR :C'0D ACä2, C00D ;CT4D45CÄKDR >D" F00 -ä0&|V7M
/// </summary>
public abstract class DomainObjectValidator<T> : AbstractValidator<T> where T : DomainObject
{
    protected DomainObjectValidator()
    {
        RuleFor(x => x.Id)
            .NotEmpty().WithMessage("A,,4CT=D$8DD8C00D$>D >C JCT:D$0 C05 CÄ>Cd5D" 1D'BDÄ ?D4AD$KCÄÄ""½
}
        RuleFor(x => x.CreatedAt)
            .Must(date => date <= DateTime.UtcNow.AddSeconds(1))
            .WithMessage("AD0D$0 D >Ct4C =C,,0 C05 CÄ>Cd5D" 1D'BDÄ 2 C CCDCD"5CÄÄ""½
    }
}
</file>

<file path="Domain/EntityCloner/JsonEntityCloner.cs">
using System.Text.Json;
using System.Text.Json.Serialization;
using System.Text.Json.Serialization.Metadata;
using Promatis.Net.Domain.Interface;
{
namespace Promatis.Net.Domain;
{
public class JsonEntityCloner : IEntityCloner
{
    private static readonly JsonSerializerOptions JsonOptions = new()
    {
        ReferenceHandler = ReferenceHandler.IgnoreCycles,
        TypeInfoResolver = new DefaultJsonTypeInfoResolver
        {
            Modifiers = { typeInfo =>
                {
                    foreach (var property in typeInfo.Properties)
                    {
                        // 1. A$KD 5Ct0CT< Dd8C;C,,GCTAC08CR =C 2C,,3C FC,,>C0=D'5 D 2Cä9D BC$0 C,,5D 0D EC,,9D
                        if (property.Name is "Parent" or "Children")
                        {
                            property.ShouldSerialize = (_, _) => false;
                            continue;
                        }
                        // 2. A$KD 5Ct0CT< D ;Cä6C0KCR AC$0Ct0C0=D'5 CD>CÄ5C0=D'5 Cä1D05C0BD² 1D0:CT=CD0 C, :Cä;C
                        Type propType = property.PropertyType;
                        bool isDomainClass = typeof(Domain.DomainObject).IsAssignableFrom(propType);
                        bool isDomainCollection = propType.IsGenericType &&
                            typeof(System.Collections.IEnumerable).IsAssignableFrom(propType) &&
                            typeof(Domain.DomainObject).IsAssignableFrom(propType.GetGenericArguments().FirstOrDefault());
                        if (isDomainClass || isDomainCollection)
                    }
                }
            }
        }
    }
}
}

```



```

        property.ShouldSerialize = (_, _) => false;
    }
}
}
};
}

public T CloneEntity<T>(T entity) where T : class
{
    if (entity == null) throw new ArgumentNullException(nameof(entity));

    // B 5D 8C ;C,,7D45CÂ 8 CD5D 5D 8C ;C,,7D45CÂÂ AD$@Cä3Câ ACäED 0C00D0 @CT0C'LC0KC' @C =D$0C"<-D$8C0 =C
    string json = JsonSerializer.Serialize(entity, entity.GetType(), JsonOptions);
    return (T)JsonSerializer.Deserialize(json, entity.GetType(), JsonOptions)!;
}
}
</file>

<file path="Domain/Promatis.Net.Domain.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="FluentValidation" Version="12.1.1" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Domain.Interface\Promatis.Net.Domain.Interface.csproj" />
    </ItemGroup>
</Project>
</file>

<file path="Domain/ReferenceBase/ReferenceBase.cs">
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Domain;
/// <summary>
/// A 0Ct>C$KC' :C'0D A CD;D0 2D 5DR AC0@C 2CäGC08C">C-
/// </summary>
public abstract class ReferenceBase : DomainObject, ISoftDeletable
{
    public string Name { get; set; } = string.Empty;

    public string? Description { get; set; }

    public string? Code { get; set; }

    public DateTime? DeletedAt { get; set; }

    public string? DeletedBy { get; set; }
}
</file>

<file path="Domain/ReferenceBase/ReferenceBaseValidator.cs">
using FluentValidation;
using FluentValidation.Results;
namespace Promatis.Net.Domain;
/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 2C ;C,,4C BCä@ CD;D0 2D 5DR :C'0D ACä2, C00D ;CT4D45CÄKDR >D" &VfW&Væ6T& 6]
/// </summary>
public abstract class ReferenceBaseValidator<T> : DomainObjectValidator<T> where T : ReferenceBase
{
    /// <summary>
    /// A$8D BD40C'LC0KC' ?C BD$5D = D 5C4CC'0D =Cä3Câ 2D'@C 6CT=C,,0 CD;D0 ?D >C$5D :C, Cä4C í
    /// A0> D4<Cä;Dt0C08Dä¢ AD$@Cä3Câ ;C BC,,=C,,FC 2 C$5D EC05CÂ @CT3C,,AD$@CRÂ FC,,DD K C, 0í
    /// </summary>
    protected virtual string CodeRegexPattern => @"^[A-Z0-9_]*$";
}

```

```

Ð
/// <summary>Ð
/// A$8D BD40C´LCÔ>CR ACä>C ICT=C,,5 Că1 CăHC,,1C¤5 C$0C´8CD0Dd8C, @CT3D4;Dô@CÔ>C4> C$KD 0Cd5CÔ8Dò Că4C í
/// </summary>Ð
protected virtual string CodeValidationMessage => "A¤>CB <Că6CTB D >CD5D 6C BDÂ BCä;DÄ:Că ;C BC,,=C,,FD2 2
Ð
protected ReferenceBaseValidator()Ð
{Ð
    RuleFor(x => x.Name)Ð
        .NotEmpty().WithMessage("AÔ0C,,<CT=Că2C =C,,5 Că1Dô7C BCT;DÄ=Că 4C´O Ct0Cô>C´=CT=C,,0")Ð
        // Aô@Că2CT@C¤0 CÔ0 Cô@Că1CT;D² 2 CÔ0Dt0C´5 C, :Că=Dd5Ð
        .Must(name => name == null || name.Trim() == name)Ð
        .WithMessage("AÔ0C,,<CT=Că2C =C,,5 CÔ5 CĂ>Cd5D" =C GC,,=C BDÄADò 8C´8 Ct0C¤0CÔGC,,2C BDÄADò ?D >C 5C´
        .MinimumLength(2).WithMessage("AÔ0C,,<CT=Că2C =C,,5 CD>C´6CÔ> D >CD5D 6C BDÂ <C,,=C,,<D4< 2 D 8CĂ2Că;
        .MaximumLength(250).WithMessage("Aô@CT2D´HCT=C ´<C :D 8CĂ0C´LCÔ0Dò 4C´8CÔ0 CÔ0C,,<CT=Că2C =C,,0 (25
Ð
    RuleFor(x => x.Code)Ð
        .Matches(x => CodeRegexPattern)Ð
        .When(x => !string.IsNullOrEmpty(x.Code))Ð
        .WithMessage(x => CodeValidationMessage);Ð
    }Ð
Ð
    public Func<object, string, Task<IEnumerable<string>>> ValidateValue => async (model,
propertyName) =>Ð
    {Ð
        ValidationResult? result = await
ValidateAsync(ValidationContext<T>.CreateWithOptions((T)model, x =>
x.IncludeProperties(propertyName)));Ð
        return result.IsValid ? [] : result.Errors.Select(e => e.ErrorMessage);Ð
    };Ð
}
</file>

<file path="Domain/ReferenceTreeBase/ReferenceTreeBase.cs">
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.Domain;Ð
Ð
/// <summary>Ð
/// A 0Ct>C$KC´ :C´0D A CD;Dò 2D 5DR ACô@C 2CăGCÔ8C¤>C" ACă4CT@Cd0D"8DR 4CT@CT2DÄ0Ð
/// </summary>Ð
/// <summary>Ð
/// A 0Ct>C$KC´ :C´0D A CD;Dò 2D 5DR 4D 5C$>C$8CD=D´E D ?D 0C$>Dt=C,,:Că2 (MDM).Ð
/// A,,ACô>C´LCtCCTB Cô0D$BCT@CÔ 5%E 4C´O Cô5D 5CD0Dt8 D BD >C4> D$8Cô8Ct8D >C$0CÔ=D´E CÔ0C$8C40Dd8Că=CÔKDR A
/// </summary>Ð
/// <typeparam name="T">B$8Cò :Că=C¤@CTBCÔ>C4> CD>CĂ5CÔ=Că3Că >C JCT:D$0 (CÔ0D ;CT4CÔ8C¤0).</typeparam>Ð
public abstract class ReferenceTreeBase<T> : ReferenceBase, ITreeNode<T> where T : classÐ
{Ð
    /// <summary>Ð
    /// A,,4CT=D$8DD8C¤0D$>D @Că4C,,BCT;DÄAC¤>C4> D47C´0 C" !B4 ABí
    /// </summary>Ð
    public Guid? ParentId { get; set; }Ð
Ð
    /// <summary>Ð
    /// AÔ0C$8C40Dd8Că=CÔ>CR AC$>C"AD$2Că AD KC´:C, =C @Că4C,,BCT;DÄAC¤8C´ >C JCT:D" 2 Că?CT@C BC,,2CÔ>C´ ?C <
    /// Aô5D 5Că?D 5CD5C´OCTBD 0 C¤0C¤ f-´GV Â 2 C¤>CÔ5Dt=D´E C¤;C AD 0DR 4C´O Cô>CD4CT@Cd:C, Æ $´ Æö F-ærí
    /// </summary>Ð
    public virtual T? Parent { get; set; }Ð
Ð
    /// <summary>Ð
    /// A¤>C´;CT:Dd8Dò 4CăGCT@CÔ8DR MC´5CĂ5CÔBCă2 (Cô>CDGC,,=CT=CÔKDR Cct;Că2).Ð
    /// </summary>Ð
    public virtual ICollection<T> Children { get; set; } = new List<T>();Ð
}
</file>

<file path="Domain/ReferenceTreeBase/ReferenceTreeBaseValidator.cs">
using FluentValidation;Ð
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.Domain;Ð
Ð
/// <summary>Ð
/// B4=C,,2CT@D 0C´LCÔKC´ 2C ;C,,4C BCă@ CD;Dò 2D 5DR :C´0D ACă2, CÔ0D ;CT4D45CĂKDR >D" &Vfw&Væ6UG&VT& 6]
/// </summary>Ð
public abstract class ReferenceTreeBaseValidator<T> : ReferenceBaseValidator<T>Ð

```

```

where T : ReferenceBase, ITreeNode<T>
{
    protected ReferenceTreeBaseValidator() : base()
    {
        // A0@Cä2CT@C00 C00 D 0CÄ>Dd8D$8D >C$0C08CR „>C JCT:D" =CR <Cä6CTB D4:C 7D´2C BDÄ =C ACT1D0 :C : C00
        RuleFor(x => x.ParentId)
            .Must((model, parentId) => parentId != model.Id)
            .WithMessage("Aä1D05C0B C05 CÄ>Cd5D" 1D´BDÄ @Cä4C„BCT;CT< D 0CÄ>CÄC D 5C 5");
    }
    // A 0Ct>C$0D0 ?D >C$5D :C DCä@CÄ0D$0 GUID0
    RuleFor(x => x.ParentId)
        .NotEqual(Guid.Empty)
        .When(x => x.ParentId.HasValue)
        .WithMessage("B >CD8D$5C´LD :C„9 C„4CT=D$8DD8C00D$>D =CR <Cä6CTB C KD$L C0CD BD´< GUID");
}
}
</file>

<file path="Promatis.Net.UI/_Imports.razor">
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using MudBlazor
@using Promatis.Net.UI.Components.Workspaces
@using Promatis.Net.Domain.Interface
</file>

<file path="Promatis.Net.UI/Components/ActionContext/CrudWorkspaceContext.cs">
using MudBlazor;
using Promatis.Net.Service;
using Promatis.Net.UI.Controls;
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// Aä1Cä1D"5C0=D´9 C 0Ct>C$KC´ :Cä=D$5C0AD" 4C´0 D BC =CD0D BC0KDR 5%TB0MC0@C =Cä2 C0;C BDD>D <D²i
    /// A0>C´=CäAD$LDä 0C$BCä<C BC„7C„@D45D" DCä=Cä2D4N Ct0C4@D47C0C CD0C0=D´E C00 C 0Ct5 IBaseService.
    /// </summary>
    /// <typeparam name="TEntity">B$8C0 4Cä<CT=C0>C´ AD4IC0>D BC, „4Cä;Cd5C0 1D´BDÄ :C´0D ACä< D :Cä=D BD CC0BCä
    /// <typeparam name="TKey">B$8C0 ?CT@C$8Dt=Cä3Cä :C´NDt0 D CD"=CäAD$8 (Guid, int, string C, B.CBä´äÄ=G- W &
    public abstract class CrudWorkspaceContext<TEntity, TKey> : WorkspaceActionContext,
    IHasSelectedData<TEntity>
    {
        where TEntity : class, new()
        where TKey : notnull
    {
        private readonly IBaseService<TEntity, TKey> _baseService;
        private TEntity? _selectedData;
        private bool _isLoading;

        // A„=DD@C AD$@D4:D$CD =D´5 C0>CÄ?Cä=CT=D$K CD0C0=D´E C„=D BC =D 0 DD>D <D½
        public UiDataBroker<TEntity, GridState<TEntity>, GridData<TEntity>> Broker { get; }
        public UIiOzuCache<TEntity> OzuCache { get; }

        // B 5C ;C„7C FC„0 C0>C0BD 0C0BC DCä:D4AC AD$@Cä:C•
        public TEntity? SelectedData
        {
            get => _selectedData;
            set
            {
                if (_selectedData != value)
                {
                    _selectedData = value;
                    OnContextUpdated?.Invoke();
                    NotifyStateChanged();
                }
            }
        }

        public Action? OnContextUpdated { get; set; }

        /// <summary>
        /// BD;C 3 DD>C0>C$>C´ 7C 3D Cct:C, 4C´0 CäBCä1D 0Cd5C08D0 AC05C´5D$>C00 C„;C, :D CD$8C´:C, 2 GridPage.
        /// </summary>
        public bool IsLoading => _isLoading;

        protected CrudWorkspaceContext(IBaseService<TEntity, TKey> baseService, bool isInMemoryMode)

```

```

{
    _baseService = baseService ?? throw new ArgumentNullException(nameof(baseService));
    OzuCache = new UiOzuCache<TEntity>();
    Broker = new UiDataBroker<TEntity, GridState<TEntity>, GridData<TEntity>>();

    // AD5C;C @C BC,,2C0> C00C0>C'=D05CÂ BD4;C 0D AD$0C04C @D$=D'<C, 5%TB0:C0>C0:C <C, OCD@C
    AddControl(new CreateEntityButton<TEntity>());
    AddControl(new EditEntityButton<TEntity>());
    AddControl(new DeleteEntityButton<TEntity>());
    AddControl(new ToolbarDivider());

    // A0 B "B A" A Aä AT AT A,,/: A$KC >D 8D BCäGC08C=0 CD0C0=D'E C00 CäAC0>C$5 D 5D,,5C08D0 ?D >C4@C
    if (IsInMemoryMode)
    {
        Broker.ConfigureInMemoryMode(OzuCache, EvaluateGridStateInMemory);
    }
    else
    {
        Broker.ConfigureServerMode(FetchGridDataFromServerAsync);
    }
}

/// <summary>
/// B$>Dt:C ;CT=C,,2Cä9, C 5Ct>C00D =Cä9 DD>C0>C$>C' 7C 3D CCt:C, ?CT@C$8Dt=D'E CD0C0=D'E.
/// A$KCtKC$0CTBD 0 D BD 0C08Dd5C' ?D 8 C,,=C,,FC,,0C'8Ct0Dd8C, 8 C40D 0C0BC,,@Cä2C =C0> C05 C ;Cä:C,,@D45D" T
/// </summary>
public async Task LoadInitialDataAsync()
{
    if (!Broker.IsInMemoryMode) return; // A" ACT@C$5D =Cä< D 5Cd8CÄ5 DD>C0>C$>CR =C ?Cä;C05C08CR :D0HC

    try
    {
        _isLoading = true;
        NotifyStateChanged(); // A$:C'NDt0CT< C,,=CD8C=0D$>D 7C 3D CCt:C, 2 D$0C ;C,,FC]

        // A$KC=0Dt8C$0CT< C$5D L D ?D 0C$>Dt=C,,: Dt5D 5Cr 2C H D 5C ;DÄ=D'9 CÄ5D$>CB vWD ÄÄ 7-æ2, •
        List<TEntity> serverItems = await _baseService.GetAllAsync();
        OzuCache.InMemoryItems = serverItems ?? new List<TEntity>();
    }
    finally
    {
        _isLoading = false;
        NotifyStateChanged(); // A$KC;DäGC 5CÄ 8C04C,,:C BCä@, D$0C ;C,,FC ?C'0C$=Cä >D$>C @C 6C 5D" AD$@
    }
}

// --- B "A AD B "A0+AR AT"Aä B²0 B'!A' B$ A' AD B0 B A² B ---
private GridData<TEntity> EvaluateGridStateInMemory(GridState<TEntity> state, List<TEntity> inMemoryList)
{
    // A00C48C00Dd8D0 ?C'>D :Cä3Cä AC08D :C Ä"ä <CTBCä4C <C, 2 C00CÄ0D$8 Aä B2 :Cä=C@CTBC0>C' 2C;C 4C
    return new GridData<TEntity>
    {
        Items = inMemoryList.Skip(state.Page * state.PageSize).Take(state.PageSize).ToList(),
        TotalItems = inMemoryList.Count
    };
}

private async Task<GridData<TEntity>> FetchGridDataFromServerAsync(GridState<TEntity> state, Cancellation token ct)
{
    // A0@D0<Cä9 D 5D 2CT@C0KC' @CT6C,,<: C0@C, :C 6CD>CÄ 7C ?D >D 5 C,,4CT< C" !B4 AB ,,=C ?D 8CÄ5D Ä 4C'0
    List<TEntity> serverData = await _baseService.GetAllAsync();
    return new GridData<TEntity>
    {
        Items = serverData.Skip(state.Page * state.PageSize).Take(state.PageSize).ToList(),
        TotalItems = serverData.Count
    };
}
}
</file>

```

```

<file path="Promatis.Net.UI/Components/ActionContext/IWorkspaceActionContext.cs">
namespace Promatis.Net.UI.Components;

```

```

/// <summary>D
/// AT4C,,=D´9 C¤>CÔBD 0C¤B C¤>CÔBCT:D BC @C 1CäGCT9 Cä1C´0D BC,Â CCô@C 2C´ODäIC,,9 C45Cä<CTBD 8CT9 Cô0D$8 Ct>
/// D >D BC 2Cä< CD>D BD4?CÔKDR T´Ô4CT9D BC$8C´ 8 D$5C¤CD"8CÂ 2D´4CT;CT=C,,5CÂ 4C =CÔKDR =C MC¤@C =CRí
/// </summary>D
public interface IWorkspaceActionContextD
{D
    // --- B Aä B4 B A$ AT A,,/ A¤ AÄ Aä AT B$ AÄ A, A AÔ+AÄ ---D
D
    /// <summary>D
    /// A¤>C´;CT:Dd8Dò 8CÔBCT@C :D$8C$=D´E DÔ;CT<CT=D$>C" CCô@C 2C´5CÔ8Dò „:CÔ>Cô>C$Â DC,,;DÄBD >C"Â ?CT@CT:C´
    /// </summary>D
    IEnumerable<IUIControl> Controls { get; }D
D
    event Action? OnContextStateChanged;D
    void NotifyStateChanged();D
D
    // --- Aô B AÄ B$ B² !B$ A´ At Bd A, A4 Aä AT"B A, R Aä ---D
    int PaperElevation { get; }D
    string PaperClass { get; }D
    string WorkspaceHeight { get; }D
    string TopZoneHeight { get; }D
    string BottomZoneHeight { get; }D
    string LeftZoneWidth { get; }D
    string RightZoneWidth { get; }D
D
    bool IsTopZoneCollapsed { get; }D
    bool IsBottomZoneCollapsed { get; }D
    bool IsLeftZoneCollapsed { get; }D
    bool IsRightZoneCollapsed { get; }D
}
</file>

```

```

<file path="Promatis.Net.UI/Components/ActionContext/WorkspaceActionContext.cs">
namespace Promatis.Net.UI.Components;D
D
/// <summary>D
/// A 0Ct>C$KC´ :Cä=D$5C¤AD" ;Dä1Cä9 D 0C >Dt5C´ >C ;C AD$8 (C$:C´0CD:C,' 2 D 8D BCT<CRí
/// Aä1D"8C´ 7CÔ0CÄ5CÔ0D$5C´L CD;Dò <CÔ5CÄ>D ECT<, D ?D 0C$>Dt=C,,:Cä2, CD0D,,1Cä@CD>C" 8 C4@C DC,,:Cä2.D
/// </summary>D
public class WorkspaceActionContext : IWorkspaceActionContextD
{D
    protected readonly List<IUIControl> _controls = new();D
D
    public IEnumerable<IUIControl> Controls => _controls;D
D
    public event Action? OnContextStateChanged;D
D
    public void NotifyStateChanged() => OnContextStateChanged?.Invoke();D
D
    // --- AD BD A´"AÔ Bò A !B$ Aä A¤ A4 Aä AT"B A, B "A,, AT ---D
    public virtual int PaperElevation => 1;D
    public virtual string PaperClass => "pa-4 d-flex flex-column flex-grow-1 w-100 h-100";D
    public virtual string WorkspaceHeight => "100%";D
    public virtual string TopZoneHeight=> "auto";D
    public virtual string BottomZoneHeight => "auto";D
    public virtual string LeftZoneWidth => "250px";D
    public virtual string RightZoneWidth => "300px";D
D
    public virtual bool IsTopZoneCollapsed => false;D
    public virtual bool IsBottomZoneCollapsed => false;D
    public virtual bool IsLeftZoneCollapsed => false;D
    public virtual bool IsRightZoneCollapsed => false;D
D
    // --- AÄ B$ AD+ B4 B A$ AT A,,/ Aô AÔ A´,Bä A´/ Aô B$ AÄ Aä ---D
    protected void AddControl(IUIControl control)D
    {D
        _controls.Add(control);D
        NotifyStateChanged();D
    }D
D
    protected void RemoveControl(string controlId)D
    {D
        _controls.RemoveAll(c => c.Id == controlId);D
        NotifyStateChanged();D
    }D
}

```

</file>

<file path="Promatis.Net.UI/Components/EditDialogs/DialogTab.razor">

@* A?CÄ?Cä=CT=D" =CR @CT=CD5D 8D" ...DÔÂ AC <, Cä= D ;D46C,,B CD5C;C @C BC,,2CÔKCÂ >Cô8D 0D\$5C'5CÂ 4C'0 EditDia
@code {D

[Parameter] public required string Title { get; set; }D
[Parameter] public string? Icon { get; set; }D
[Parameter] public required RenderFragment Content { get; set; }D

}

</file>

<file path="Promatis.Net.UI/Components/EditDialogs/DialogTab.razor.cs">

using Microsoft.AspNetCore.Components;D

D

namespace Promatis.Net.UI.Components.EditDialogs;D

D

public partial class DialogTab : ComponentBaseD

{D

[CascadingParameter] protected EditDialog ParentDialog { get; set; } = null!;D

D

protected override void OnInitialized()D

{D

base.OnInitialized();D

D

if (ParentDialog != null)D

{D

ParentDialog.RegisterTab(this); // A00D\$8C\$=Cä >D\$4C 5CÂ ACT1D0 =C 2CT@DR 2 C>C';CT:Dd8Dâ @Cä4C,,

}D

}D

}

</file>

<file path="Promatis.Net.UI/Components/EditDialogs/EditDialog.razor">

@inherits Microsoft.AspNetCore.Components.ComponentBaseD

D

<MudDialog>D

<TitleContent>D

<MudText Typo="Typo.h6">D

<MudIcon Icon="@{(IsNew ? Icons.Material.Filled.Add : Icons.Material.Filled.Edit)"

Class="mr-2 mb-n1" />D

@TitleD

</MudText>D

</TitleContent>D

<DialogContent>D

@* B :D KD\$K' AC >D IC,,: C\$:C'0CD>C 8Cr 4CäGCT@C05C4> C>C0BCT=D\$0 *@D

<CascadingValue Value="this" IsFixed="true">D

@TabsD

</CascadingValue>D

D

<MudForm @ref="_form"D

Model="@Model"D

Validation="@{async (object m) => await ExecuteFluentValidationAsync(m)}"D

ValidationDelay="200">D

D

@* A,,!A0 A A' A0 : A\$=CTHC08C' :Cä=D\$5C'=CT@ Cd5D BC> CD5D 6C,,B C\$KD >D\$C Cä:C00 DD8C=AC,,@Cä2C

<div class="d-flex flex-column pt-2" style="min-height: 340px; max-height: 65vh; min-

width: 0;">D

@if (_isProcessing)D

{D

<div class="d-flex justify-center my-4">D

<MudProgressCircular Color="Color.Primary" Indeterminate="true" />D

</div>D

}D

elseD

{D

@* A B\$ AÄ B\$ A B A0 AT A,, A4 : ATAC'8 C\$:C'0CD:C >CD=C r 4C 5CÂ 5C' AC@Cä;C'1C @

@if (_collectedTabs.Count == 1)D

{D

<div style="max-height: 60vh; overflow-y: auto; padding-right: 4px;">D

@_collectedTabs.First().ContentD

</div>D

}D

else if (_collectedTabs.Count > 1)D

{D

@* A,,!A0 A A' A0 : At0CD0CT< PanelStyle CD;D0 =C <CT@D\$2Cä DC,,:D 8D >C\$0C0=Cä3Cä AC

<MudTabs Elevation="0"D

```

        Rounded="true"@
        ApplyEffectsToInnerContainer="true"@
        PanelClass="pa-4"@
        PanelStyle="max-height: 50vh; overflow-y: auto; padding-right:
4px;">@
        @foreach (DialogTab tab in _collectedTabs)@
        {
            <MudTabPanel Text="@tab.Title" Icon="@tab.Icon">@
                @tab.Content@
            </MudTabPanel>@
        }@
    </MudTabs>@
}
</div>@
</MudForm>@
</DialogContent>@
<DialogActions>@
    <MudButton Variant="Variant.Text" OnClick="Cancel" Disabled="_isProcessing">AäBCÄ5CÔ0</MudButton>@
    <MudButton Variant="Variant.Filled" Color="Color.Primary" OnClick="Submit"
Disabled="_isProcessing">@
        @if (_isProcessing)@
        {
            <MudProgressCircular Class="ms-n1" Size="Size.Small" Indeterminate="true" />@
            <MudText Class="ms-2">B >DT@C =CT=C,,5...</MudText>@
        }@
        else@
        {
            <MudText>B >DT@C =C,,BDÄÄô×VEFW†Cí
        }@
    </MudButton>@
</DialogActions>@
</MudDialog>
</file>

```

```

<file path="Promatis.Net.UI/Components/EditDialogs/EditDialog.razor.cs">
using FluentValidation;@
using FluentValidation.Results;@
using Microsoft.AspNetCore.Components;@
using MudBlazor;@
using Severity = MudBlazor.Severity;@
@
@
namespace Promatis.Net.UI.Components.EditDialogs;@
@
public partial class EditDialog : ComponentBase@
{
    protected MudForm _form = null!;@
    protected bool _isProcessing;@
    protected readonly List<DialogTab> _collectedTabs = [];@
@
    [CascadingParameter] protected IMudDialogInstance MudDialog { get; set; } = null!;@
@
    [Parameter] public string Title { get; set; } = "B 5CD0C▯BC,,@Cä2C =C,,5";@
    [Parameter] public bool IsNew { get; set; }@
    [Parameter] public object Model { get; set; } = null!;@
    [Parameter] public IValidator Validator { get; set; } = null!;@
    [Parameter] public Func<Task> OnSaveAction { get; set; } = null!;@
@
    /// <summary>@
    /// B NCD0 Cö@C,,:C´0CD=Cä9 D 0Ct@C 1CäBDt8C¢ 4CT:C´0D 0D$8C$=Cä ?C,,HCTB D$5C48 <DialogTab>@
    /// </summary>@
    [Parameter] public RenderFragment? Tabs { get; set; }@
@
    [Inject] protected ISnackbar Snackbar { get; set; } = null!;@
@
    /// <summary>@
    /// AÄ5D$>CB 4C´0 D 5C48D BD 0Dd8C, 2C▯;C 4Cä:, C$KctKC$0CT<D´9 CD>Dt5D =C,,<C, :Cä<Cö>C05C0BC <C, F- ÆöuF
    /// </summary>@
    public void RegisterTab(DialogTab tab)@
    {
        if (!_collectedTabs.Contains(tab))@
        {
            _collectedTabs.Add(tab);@
            StateHasChanged();@
        }@
    }
}

```

```

    }
    protected void Cancel() => MudDialog.Cancel();
    protected async Task<IEnumerable<string>> ExecuteFluentValidationAsync(object model)
    {
        if (model is not Domain.DomainObject || Validator == null) return Array.Empty<string>();
        var context = new ValidationContext<object>(model);
        ValidationResult result = await Validator.ValidateAsync(context);
        return result.IsValid ? Array.Empty<string>() : result.Errors.Select(e => e.ErrorMessage);
    }
    protected async Task Submit()
    {
        await _form.ValidateAsync();
        if (!_form.IsValid)
        {
            Snackbar.Add("Aô>CdθC´CC"AD$, C,,ACô@C 2DÄBCR >D,,8C :C, 2 DD>D <CR ?CT@CT4 D >DT@C =CT=C,,5CÂâ"Â 6
            return;
        }
        _isProcessing = true;
        StateHasChanged();
        try
        {
            if (OnSaveAction != null) await OnSaveAction();
            MudDialog.Close(DialogResult.Ok(Model));
        }
        catch (Exception ex)
        {
            string message = ex.InnerException?.Message ?? ex.Message;
            Snackbar.Add($"AäHC,,1Cθ D >DT@C =CT=C,,0: {message}", Severity.Error);
        }
        finally
        {
            _isProcessing = false;
            StateHasChanged();
        }
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor">
@inherits RenderBase
<MudTooltip Text="@((Control.Tooltip ?? Control.Title)" Arrow="true" Placement="Placement.Bottom">
    <MudButton Variant="Variant.Text"
        StartIcon="@((Control.IsRunning ? null : Control.Icon)"
        Disabled="@(!Control.IsEnabled || Control.IsRunning || !
Control.IsEnabledForData(CurrentSelectedData))"
        Color="Color.Inherit"
        OnClick="@HandleClickAsync">
    @if (Control.IsRunning)
    {
        <MudProgressCircular Class="ms-n1 me-2" Size="Size.Small" Indeterminate="true"
Color="Color.Inherit" />
    }
    @Control.Title
</MudButton>
</MudTooltip>
</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor.cs">
using Microsoft.AspNetCore.Components;
namespace Promatis.Net.UI.Components.ElementRenderBase;
public partial class ButtonRenderBase : RenderBase
{
    protected async Task HandleClickAsync()
    {

```



```

        await Control.TriggerAsync(CurrentSelectedData);
    }
}
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor">
@inherits RenderBase
{
<MudDateRangePicker Label="@Control.Title"
    DateRange="@GetDateRange()"
    DateRangeChanged="@HandleDateRangeChangedAsync"
    Disabled="@(!Control.IsEnabled || Control.IsRunning)"
    Margin="Margin.Dense"
    DateFormat="dd.MM.yyyy"
    Variant="Variant.Outlined"
    Style="max-width: 260px;" />

</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor.cs">
using MudBlazor;
{
namespace Promatis.Net.UI.Components.ElementRenderBase;
{
public partial class DateRangeRenderBase : RenderBase
{
    /// <summary>
    /// A 5Ct>C00D =Câ 8Ct2C'5C=0CTB Că1D=5C=B DateRange C,,7 CÄ>CD5C'8 C= >C0BD >C'0 Dt5D 5Cr 8C0BCT@DD5C"A-D
    /// </summary>
    protected DateRange? GetDateRange()
    {
        if (Control is IHasValue valueProvider)
        {
            return valueProvider.Value as DateRange;
        }
        return null;
    }
}
{
    /// <summary>
    /// At0C08D KC$0CTB C0>C$>CR 7C00Dt5C08CR 2D'1D 0C0=Că3Că 4C,,0C00Ct>C00 CD0D" 2 CÄ>CD5C'L C, 7C ?D4AC=0CT
    /// </summary>
    protected async Task HandleDateRangeChangedAsync(DateRange? newRange)
    {
        if (Control is IHasValue valueProvider)
        {
            valueProvider.Value = newRange;
            await Control.TriggerAsync(CurrentSelectedData);
        }
    }
}
}
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor">
@inherits RenderBase
@using MudBlazor
{
<MudDivider Vertical="true" FlexItem="true" Class="my-2mx-1" />

</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor.cs">
namespace Promatis.Net.UI.Components.ElementRenderBase;
{
public partial class DividerRenderBase : RenderBase
{
}
}
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/RenderBase.cs">
using Microsoft.AspNetCore.Components;
{
namespace Promatis.Net.UI.Components.ElementRenderBase;
{
    /// <summary>
    /// A 1D BD 0C=BC0KC' 1C 7Că2D'9 C= >CÄ?Că=CT=D" 4C'0 C$ACTE Razor-D 5C04CT@CT@Că2 D0;CT<CT=D$>C" CC0@C 2C'5C0
    /// </summary>
    public abstract class RenderBase : ComponentBase, IDisposable
    {

```

```

    /// <summary>Ð
    /// A 8Ct=CTA-CÄ>CD5C´L DÔ;CT<CT=D$0 D4?D 0C$;CT=C,,0, C05D 5CD0C$0CT<C 0 C,,7 DynamicComponent.Ð
    /// </summary>Ð
    [Parameter]Ð
    public UIControl Control { get; set; } = null!;Ð

Ð
    /// <summary>Ð
    /// A0D :C 4C0KC´ :Cä=D$5C0AD" ECä;D BC @C 1CäGCT9 Cä1C´0D BC,i
    /// </summary>Ð
    [CascadingParameter]Ð
    protected IWorkspaceActionContext? ActionContext { get; set; }Ð

Ð
    /// <summary>Ð
    /// A 5Ct>C00D =Cä 8Ct2C´5C00CTB D$5C0CD"8CR 2D´4CT;CT=C0KCR 4C =C0KCR 8Cr :Cä=D$5C0AD$0 DD>D <D²i
    /// </summary>Ð
    protected object? CurrentSelectedData => GetSelectedDataFromContext();Ð

Ð
    protected override void OnInitialized()Ð
    {Ð
        base.OnInitialized();Ð

Ð
        // B 5C48D BD 8D CCT< D 5C :D$8C$=Cä5 Cä1C0>C$;CT=C,,5 C,,=D$5D DCT9D 0 C0@C, ACÄ5C05 D >D BCä0C08D0 1C
        Control.OnStateChanged += HandleStateChanged;Ð
    }Ð

Ð
    private void HandleStateChanged() => InvokeAsync(StateHasChanged);Ð

Ð
    private object? GetSelectedDataFromContext()Ð
    {Ð
        if (ActionContext == null) return null;Ð

Ð
        Type? interfaceType = ActionContext.GetType().GetInterface("IHasSelectedData`1");Ð
        return interfaceType?.GetProperty("SelectedData")?.GetValue(ActionContext);Ð
    }Ð

Ð
    public virtual void Dispose() => Control.OnStateChanged -= HandleStateChanged;Ð
}
</file>

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor">

@inherits RenderBaseÐ

<MudSelect T="string"Ð

Value="@GetControlValue()"Ð

ValueChanged="@HandleValueChangedAsync"Ð

Disabled="@(!Control.IsEnabled || Control.IsRunning)"Ð

Label="@Control.Title"Ð

Margin="Margin.Dense"Ð

Variant="Variant.Outlined"Ð

Style="min-width: 160px; max-width: 250px;">Ð

Ð

@foreach (string option in GetOptions())Ð

{Ð

<MudSelectItem T="string" Value="@option">@option</MudSelectItem>Ð

}Ð

</MudSelect>

</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor.cs">

namespace Promatis.Net.UI.Components.ElementRenderBase;Ð

Ð

public partial class SelectRenderBase : RenderBaseÐ

{Ð

protected string GetControlValue()Ð

{Ð

if (Control is IHasValue valueProvider && valueProvider.Value != null)Ð

{Ð

return valueProvider.Value.ToString() ?? string.Empty;Ð

}Ð

return string.Empty;Ð

}Ð

Ð

protected async Task HandleValueChangedAsync(string newValue)Ð

{Ð

if (Control is IHasValue valueProvider)Ð

{Ð

valueProvider.Value = newValue;Ð

```

        await Control.TriggerAsync(CurrentSelectedData);
    }
}

protected IEnumerable<string> GetOptions()
{
    if (Control is IHasOptions optionsProvider)
    {
        return optionsProvider.Options;
    }
    return Array.Empty<string>();
}
}
</file>

<file path="Promatis.Net.UI/Components/UIControls/BaseUiControl.cs">
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// A 0Ct>C$0Dò 0C AD$@C :Dd8Dò 4C`O C$ACTE DÔ;CT<CT=D$>C" CCô@C 2C`5CÔ8Dò ?C`0D$DCä@CÄK (C=Ca?Ca:, Dt5C=1Cä
    /// B 5C ;C,,7D45D" 8CÔDD 0D BD CC=BD4@CÔCDâ @D4BC,,=D2 8 Cô@CT4CäAD$0C$;Dô5D" <CäAD" : CD8CÔ0CÄ8Dt5D :Cä<D2 @C
    /// </summary>
    public abstract class BaseUiControl : IUiControl
    {
        private bool _isVisible = true;
        private bool _isEnabled = true;
        private bool _isRunning;

        public abstract string Id { get; }
        public abstract Type ComponentType { get; }

        public virtual string? Title { get; init; }
        public virtual string? Icon { get; init; }
        public virtual string? Tooltip { get; init; }

        public Dictionary<string, object> ComponentParameters { get; } = new();

        public bool IsVisible
        {
            get => _isVisible;
            set { if (_isVisible != value) { _isVisible = value; NotifyStateChanged(); } }
        }

        public bool IsEnabled
        {
            get => _isEnabled;
            set { if (_isEnabled != value) { _isEnabled = value; NotifyStateChanged(); } }
        }

        public bool IsRunning => _isRunning;

        public event Action? OnStateChanged;

        protected BaseUiControl()
        {
            ComponentParameters.Add("Control", this);
        }

        protected void NotifyStateChanged() => OnStateChanged?.Invoke();

        public virtual bool IsEnabledForData(object? targetData) => true;

        public async Task TriggerAsync(object? targetData)
        {
            if (!_isEnabled || _isRunning || !IsEnabledForData(targetData)) return;

            try
            {
                _isRunning = true;
                NotifyStateChanged();
                await HandleTriggerAsync(targetData);
            }
            finally
            {
                _isRunning = false;
                NotifyStateChanged();
            }
        }
    }
}

```

```

    }
}
protected abstract Task HandleTriggerAsync(object? targetData);
}
</file>

<file path="Promatis.Net.UI/Components/UIControls/IHasOptions.cs">
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// A<C>BD 0C=B CD;Dò MC´5CÄ5CÔBCä2 D4?D 0C$;CT=C,,0, Cä1C´0CD0DäIC,,E D ?C,,AC>CÄ 4CäAD$CCô=D´E Cä?Dd8C´ 2D´1
    /// </summary>
    public interface IHasOptions : IUiControl
    {
        IEnumerable<string> Options { get; }
    }
}
</file>

<file path="Promatis.Net.UI/Components/UIControls/IHasValue.cs">
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// A<C>BD 0C=B CD;Dò MC´5CÄ5CÔBCä2 D4?D 0C$;CT=C,,0, C>D$>D KCR ED 0C0D" 8 C,,7CÄ5C0DäB D 0C0BC 9CÄ07C00Dt
    /// </summary>
    public interface IHasValue : IUiControl
    {
        object? Value { get; set; }
    }
}
</file>

<file path="Promatis.Net.UI/Components/UIControls/IUiControl.cs">
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// B4=C,,2CT@D 0C´LC0KC´ :Cä=D$@C :D" MC´5CÄ5CÔBC CCô@C 2C´5C08Dò ,,C0>Cô:C Ä GCT:C >C=A, C$KCô0CD0DäIC,,9 D
    /// </summary>
    public interface IUiControl
    {
        string Id { get; }
        Type ComponentType { get; }
        Dictionary<string, object> ComponentParameters { get; }

        string? Title { get; }
        string? Icon { get; }
        string? Tooltip { get; }

        bool IsVisible { get; set; }
        bool IsEnabled { get; set; }
        bool IsRunning { get; }

        bool IsEnabledForData(object? targetData);
        Task TriggerAsync(object? targetData);

        event Action? OnStateChanged;
    }
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/GridPage.razor">
@typeparam TEntity where TEntity : class
{
    <MudDataGrid @ref="_mudGrid"
        ServerData="@ServerDataProvider"
        T="TEntity"
        SelectedItem="@SelectedRow"
        SelectedItemChanged="@OnSelectedRowChanged"
        RowStyleFunc="@FormatSelectedRowStyle"
        SelectOnRowClick="true"
        Hover="true"
        Dense="true"
        Bordered="true"
        Elevation="0"
        FixedHeader="true"
        Height="100%"
        Class="flex-grow-1 w-100 cursor-pointer">

```

```

        <Columns>@GridColumns
    </Columns>
</MudDataGrid>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/GridPage.razor.cs">
using Microsoft.AspNetCore.Components;
using MudBlazor;

namespace Promatis.Net.UI.Components.Workspaces;

public partial class GridPage<TEntity> : ComponentBase where TEntity : class
{
    private MudDataGrid<TEntity>? _mudGrid;

    [CascadingParameter]
    protected IWorkspaceActionContext? ActionContext { get; set; }

    /// <summary>
    /// BD>C$>CR ACT@C$D =Cä5 D >C KD$8CR 7C 3D Cct:C, 4C =CÔKDR „<C ?C„BD 0 CÔ0 MudBlazor ServerData).
    /// </summary>
    [Parameter]
    public Func<GridState<TEntity>, CancellationToken, Task<GridData<TEntity>>>? ServerDataProvider { get; set; }

    [Parameter]
    public RenderFragment? GridColumns { get; set; }

    protected TEntity? SelectedRow { get; set; }

    /// <summary>
    /// AôCC ;C„GCÔKC' <CTBCä4, Cö>Ct2Cä;DôND"8C' AD$@C =C„FC <-C$;C 4CT;DÄFC < Cö@C„=D44C„BCT;DÄ=Cä >C =Cä2C
    /// </summary>
    public Task ReloadServerData()
    {
        return _mudGrid != null ? _mudGrid.ReloadServerData() : Task.CompletedTask;
    }

    protected string FormatSelectedRowStyle(TEntity item, int rowNumber)
        => item == SelectedRow ? "background-color: var(--mud-palette-action-disabled-background);
font-weight: 500;" : string.Empty;

    protected void OnSelectedRowChanged(TEntity? newSelection)
    {
        if (newSelection == null && SelectedRow != null) return;

        SelectedRow = newSelection;

        if (ActionContext is IHasSelectedData<TEntity> bindableContext)
        {
            bindableContext.SelectedData = newSelection;
            bindableContext.OnContextUpdated?.Invoke();
        }
    }
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/TreePage.razor">
@typeparam TEntity where TEntity : class

<MudTreeView T="TEntity"
    Items="@TreeItems"
    SelectedValue="@SelectedNode"
    SelectedValueChanged="@OnSelectedNodeChanged"
    Hover="true"
    Dense="true"
    SelectionMode="SelectionMode.SingleSelection"
    Class="flex-grow-1 w-100 h-100 overflow-y-auto">
    <ItemTemplate>
        <MudTreeViewItem T="TEntity"
            Value="@context.Value"
            Icon="@GetNodeIcon(context.Value)"
            Text="@GetNodeText(context.Value)"
            Items="@GetChildTreeItemData(context.Value)" />
    </ItemTemplate>
</MudTreeView>

```

```

    </ItemTemplate>@
</MudTreeView>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/TreePage.razor.cs">
using Microsoft.AspNetCore.Components;@
using MudBlazor;@
@
namespace Promatis.Net.UI.Components.Workspaces;@
@
public partial class TreePage<TEntity> : ComponentBase where TEntity : class@
{@
    [CascadingParameter]@
    protected IWorkspaceActionContext? ActionContext { get; set; }@
@
    [Parameter] public IEnumerable<TEntity>? RootItems { get; set; }@
    [Parameter] public Func<TEntity, IEnumerable<TEntity>>? ChildSelector { get; set; }@
    [Parameter] public Func<TEntity, string>? TextSelector { get; set; }@
    [Parameter] public Func<TEntity, string>? IconSelector { get; set; }@
@
    // A,,ACô>C´LctCCT< Dt8D BD´9 TEntity, D$0C¢ :C : Că3D 0C08Dt5C08CR 6Æ 72 CCd5 CD>CôCD :C 5D" çVÆÂ =C CD
    protected List<TreeItemData<TEntity>> TreeItems { get; set; } = new();@
    protected TEntity? SelectedNode { get; set; }@
@
    protected override void OnInitialized()@
    {@
        base.OnInitialized();@
@
        if (ActionContext is IHasSelectedData<TEntity> bindableContext)@
        {@
            SelectedNode = bindableContext.SelectedData;@
        }@
    }@
@
    protected override void OnParametersSet()@
    {@
        base.OnParametersSet();@
@
        if (RootItems != null)@
        {@
            TreeItems = RootItems@
                .Where(item => item != null)@
                .Select(item => new TreeItemData<TEntity> { Value = item })@
                .ToList();@
        }@
        else@
        {@
            TreeItems = new();@
        }@
    }@
@
    protected void OnSelectedNodeChanged(TEntity? newSelection)@
    {@
        SelectedNode = newSelection;@
@
        if (ActionContext is IHasSelectedData<TEntity> bindableContext)@
        {@
            bindableContext.SelectedData = newSelection;@
            bindableContext.OnContextUpdated?.Invoke();@
        }@
    }@
@
    // AD>C 0C$;Dô5CÂ ?D >C$5D :D2 =C çVÆÂÂ BC : C¢0C¢ @C =D$0C" <-Că1D¢5C¢B C" 6öçFW†Bâf çVR <Că6CTB CăBD CD
    protected string GetNodeText(TEntity? item) =>@
        item == null ? string.Empty : (TextSelector?.Invoke(item) ?? item.ToString()) ??
string.Empty);@
@
    protected string GetNodeIcon(TEntity? item) =>@
        item == null ? Icons.Material.Filled.Folder : (IconSelector?.Invoke(item) ??
Icons.Material.Filled.Folder);@
@
    protected List<TreeItemData<TEntity>> GetChildTreeItemData(TEntity? item)@
    {@
        if (item == null || ChildSelector == null) return new();@
@
        IEnumerable<TEntity>? children = ChildSelector.Invoke(item);@

```

```

        if (children == null) return new();
    }
    return children
        .Where(child => child != null)
        .Select(child => new TreeItemData<TEntity> { Value = child })
        .ToList();
}
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/UiToolbar.razor">
<div class="mud-toolbar mud-toolbar-dense gap-4 pa-0 w-100 align-center" style="display: flex; min-height: 48px;">
    @if (Controls != null)
    {
        @foreach (IUIControl control in Controls)
        {
            @if (!control.IsVisible) continue;

            <!-- AD8C00CÄ8Dt5D :C,,9 CÄ0D HC ;C,,=C2 :Cä<Cö>C05C0BC 8 C00D 0CÄ5D$@Cä2 C´NC >C4> D4@Cä2C00 (Cor
            <DynamicComponent Type="@control.ComponentType"
                Parameters="@control.ComponentParameters" />
        }
    }
</div>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/UiToolbar.razor.cs">
using System.Reflection;
using Microsoft.AspNetCore.Components;
namespace Promatis.Net.UI.Components.Workspaces;
public partial class UiToolbar : ComponentBase, IDisposable
{
    /// <summary>
    /// B ?C,,ACä: C0>C´8CÄ>D DC0KDR :Cä=D$@Cä;Cä2 CD;D0 >D$@C,,ACä2C=8 C00 C00C05C´8.D
    /// </summary>
    [Parameter]
    public IEnumerable<IUIControl>? Controls { get; set; }

    /// <summary>
    /// A=0D :C 4C0KC´ :Cä=D$5C=AD" ECä;D BC 4C´O CäBD ;CT6C,,2C =C,,0 D <CT=D² 2D´4CT;CT=C,,0 CD0C0=D´E.D
    /// </summary>
    [CascadingParameter]
    protected IWorkspaceActionContext? ActionContext { get; set; }

    /// <summary>
    /// A,,7C$;CT:C 5D" BCT:D4IC,,9 C$KCD5C´5C0=D´9 Cä1D=5C=B CD;D0 ?CT@CT4C GC, 2 C 8Ct=CTA-C´>C48C=C C= >C0BD
    /// </summary>
    protected object? CurrentSelectedData => GetSelectedDataFromContext();

    protected override void OnInitialized()
    {
        base.OnInitialized();

        SubscribeToSelectionChanged(true);

        if (Controls != null)
        {
            foreach (IUIControl control in Controls)
            {
                control.OnStateChanged += HandleStateChanged;
            }
        }
    }

    private void HandleStateChanged() => InvokeAsync(StateHasChanged);

    private object? GetSelectedDataFromContext()
    {
        if (ActionContext == null) return null;

        Type? interfaceType = ActionContext.GetType().GetInterface("IHasSelectedData`1");
        if (interfaceType != null)
        {

```

```

        return interfaceType.GetProperty("SelectedData")?.GetValue(ActionContext);
    }
    return null;
}

private void SubscribeToSelectionChanged(bool subscribe)
{
    if (ActionContext == null) return;

    Type? interfaceType = ActionContext.GetType().GetInterface("IHasSelectedData`1");
    if (interfaceType != null)
    {
        PropertyInfo? eventInfo = interfaceType.GetProperty("OnContextUpdated");
        if (eventInfo != null)
        {
            var currentDelegate = (Action?)eventInfo.GetValue(ActionContext);
            if (subscribe)
                currentDelegate += HandleStateChanged;
            else
                currentDelegate -= HandleStateChanged;

            eventInfo.SetValue(ActionContext, currentDelegate);
        }
    }
}

public void Dispose()
{
    SubscribeToSelectionChanged(false);

    if (Controls != null)
    {
        foreach (IUIControl control in Controls)
        {
            control.OnStateChanged -= HandleStateChanged;
        }
    }
}
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor">
<MudPaper Elevation="@GetPaperElevation()"
Class="@GetPaperClass()"
Style="@($"width: 100%; min-height: {GetWorkspaceHeight()}; overflow: hidden;")">
    <!-- 1. A$ B %A0/B0 Aä A „ A Aä A0/AT"B / B$ A´,A0 A0 A, A A„'A„ TOPCONTENT) -->
    @if (TopContent != null && !IsTopCollapsed())
    {
        <div class="workspace-zone-top w-100 mb-2"
            style="@($"height: {GetTopHeight()}; min-height: {GetTopHeight()}; max-height: {GetTopHeight()};")">
            @TopContent
        </div>
    }

    <!-- B AT A0 A´ !A´ A´ „ AT A / At A0 + A AD + A0 A A / At A0 ) -->
    <div class="d-flex flex-row align-stretch flex-grow-1 gap-4 w-100" style="min-height: 0;">

        <!-- 2. A´ A$ B0 Aä A „ A A„ A &A„/ / AD B A$ ) -->
        @if (LeftContent != null && !IsLeftCollapsed())
        {
            <div class="workspace-zone-left d-flex flex-column align-stretch justify-space-between"
                style="@($"width: {GetLeftWidth()}; min-width: {GetLeftWidth()}; max-width: {GetLeftWidth()};")">
                @LeftContent
            </div>
        }

        <!-- 3. Bd A0"B A´,A0 B0 Aä A „ Aä A¢ B AB 0 A0 Aä B %AT A ´ 00í
        <div class="workspace-zone-body flex-grow-1 h-100" style="min-width: 0; min-height: 0;">
            @if (BodyContent != null)
            {
                @BodyContent
            }
        </div>
    </div>

```



```

D
    <!-- 4. A0 A A / At A0 (A,, B AT B$ B !A$ A"!B$ ) -->D
    @if (RightContent != null && !IsRightCollapsed())D
    {D
        <div class="workspace-zone-right d-flex flex-column align-stretch justify-space-between"
            style="@($ "width: {GetRightWidth()}; min-width: {GetRightWidth()}; max-width:
{GetRightWidth()};")">D
            @RightContentD
        </div>D
    }D
D
</div>D
D
<!-- 5. A0 Ad B0/ At A0 (B "A "B4!-A B ' 00i
@if (BottomContent != null && !IsBottomCollapsed())D
{D
    <div class="workspace-zone-bottom w-100 mt-2"D
        style="@($ "height: {GetBottomHeight()}; min-height: {GetBottomHeight()}; max-height:
{GetBottomHeight()};")">D
        @BottomContentD
    </div>D
    }D
D
</MudPaper>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor.cs">
using Microsoft.AspNetCore.Components;D
D
namespace Promatis.Net.UI.Components.Workspaces;D
D
public partial class WorkspacePage : ComponentBase, IDisposable
{D
    /// <summary>D
    /// A0D :C 4C0KC' 8C0BCT@DD5C"A C0>C0BCT:D BC BCT:D4ICT9 D 0C >Dt5C' >C ;C AD$8.D
    /// </summary>D
    [CascadingParameter]D
    protected IWorkspaceActionContext? ActionContext { get; set; }D
D
    [Parameter] public RenderFragment? BodyContent { get; set; }D
    [Parameter] public RenderFragment? TopContent { get; set; }D
    [Parameter] public RenderFragment? BottomContent { get; set; }D
    [Parameter] public RenderFragment? LeftContent { get; set; }D
    [Parameter] public RenderFragment? RightContent { get; set; }D
D
    protected override void OnInitialized()D
    {D
        base.OnInitialized();D
D
        // A0>CD?C,,AC00 C00 D 5C :D$8C$=Cä5 Cä1C0>C$;CT=C,,5 C00D 0CÄ5D$@Cä2 C45Cä<CTBD 8C•
        if (ActionContext != null)D
        {D
            ActionContext.OnContextStateChanged += HandleContextStateChanged;D
        }D
D
    private void HandleContextStateChanged()D
    {D
        InvokeAsync(StateHasChanged);D
    }D
D
    public void Dispose()D
    {D
        if (ActionContext != null)D
        {D
            ActionContext.OnContextStateChanged -= HandleContextStateChanged;D
        }D
D
    // --- AÄ B$ AD+ B B 'AT"A A,, B4 A',A0 A4 B "A,, B0 Aä A' Ad A, 00Y
    protected int GetPaperElevation() => ActionContext?.PaperElevation ?? 1;D
    protected string GetPaperClass() => ActionContext?.PaperClass ?? "pa-4 d-flex flex-column flex-
grow-1 w-100 h-100";D
D
    // --- AÄ B$ AD+ B B 'AT"A AT AÄ B$ A,, A0 B A !A %Aä B "A 00Y
    protected string GetWorkspaceHeight() => ActionContext?.WorkspaceHeight ?? "100%";D

```

```

        protected string GetTopHeight() => ActionContext?.TopZoneHeight ?? "auto";
        protected string GetBottomHeight() => ActionContext?.BottomZoneHeight ?? "auto";
        protected string GetLeftWidth() => ActionContext?.LeftZoneWidth ?? "250px";
        protected string GetRightWidth() => ActionContext?.RightZoneWidth ?? "300px";
    }
}
</file>

<file path="Promatis.Net.UI/Controls/CreateEntityButton.cs">
using MudBlazor;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;

namespace Promatis.Net.UI.Controls;

/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 8C0DD 0D BD CC=BD4@C0KC' MC'5CÄ5C0B C=Ca?C=8 "B >Ct4C BDÄ" 4C'O D CD"=CäAD$5C' ;Dä1
/// </summary>
public class CreateEntityButton<TEntity> : BaseUiControl where TEntity : class, new()
{
    public override string Id => $"crud_create_{typeof(TEntity).Name.ToLower()}";
    public override Type ComponentType => typeof(ButtonRenderBase); // A0@C,,2D07C=0 C@ @CT=CD5D 5D C C=Ca?Cä
    public override string Title => "B >Ct4C BDÄ#½
    public override string Icon => Icons.Material.Filled.Add;
    public override string Tooltip => $"B >Ct4C BDÄ =Cä2D4N Ct0C08D L ({typeof(TEntity).Name})";

    public CreateEntityButton()
    {
        IsEnabled = true;
    }

    public override bool IsEnabledForData(object? targetData) => true;

    protected override Task HandleTriggerAsync(object? targetData)
    {
        // UI-Ct0C4;D4HC=0: C" 1D44D4ICT< Ct4CTADÄ 1D44CTB CäBC=@D'BC,,5 CD8C ;Cä3C VF-DF- Åö}
        return Task.CompletedTask;
    }
}
</file>

<file path="Promatis.Net.UI/Controls/DeleteEntityButton.cs">
using MudBlazor;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;

namespace Promatis.Net.UI.Controls;

/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 8C0DD 0D BD CC=BD4@C0KC' MC'5CÄ5C0B C=Ca?C=8 "B44C ;C,,BDÄ" 4C'O C$KC @C =C0>C' AD4I
/// </summary>
public class DeleteEntityButton<TEntity> : BaseUiControl where TEntity : class
{
    public override string Id => $"crud_delete_{typeof(TEntity).Name.ToLower()}";
    public override Type ComponentType => typeof(ButtonRenderBase);
    public override string Title => "B44C ;C,,BDÄ#½
    public override string Icon => Icons.Material.Filled.Delete;
    public override string Tooltip => "B44C ;C,,BDÄ 2D'1D 0C0=D4N Ct0C08D L";

    public override bool IsEnabledForData(object? targetData)
    {
        if (targetData is not TEntity typedEntity) return false;

        if (typedEntity is Promatis.Net.Domain.Interface.ISoftDeletable softDeletable)
        {
            return softDeletable.DeletedAt == null;
        }

        return true;
    }

    protected override Task HandleTriggerAsync(object? targetData)

```

```

    {
        if (targetData is TEntity typedEntity)
        {
            // UI-Ct0C4;D4HC=0 CD;D0 1D44D4ICT9 CăBCô@C 2C=8 C=CA0C04D² CCD0C'5C08D0 2 API0
        }
        return Task.CompletedTask;
    }
}
</file>

<file path="Promatis.Net.UI/Controls/EditEntityButton.cs">
using MudBlazor;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;

namespace Promatis.Net.UI.Controls;

/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 8C0DD 0D BD CC=BD4@C0KC' MC'5CĂ5C0B C=Că?C=8 "B 5CD0C=BC,,@Că2C BDÂ" 4C'O C$KC @C =C
/// </summary>
public class EditEntityButton<TEntity> : BaseUiControl where TEntity : class
{
    public override string Id => $"crud_edit_{typeof(TEntity).Name.ToLower()}";
    public override Type ComponentType => typeof(ButtonRenderBase);
    public override string Title => "B 5CD0C=BC,,@Că2C BDÂ#½
    public override string Icon => Icons.Material.Filled.Edit;
    public override string Tooltip => "B 5CD0C=BC,,@Că2C BDÂ 2D'1D 0C0=D4N Ct0C08D L";

    public override bool IsEnabledForData(object? targetData)
    {
        return targetData is TEntity;
    }

    protected override Task HandleTriggerAsync(object? targetData)
    {
        if (targetData is TEntity typedEntity)
        {
            // UI-Ct0C4;D4HC=0 CD;D0 1D44D4ICT3Că >D$:D KD$8D0 VF-DF- Åör A CD0C0=D'C, AD4IC0>D BC•
        }
        return Task.CompletedTask;
    }
}
</file>

<file path="Promatis.Net.UI/Controls/ToolbarDivider.cs">
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;

namespace Promatis.Net.UI.Controls;

/// <summary>
/// B 8D BCT<C0KC' MC'5CĂ5C0B C$5D BC,,:C ;DĂ=Că3Că @C 7CD5C'8D$5C'O, D 0Ct<CTIC 5CĂKC' <CT6CDC D0;CT<CT=D$0CĂ
/// </summary>
public class ToolbarDivider : BaseUiControl
{
    private readonly string _id = "core_toolbar_divider_" + Guid.NewGuid().ToString("N");

    public override string Id => _id;

    /// <summary>
    /// A0@C,,2D07C=0 C= =C HCT<D2 7C DC,,:D 8D >C$0C0=Că<D2 1C 7Că2Că<D2 @CT=CD5D 5D C DividerRenderBase.
    /// </summary>
    public override Type ComponentType => typeof(DividerRenderBase);

    protected override Task HandleTriggerAsync(object? targetData)
    {
        // AD5C=D 0D$8C$=D'9 D0;CT<CT=D" ?C =CT;C, 8C0AD$@D4<CT=D$>C" =CR 2D'?Că;C00CTB C'C48C=C C0> D$@C,,3
        return Task.CompletedTask;
    }
}
</file>

<file path="Promatis.Net.UI/Data/IUiDataBroker.cs">
using Promatis.Net.Data;

namespace Promatis.Net.UI;

```

```

Ð
/// <summary>Ð
/// Að>CÔBD 0CðB C @Cä:CT@C 4C =CÔKDRÂ CCô@C 2C´ODäICT3Cä 2D´1Cä@Cä< C,,AD$>Dt=C,,:C ...6W'fW"ôö¥R' 8 D BD 0D$5
/// A0$ D >CD5D 6C,,B CD5DD>C´BCÔKDR @CT6C,,<Cä2 – Cô@Cä3D 0CÄ<C,,AD" >C 0Ct0C0 OC$=Cä ACð>CÔDC,,3D4@C,,@Cä2C BDÄ
/// </summary>Ð
public interface IUiDataBroker<TEntity, TQueryState, TResultData> where TEntity : classÐ
{Ð
    /// <summary>Ð
    /// A0@C,,7C00C#Â CCð0CtKC$0DäIC,,9, DtBCâ BCT:D4IC,,9 C,,=D BC =D 1D >Cð5D 0 C00 DD>D <CR @C 1CäBC 5D" 2 D
    /// </summary>Ð
    bool IsInMemoryMode { get; }Ð
Ð
    /// <summary>Ð
    /// Ad5D BCð0D0 :Cä=DD8C4CD 0Dd8D0 @C 1CäBD² =C ?D OCÄCDâ A B #A / gRPC API (Server Mode).Ð
    /// </summary>Ð
    void ConfigureServerMode(Func<TQueryState, CancellationToken, Task<TResultData>>
serverDataProvider);Ð
Ð
    /// <summary>Ð
    /// Ad5D BCð0D0 :Cä=DD8C4CD 0Dd8D0 @C 1CäBD² GCT@CT7 C,,7Cä;C,,@Cä2C =C0>CR ;Cä:C ;DÄ=Cä5 Aä B20ED 0C08C´8D
    /// </summary>Ð
    void ConfigureInMemoryMode(IUiOzuCache<TEntity> ozuCache, Func<TQueryState, List<TEntity>,
TResultData> inMemoryEvaluator);Ð
Ð
    /// <summary>Ð
    /// B4=C,,2CT@D 0C´LC00D0 BCäGCð0 C AC,,=DT@Cä=C0>C4> Ct0C0@CäAC 4C =CÔKDR 2C,,7D40C´8Ct0D$>D 0CÄ8 C,,=D$5D
    /// ATAC´8 C08 Cä4C,,= D 5Cd8CÄ =CR 1D´; D :Cä=DD8C4CD 8D >C$0C0 ?D >C4@C <CÄ8D BCä<, C$Kct>C$5D" -çf Æ-D÷
    /// </summary>Ð
    Task<TResultData> FetchDataAsync(TQueryState state, CancellationToken cancellationToken =
default);Ð
Ð
    /// <summary>Ð
    /// B$>Dt:C 2DT>CD0 CD;D0 ?D >C @CäAC 3C´>C 0C´LC0>C4> D 8C4=C ;C öäVçF-G"6ö0Ö-GFVB 2 C´>Cð0C´LC0KC´ :
    /// </summary>Ð
    void HandleDatabaseCommit(EntityStateChangeEnum state, object entity);Ð
}
</file>

<file path="Promatis.Net.UI/Data/IUiOzuCache.cs">
using Promatis.Net.Data;Ð
Ð
namespace Promatis.Net.UI;Ð
Ð
/// <summary>Ð
/// Að>CÔBD 0CðB C,,7Cä;C,,@Cä2C =C0>C4> C´>Cð0C´LC0>C4> Aä B20ED 0C08C´8D"0 CD0C0=D´E Cð>C0:D 5D$=Cä9 D0:D 0C0
/// A,,AC0>C´LctCCTB D 8D BCT<C0KC´ VçF-G•7F FT6† ævTVçV0 4C´O C0@C,,<CT=CT=C,,0 D$>Dt5Dt=D´E CD5C´LD" 8Ct<CT=CT
/// </summary>Ð
/// <typeparam name="TEntity">B$8C0 4Cä<CT=C0>C´ AD4IC0>D BC,äÄ÷G- W & Óí
public interface IUiOzuCache<TEntity> where TEntity : classÐ
{Ð
    /// <summary>Ð
    /// A0D0D0<Cä9 CD>D BD4? Cç BCT:D4ICT<D2 <C AD 8C$C CD0C0=D´E, DT@C =D0ICT<D4AD0 2 Cä?CT@C BC,,2C0>C´ ?C <D
    /// </summary>Ð
    List<TEntity> InMemoryItems { get; set; }Ð
Ð
    /// <summary>Ð
    /// A 2D$>C0>CÄ=D´9 Aä B204C$8Cd>Cç <D4BC FC,,9. BT8D CD 3C,,GCTACð8 D$>Dt=Cä >C =Cä2C´OCTB Cð>C´;CT:Dd8Dä
    /// C0@C,,=C,,<C O D 8D BCT<C0KC´ AD$5C"B D$@C =Ct0CðFC,,8 B #A .Ð
    /// </summary>Ð
    void ApplyOzuDelta(EntityStateChangeEnum state, TEntity entity);Ð
}
</file>

<file path="Promatis.Net.UI/Data/UiDataBroker.cs">
using MudBlazor;Ð
using Promatis.Net.Data;Ð
using System.Reflection;Ð
Ð
namespace Promatis.Net.UI;Ð
Ð
/// <summary>Ð
/// A0;C BDD>D <CT=C0KC´ 1D >Cð5D 4C =CÔKDRâ &CT=D$@C ;C,,7Cä2C =C0> D4?D 0C$;D05D" ?CäAD$0C$:Cä9 CD0C0=D´E C
/// A0>C´=CäAD$LDä 0C AD$@C 3C,,@D45D" T´ >D" :Cä=Cð@CTBCÔKDR 8C0BCT@DD5C"ACä2 CD>CÄ5C0=D´E D ;D46C 1D0:CT=CD
/// </summary>Ð
/// <typeparam name="TEntity">B$8C0 4Cä<CT=C0>C´ AD4IC0>D BCfÄ÷G- W & Óí
public class UiDataBroker<TEntity, TQueryState, TResultData> : IUiDataBroker<TEntity, TQueryState,
TResultData>Ð

```

```

where TEntity : class
{
    private Func<TQueryState, CancellationToken, Task<TResultData>>? _serverDataProvider;
    private Func<TQueryState, List<TEntity>, TResultData>? _inMemoryEvaluator;
    private IUiOzuCache<TEntity>? _ozuCache;

    private bool _isInitialized;
    private bool _isInMemoryMode;

    public bool IsInMemoryMode => _isInMemoryMode;

    public void ConfigureServerMode(Func<TQueryState, CancellationToken, Task<TResultData>>
serverDataProvider)
    {
        _serverDataProvider = serverDataProvider ?? throw new
ArgumentNullException(nameof(serverDataProvider));
        _inMemoryEvaluator = null;
        _ozuCache = null;
        _isInMemoryMode = false;
        _isInitialized = true;
    }

    public void ConfigureInMemoryMode(IUiOzuCache<TEntity> ozuCache, Func<TQueryState,
List<TEntity>, TResultData> inMemoryEvaluator)
    {
        _ozuCache = ozuCache ?? throw new ArgumentNullException(nameof(ozuCache));
        _inMemoryEvaluator = inMemoryEvaluator ?? throw new
ArgumentNullException(nameof(inMemoryEvaluator));
        _serverDataProvider = null;
        _isInMemoryMode = true;
        _isInitialized = true;
    }

    public async Task<TResultData> FetchDataAsync(TQueryState state, CancellationToken
cancellationToken = default)
    {
        // A A,, "A,, 'AT!A AR "B A A$ AÔ AS¢ D ;C, ?D >C4@C <CÄ8D B Ct0C KC² 2D´7C$0D$L CÄ5D$>CB :Cä=DD8C4C
if (!_isInitialized)
        {
            throw new InvalidOperationException(
                $"A @Cä:CT@ CD0CÔ=D´E CD;Dò w·G— Vöb...DVçF—G´´äæ öWòr =CR 1D´; D :Cä=DD8C4CD 8D >C$0C0â " ½
                $"A$K Cä1D07C =D² 2D´7C$0D$L ConfigureServerMode C,,;C, 6öæf-ww&T-äÖVÖ÷´´"ööFR 2 C¤>CÔAD$@D4:D$
        )
        }

        // B Ad AÂ ¢ D OCÄ0Dò @C 1CäBC A D 5D 2CT@Cä< (A 0Ct0 CD0CÔ=D´E / API)
        if (_serverDataProvider != null)
        {
            return await _serverDataProvider(state, cancellationToken);
        }

        // B Ad AÂ #¢ C 1CäBC GCT@CT7 C´>C¤0C´LC0KC´ At#-C¤MD, DCä@CÄK (B BD 0D$5C48Dò @C ADt5D$0 Cò>C´=C
        if (_isInMemoryMode && _ozuCache != null && _inMemoryEvaluator != null)
        {
            // Aô@CäAD$> CäBCD0CT< D BCT9D" 8 Aä B2ÔAC08D >C¢ =C @D46D2 r ?D4AD$L Cò@C,, :C´0CD=Cä9 D 5C´5C¤BCä
            // C¤0C¢ ACD5C´0D$L Cò0C48C00Dd8Dâ 3D 8CD0 C,,;C, 2D´BC IC,,BDÄ 4CTBCT9 CD5D 5C$0D
            return _inMemoryEvaluator(state, _ozuCache.InMemoryItems);
        }

        throw new InvalidOperationException("A¤@C,,BC,,GCTAC¤8C´ AC >C´ 2C0CD$@CT=C05C´ :Cä=DD8C4CD 0Dd8C, 1D >
    )
}

public void HandleDatabaseCommit(EntityStateChangeEnum state, object entity)
{
    // ATAC´8 DD>D <C @C 1CäBC 5D" =C ?D OCÄCDâ A D 5D 2CT@Cä< — C4;Cä1C ;DÄ=D´5 CÄCD$0Dd8C, At# Cò>C´=
    if (!_isInMemoryMode || _ozuCache == null) return;

    // Aô@Cä2CT@D05CÄ¢ >D$=CäAC,,BD 0 C´8 Cò@C,,;CTBCT2D,,8C´ 8Cr !B4 AB >C JCT:D" : D$8C6C CD0CÔ=D´E D$5C¤C
    if (entity is TEntity typedEntity)
    {
        // A05D 5C00C0@C 2C´0CT< CD5C´LD$C C,,7CÄ5C05C08C´ 2 C,,7Cä;C,,@Cä2C =CÔ>CR ED 0C08C´8D"5 DôBCä9 C¤>
        _ozuCache.ApplyOzuDelta(state, typedEntity);
    }
}
}
</file>

```

```

<file path="Promatis.Net.UI/Data/UiOzuCache.cs">
using Promatis.Net.Data;
using Promatis.Net.Domain.Interface;

namespace Promatis.Net.UI;

/// <summary>
/// B 5C ;C,,7C FC,,0 C,,7Cä;C,,@Cä2C =CÔ>C4> C=MD,,0 C" >Cô5D 0D$8C$=Cä9 Cô0CÄ0D$8 CD;Dò :Cä=C@CTBCÔ>C4> C,,=D BC
/// </summary>
public class UiOzuCache<TEntity> : IUiOzuCache<TEntity> where TEntity : class
{
    public List<TEntity> InMemoryItems { get; set; } = new();
    
    public UiOzuCache() { }
    
    public UiOzuCache(List<TEntity> initialItems)
    {
        InMemoryItems = initialItems ?? throw new ArgumentNullException(nameof(initialItems));
    }
    
    /// <summary>
    /// A$KD >C= >Cô@Cä8Ct2Cä4C,,BCT;DÄ=D´9 Aä B2Ô4C$8Cd>C¢ <D4BC FC,,9. A 5Ct>Cô0D =Cä >C =Cä2C´OCTB C´>C=0C´LC
    /// </summary>
    public void ApplyOzuDelta(EntityStateChangeEnum state, TEntity entity)
    {
        object? entityId = GetEntityId(entity);
        if (entityId == null) return;
        
        switch (state)
        {
            case EntityStateChangeEnum.Added:
                // At0D"8D$0 CäB CDCC ;C,,@Cä2C =C,,0 D BD >C¢ ?D 8 C AC,,=DT@Cä=CÔKDR ACTBCT2D´E C$ACô;CTAC=0D]
                if (!InMemoryItems.Any(x => Equals(GetEntityId(x), entityId)))
                {
                    InMemoryItems.Add(entity);
                }
                break;
            
            case EntityStateChangeEnum.Modified:
                int index = InMemoryItems.FindIndex(x => Equals(GetEntityId(x), entityId));
                if (index >= 0)
                {
                    // Aô>CD<CT=Dô5CÂ AD$0D KC´ >C JCT:D" =C =Cä2D´9 C :D$CC ;DÄ=D´9 D ;CT?Cä: C,,7 B #A Dd
                    InMemoryItems[index] = entity;
                }
                break;
            
            case EntityStateChangeEnum.Deleted:
            case EntityStateChangeEnum.SoftDeleted:
                TEntity? itemToRemove = InMemoryItems.FirstOrDefault(x => Equals(GetEntityId(x),
entityId));
                if (itemToRemove != null)
                {
                    InMemoryItems.Remove(itemToRemove);
                }
                break;
        }
    }
    
    /// <summary>
    /// A,,7C$;CTGCT=C,,5 ID CÔ0 CäACÔ>C$5 D 5C ;DÄ=D´E CD>CÄ5CÔ=D´E C,,=D$5D DCT9D >C" ?C´0D$DCä@CÄK.D
    /// </summary>
    private object? GetEntityId(TEntity item)
    {
        var hasKeyInterface = item.GetType()
            .GetInterfaces()
            .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(IDomainObjectHasKey<>));
        
        if (hasKeyInterface != null)
        {
            return hasKeyInterface.GetProperty("Id")?.GetValue(item);
        }
        
        return item.GetType().GetProperty("Id")?.GetValue(item);
    }
}

```

```

}
</file>

<file path="Promatis.Net.UI/Extensions/MudColorExtensions.cs">
using MudBlazor;
{
namespace Promatis.Net.UI;
{
public static class MudColorExtensions
{
    public static Color GetActionColor(this string? action)
    {
        if (string.IsNullOrEmpty(action)) return Color.Default;

        return action.ToLower().Trim() switch
        {
            "create" or "insert" or "added" or "CD>C 0C$;CT=C,,5" or "D >Ct4C =C,,5" => Color.Success,
            "update" or "edit" or "modified" or "C,,7CÄ5C05C08CR" ÷" $@CT4C :D$8D >C$0C08CR" Óâ 6ÖÆ÷"äv &æ-ærf
            "delete" or "remove" or "deleted" or "softdeleted" or "D44C ;CT=C,,5" => Color.Error,
            _ => Color.Default
        };
    }
}
</file>

<file path="Promatis.Net.UI/Interfaces/IHasSelectedData.cs">
namespace Promatis.Net.UI.Components;
{
/// <summary>
/// B4=C,,2CT@D 0C´LC0KC´ 8C0BCT@DD5C"A-CÄ0D :CT@ CD;Dò 2C,,7D40C´8Ct0D$>D >C"Â :CäBCä@D´< C05Cä1DT>CD8CÄ> D
/// D 8C0ED >C08Ct8D >C$0D$L DD>C=CD AD$@Cä:C,0Cct;C 8 C0>C´CDt0D$L C,,<C0CC´LD K C05D 5D 8D >C$:C,i
/// </summary>
public interface IHasSelectedData<TEntity> where TEntity : class
{
    TEntity? SelectedData { get; set; }
    Action? OnContextUpdated { get; set; }
}
</file>

<file path="Promatis.Net.UI/IUiModule.cs">
namespace Promatis.Net.UI;
{
/// <summary>
/// A= C0BD 0C=B CD;Dò 4C,,=C <C,,GCTAC=8DR <Cä4D4;CT9 C0>C´LCt>C$0D$5C´LD :Cä3Cä 8C0BCT@DD5C"AC í
/// </summary>
public interface IUiModule
{
    /// <summary>
    /// AäBCä1D 0Cd0CT<Cä5 C,,<Dò <Cä4D4;Dò 2 D 8D BCT<CR ,,8D ?Cä;DÄ7D45D$ADò 4C´O D :C =CT@C ´í
    /// </summary>
    string Name { get; }

    /// <summary>
    /// A$>Ct2D 0D"0CTB D ?C,,ACä: D0;CT<CT=D$>C" =C 2C,,3C FC,,8 CD;Dò 1Cä:Cä2Cä3Cä <CT=Dâ xVDG& vW"í
    /// Bt5D$2CT@D$KC´ ?C @C <CTBD >D$2CTGC 5D" 7C "0CD >C$=CT2D4N C4@D4?C08D >C$:D2 MC´5CÄ5C0BCä2.D
    /// </summary>
    /// <returns>A= >C´;CT:Dd8Dò :Cä@D$5Cd5C"¢ ,, C 7C$0C08CRÄ !D KC´:C Ä C= >C0:C Ä C 7C$0C08CT D CC0?D²"Ä÷&W
    IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems();
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor">
@page "/audit-logs"
@using Promatis.Net.Domain
{
<CascadingValue Value="Context">
    <WorkspacePage>
{
    <!-- 1. A$ B %A0/Bò Aä A ¢ C$BCä<C BC,,GCTAC=8C´ ?Cä;C,,<Cä@DD=D´9 D$CC´1C @ (AD8C00CÄ8Dt5D :C,,5 C= >C
    <TopContent>
        <UiToolbar Controls="@Context.Controls" />
    </TopContent>

    <!-- 2. Bd A0"B A´,A0 Bò Aä A ¢ "C 1C´8Dt=C O D 5D$:C Ä @C 1CäBC ND"0Dò 2 DD>C0>C$>CÄ ACT@C$5D =Cä<
    <BodyContent>
        <GridPage TEntity="AuditLog">

```

```

        @ref="_grid"
        ServerDataProvider="@Context.Broker.FetchDataAsync">
        <GridColumns>
            <PropertyColumn T="AuditLog" TProperty="DateTime" Property="x => x.ChangedAt"
Title="AD0D$0 C, 2D 5CÄ0" Format="dd.MM.yyyy HH:mm:ss" HeaderStyle="width: 160px;" />
            <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.ChangedBy"
Title="Aö>C´Lct>C$0D$5C´L" HeaderStyle="width: 150px;" />
            <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.EntityName"
Title="B$8Cð AD4ICô>D BC," †V FW%7G-ÆSò'v-GFfç S f²" óí
            <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.Action"
Title="Aä?CT@C FC,,0" HeaderStyle="width: 120px;" />
            <PropertyColumn T="AuditLog" TProperty="Guid" Property="x => x.EntityId"
Title="ID B CD"=CäAD$8" HeaderStyle="width: 300px;" />
            <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.ChangesJson"
Title="A,,7CÄ5C05C08Dò „¥4ôâ'" †V FW%7G-ÆSò'v-GFfç WFó²" 6Æ 730'FW‡B×G'Væ6 FR" óí
        </GridColumns>
    </GridPage>
</BodyContent>
</WorkspacePage>
</CascadingValue>
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor.cs">
using Microsoft.AspNetCore.Components;
using MudBlazor;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Promatis.Net.UI.Components.Workspaces;
namespace Promatis.Net.UI.Pages.AuditLogs;
public partial class AuditLogPage : ComponentBase
{
    private GridPage<AuditLog>? _grid;
    private bool _isLoading;

    [Inject]
    protected IAuditLogService AuditLogService { get; set; } = null!;

    protected AuditLogWorkspaceContext Context { get; set; } = null!;

    protected override void OnInitialized()
    {
        base.OnInitialized();

        // AÄ Aö A$ Aö Aä ACä7CD0CT< CòCD BCä9 Cæ>C0BCT:D B. B BD 0C08Dd0 D 5C04CT@C,,BD O C 5Cr 7C 4CT@Cd5Cç
        Context = new AuditLogWorkspaceContext(AuditLogService, onFilterChanged: RefreshGrid);
    }

    /// <summary>
    /// Aæ0C0>C08Dt5D :C O D$>Dt:C 7C ?D4ACæ0 D$0Cd5C´KDR 7C ?D >D >C"â !D 0C 0D$KÇ$0CTB Aö B AR BCä3CâÂ
    /// Cæ0Cç ?Cä;DÄ7Cä2C BCT;DÄ CCd5 D42C,,4CT; C4>D$>C$CDâ AD$@C =C,,FD2 =C MCæ@C =CRí
    /// </summary>
    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        await base.OnAfterRenderAsync(firstRender);

        // A$KÇ0>C´=D05CÂ 7C 3D CCT:D2 BCä;DÄ:Câ >CD8Cò @C 7 Cò@C, ?CT@C$>CÂ @CT=CD5D 5 Dô:D 0C000
        if (firstRender)
        {
            // B ?Cä:Cä9C0> D :C GC,,2C 5CÂ CC08Cæ0C´LC0KCR 8CÄ5C00 D CD"=CäAD$5C´ 8Cr !B4 AB ÷7Fw&U5 Í
            List<string> availableEntities = await AuditLogService.GetAvailableEntityNamesAsync();

            // Aô5D 5CD0CT< CD0C0=D´5 C" :Cä=D$5CæAD" 4C´O CD8C00CÄ8Dt5D :Cä9 C45C05D 0Dd8C, ACT;CT:D$0 C, ?C
            Context.InitializeFilters(availableEntities);

            _isLoading = true;
            StateHasChanged(); // Aô5D 5D 8D >C$KÇ$0CT< D$CC´1C @ C, 7C ?D4ACæ0CT< C05D 2D4N Ct0C4@D47CæC C´>
        }
    }

    protected void RefreshGrid()
    {
        if (_grid != null)
        {

```



```

        _ = _grid.ReloadServerData();
    }
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/AuditLogWorkspaceContext.cs">
using MudBlazor;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Controls;
using Promatis.Net.UI.Pages.AuditLogs.Toolbar;

namespace Promatis.Net.UI.Pages.AuditLogs;

public class AuditLogWorkspaceContext : WorkspaceActionContext, IHasSelectedData<AuditLog>
{
    private readonly IAuditLogService _auditLogService;
    private readonly Action _onFilterChanged;

    public UiDataBroker<AuditLog, GridState<AuditLog>, GridData<AuditLog>> Broker { get; }

    public AuditLog? SelectedData
    {
        get;
        set
        {
            if (field != value)
            {
                field = value;
                OnContextUpdated?.Invoke();
                NotifyStateChanged();
            }
        }
    }

    public Action? OnContextUpdated { get; set; }

    public override string TopZoneHeight => "48px";
    public override bool IsLeftZoneCollapsed => true;
    public override bool IsRightZoneCollapsed => true;
    public override bool IsBottomZoneCollapsed => true;

    public AuditLogWorkspaceContext(IAuditLogService auditLogService, Action onFilterChanged)
    {
        _auditLogService = auditLogService ?? throw new
ArgumentNullException(nameof(auditLogService));
        _onFilterChanged = onFilterChanged;

        Broker = new UiDataBroker<AuditLog, GridState<AuditLog>, GridData<AuditLog>>();
        Broker.ConfigureServerMode(FetchAuditDataFromServerAsync);

        AddControl(new ToolbarDivider());
        AddControl(new AuditExportButton());
    }

    public void InitializeFilters(List<string> entityNames)
    {
        _controls.Insert(0, new AuditEntitySelect(entityNames, _onFilterChanged));
        _controls.Insert(1, new AuditPeriodPicker(_onFilterChanged));
        NotifyStateChanged();
    }

    /// <summary>
    /// B 5D 2CT@C00D0 AD$@C BCT3C,,0. AÄ0C08D" ACäAD$>D0=C,,5 MudBlazor C" AD$@Cä3Cä ?Cä7C,,FC,,>C0=D´9 Cæ>C0AD$
    /// </summary>
    private async Task<GridData<AuditLog>> FetchAuditDataFromServerAsync(GridState<AuditLog> state,
CancellationToken ct)
    {
        var entityControl = Controls.FirstOrDefault(c => c.Id == "audit_filter_entity");
        var periodControl = Controls.FirstOrDefault(c => c.Id == "audit_filter_period");

        string? selectedEntity = entityControl is IHasValue ev ? ev.Value as string : null;
        DateRange? selectedPeriod = periodControl is IHasValue pv ? pv.Value as DateRange : null;
    }
}

```

```

        if ("A$ACR AD4ICÔ>D BC,"äw V Ç2†6VÆV7FVDVÇF–G'Â 7G&-æt6ö× &-6öä÷÷&F-æ Ä-væ÷&T6 6R'•
        {
            selectedEntity = null;
        }

        DateTime fromDate = selectedPeriod?.Start ?? DateTime.Today.AddDays(-7);
        DateTime toDate = selectedPeriod?.End ?? DateTime.Today;

        // A05D 5C$>CD8CÂ Äö6 ÄöVç7 V6-f-VB 2D 5CÃO CäB MudBlazor C" GCTAD$=D´9 DateTimeKind.Utc
        fromDate = DateTime.SpecifyKind(fromDate.Date, DateTimeKind.Utc);

        // AD;Dò :Cä=CTGCÔ>C' 4C BD² ?CT@C,,>CD0 C 5D 5CÂ AC <D´9 C÷>C05Db 4C00 (23:59:59), DtBCä1D² 7C EC$0D$
        // C, BCäGCÔ> D$0C¢ 6CR <C @C÷8D CCT< C÷0C¢ UD=
        toDate = DateTime.SpecifyKind(toDate.Date.AddDays(1).AddTicks(-1), DateTimeKind.Utc);

        // B BD >C,,< Cö>Ct8Dd8Cä=C0KC' &V6÷&B07C ?D >D Â :CäBCä@D´9 D$5C05D L C00 100% C$0C´8CD5C0 4C´0 Postg
        var searchRequest = new AuditLogSearchRequest(
            FromDate: fromDate,
            ToDate: toDate,
            EntityName: selectedEntity,
            Action: null,
            PageIndex: state.Page,
            PageSize: state.PageSize
        );

        PagedResult<AuditLog> pagedResult = await _auditLogService.SearchLogsAsync(searchRequest,
        ct);

        return new GridData<AuditLog>
        {
            Items = pagedResult.Items,
            TotalItems = pagedResult.TotalCount
        };
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditEntitySelect.cs">
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;
public class AuditEntitySelect : BaseUiControl, IHasValue, IHasOptions
{
    private readonly Action _onChanged;

    public override string Id => "audit_filter_entity";
    public override Type ComponentType => typeof(SelectRenderBase);
    public override string Title => "B$8Cò AD4ICÔ>D BC,#½

    public object? Value { get; set; }
    public IEnumerable<string> Options { get; set; }

    public AuditEntitySelect(List<string> availableEntities, Action onChanged)
    {
        _onChanged = onChanged;
        Value = "A$ACR AD4ICÔ>D BC,#½

        // BD>D <C,,@D45CÂ 4C,,=C <C,,GCTAC÷8C' ACô8D >C¢ >C0FC,,9 C00 CäACÔ>C$5 CD0C0=D´E C,,7 A  D
        var list = new List<string> { "A$ACR AD4ICÔ>D BC," ö½
        list.AddRange(availableEntities);
        Options = list;
    }

    protected override Task HandleTriggerAsync(object? targetData)
    {
        _onChanged.Invoke();
        return Task.CompletedTask;
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditExportButton.cs">
using MudBlazor;
using Promatis.Net.UI.Components;

```

```

using Promatis.Net.UI.Components.ElementRenderBase;
{
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;
{
public class AuditExportButton : BaseUiControl
{
    public override string Id => "audit_action_export";
    public override Type ComponentType => typeof(ButtonRenderBase); // B 5C04CT@CT@ D04D 0
    public override string Title => "A$KC4@D47C,,BDÄ#½
    public override string Icon => Icons.Material.Filled.Download;
    public override string Tooltip => "A$KC4@D47C,,BDÄ >D$DC,,;DÄBD >C$0C0=D´5 C´>C48 C" W†6VÂ#½

    protected override async Task HandleTriggerAsync(object? targetData)
    {
        // A,,<C,,BC,,@D45CÄ 4Cä;C4CDâ 2D´3D CCT:D2 r :C0>C0:C 0C$BCä<C BC,,GCTAC=8 Ct0C ;Cä:C,,@D45D$AD0 8 C0>C
        await Task.Delay(2000);
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditPeriodPicker.cs">
using MudBlazor;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;
{
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;
{
public class AuditPeriodPicker : BaseUiControl, IHasValue
{
    private readonly Action _onChanged;

    public override string Id => "audit_filter_period";
    public override Type ComponentType => typeof(DateRangeRenderBase); // B 5C04CT@CT@ D04D 0
    public override string Title => "A05D 8Cä4 C´>C4>C"#½

    public object? Value { get; set; }

    public AuditPeriodPicker(Action onChanged)
    {
        _onChanged = onChanged;
        Value = new DateRange(DateTime.Today.AddDays(-7), DateTime.Today);
    }

    protected override Task HandleTriggerAsync(object? targetData)
    {
        _onChanged.Invoke(); // A05D 8Cä4 C,,7CÄ5C08C´AD0 r 4C 5CÄ 8CÄ?D4;DÄA D BD 0C08Dd5
        return Task.CompletedTask;
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Pages/Dashboard.razor">
@page "/"
{
<MudText Typo="Typo.h4" Class="mb-4">AÄ>CDCCL Master Data Management</MudText>
{
<MudGrid>
    <MudItem xs="12" sm="6" md="4">
        <MudPaper Class="d-flex flex-row pt-6 pb-4 px-4" Style="height:100px;">
            <MudIcon Icon="@Icons.Material.Filled.Storage" Color="Color.Primary" Size="Size.Large"
Class="mr-4" />
            <div>
                <MudText Typo="Typo.subtitle2" Class="mud-text-secondary">B BC BD4A C0>CD:C´NDt5C08D0 : A </
                <MudText Typo="Typo.h6">A :D$8C$=Cä ...FW7D& 6R"Ä0×VEFW†Cí
            </div>
        </MudPaper>
    </MudItem>
    {
        <MudItem xs="12" sm="6" md="8">
            <MudPaper Class="pa-4" Style="height:100px;">
                <MudText Typo="Typo.subtitle1">AD>C @Cä ?Cä6C ;Cä2C BDÄ 2 D 8D BCT<D2 &öÖ F–2 U% Ä0×VEFW†Cí
                <MudText Typo="Typo.body2" Class="mud-text-secondary">
                    A$ACR 8C0BCT@D5C"AD² ?Cä4C4@D46CT=D² 4C,,=C <C,,GCTAC=8 C,,7 C05Ct0C$8D 8CÄKDR AC´>CT2 Razor CL
                </MudText>
            </MudPaper>
        </MudItem>
    }
}
</file>

```

```
</MudGrid>
</file>
```

```
<file path="Promatis.Net.UI/Pages/Dashboard.razor.css">
.my-component {
    border: 2px dashed red;
    padding: 1em;
    margin: 1em 0;
    background-image: url('background.png');
}
</file>
```

```
<file path="Promatis.Net.UI/Promatis.Net.UI.csproj">
<Project Sdk="Microsoft.NET.Sdk.Razor">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <Nullable>enable</Nullable>
        <ImplicitUsings>enable</ImplicitUsings>
    </PropertyGroup>
    <ItemGroup>
        <Compile Remove="AuditLog\*" />
        <Compile Remove="Models\*" />
        <Content Remove="AuditLog\*" />
        <Content Remove="Models\*" />
        <EmbeddedResource Remove="AuditLog\*" />
        <EmbeddedResource Remove="Models\*" />
        <None Remove="AuditLog\*" />
        <None Remove="Models\*" />
    </ItemGroup>
    <ItemGroup>
        <SupportedPlatform Include="browser" />
    </ItemGroup>
    <ItemGroup>
        <PackageReference Include="Microsoft.AspNetCore.Components.Web" Version="10.0.8" />
        <PackageReference Include="Microsoft.AspNetCore.Http.Abstractions" Version="2.3.10" />
        <PackageReference Include="MudBlazor" Version="9.4.0" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\..\
\MesCore\Promatis.Net.MES.Service\Promatis.Net.MES.Service.csproj" />
        <ProjectReference Include="..\Configuration\Promatis.Net.Configuration.csproj" />
        <ProjectReference Include="..\Service\Promatis.Net.Service.csproj" />
    </ItemGroup>
</Project>
</file>
```

```
<file path="Promatis.Net.UI/Theme/PromatisTheme.cs">
using MudBlazor;
namespace Promatis.Net.Ui.Theme;
public static class PromatisTheme
{
    public static MudTheme DefaultTheme => new()
    {
        PaletteLight = new PaletteLight
        {
            Primary = Colors.Blue.Lighten1, // A4;C 2C0KC' :Cä@Cö>D 0D$8C$=D'9 Dd2CTB (C=Cä?C=8, C :D$
            Secondary = Colors.Blue.Lighten2, // A$ACö>CÄ>C40D$5C'LC0KC' FC$5D" „2D$>D >D BCT?CT=C0KCR MC

            AppBarBackground = "#EAEAEA", // BD>C0 2CT@DT=CT9 C00C05C'8 (MudAppBar) C" 4C05C$=Cä< D 5
            AppBarText = "#2C3E50", // Bd2CTB D$5C=AD$0, Ct0C4>C'>C$:Cä2 C, 8C= >C0>C 2 C$5D EC
            DrawerBackground = "#EAEAEA", // BD>C0 1Cä:Cä2Cä3Cä <CT=Dä =C 2C„3C FC„8 (MudDrawer)D
            Background = "#F9F9F9", // Aä1D"8C' DCä=Cä2D'9 Dd2CTB Cö>CD;Cä6C=8 C$ACTE D BD 0C08
            Surface = Colors.Shades.White, // BD>C0 :Cä=D$5C"=CT@Cä2 MudPaper, C=0D BCäGCT: MudCard C

        },
        PaletteDark = new PaletteDark
        {
            Primary = Colors.Blue.Lighten1, // A4;C 2C0KC' FC$5D" 2 C0>Dt=Cä< D 5Cd8CÄ5 (CÄ0C4:C„9 D 8C

```

```

        Secondary = Colors.Blue.Lighten2, // A$ACô>CÄ>C40D$5C´LCÔKC´ FC$5D" 2 CÔ>Dt=Cä< D 5Cd8CÄ5D
    }
    AppBarBackground = "#0D0D0D", // BD>CÔ 2CT@DT=CT9 Cô0CÔ5C´8 (MudAppBar) C" 3C´CC >C¤>CÄ G
    AppBarText = Colors.Shades.White, // Bd2CTB D$5C¤AD$0 C, 8C¤>CÔ>C¤ 2 D,,0Cô:CR ,, :Cä=D$@C AD$=D
}
Background = "#121212", // A4;D41Cä:C,,9 D$QCÄ=D´9 DD>CÔ AD$@C =C,,F Cô>CD;Cä6C¤8D
Surface = "#1E1E1E", // A4@C DC,,BCä2D´9 DD>CÔ 4C´O MudPaper, C¤>CÔBCT=D$0 C$:C´0
DrawerBackground = "#1E1E1E", // BD>CÔ 1Cä:Cä2Cä9 Cô0CÔ5C´8 CÔ0C$8C40Dd8C, 2 D$QCÄ=Cä9 D$
DrawerText = "#E0E0E0", // Bd2CTB D AD´;Cä: C, BCT:D BC 2 C >C¤>C$>CÄ <CT=Dí
TextPrimary = "#E0E0E0", // AäACÔ>C$=Cä9 Dd2CTB D$5C¤AD$0 CÔ0 D BD 0CÔ8Dd0DR ,,<Dô3C¤
TextSecondary = "#A0A0A0", // Bd2CTB CD;Dô 2D$>D >D BCT?CT=CÔ>C4> D$5C¤AD$0 C, ?Cä4D
},
LayoutProperties = new LayoutProperties()
{
    DrawerWidthLeft = "260px",
    DrawerWidthRight = "300px"
}
};
}
</file>

```

```

<file path="Promatis.Net.UI/UiModule.cs">
using MudBlazor;
namespace Promatis.Net.UI;
public class UiModule : IUiModule
{
    public string Name => GetType().Assembly.GetName().Name ?? "Unknown.Module";

    public IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems()
    {
        return new List<(string Title, string Href, string Icon, string? Group)>
        {
            // BÔ;CT<CT=D" 2CT@DT=CT3Cä CD >C$=Dò ,,2CÔ5 Cô0Cô>C¤•
            ("A4;C 2CÔ0Dò"Ä "ò"Ä -6öç2äÖ FW&- Ääf-ÆÆVBäF 6†&ö &BÄ çVÆÄ´Í
            // B 0Ct4CT; D 8D BCT<CÔ>C4> C 4CÄ8CÔ8D BD 8D >C$0CÔ8Dý
            ("AdCD =C ; C CCD8D$0", "/audit-logs", Icons.Material.Filled.History, "A 4CÄ8CÔ8D BD 8D >C$0CÔ8CR
        };
    }
}
</file>

```

```

<file path="Promatis.Net.UI/wwwroot/App.css">
html, body {
    font-family: 'Helvetica Neue', Helvetica, Arial, sans-serif;
}
a, .btn-link {
    color: #0066bb7;
}
.btn-primary {
    color: #fff;
    background-color: #1b6ec2;
    border-color: #1861ac;
}
.btn:focus, .btn:active:focus, .btn-link.nav-link:focus, .form-control:focus, .form-check-
input:focus {
    box-shadow: 0 0 0 0.1rem white, 0 0 0 0.25rem #258cfb;
}
.content {
    padding-top: 1.1rem;
}
h1:focus {
    outline: none;
}
.valid.modified:not([type=checkbox]) {
    outline: 1px solid #26b050;
}

```



```

<ItemGroup>
  <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />
</ItemGroup>

<ItemGroup>
  <ProjectReference Include="..\Data\Promatis.Net.Data.csproj" />
</ItemGroup>

</Project>
</file>

<file path="Service/Services/AuditLogService/AuditLogService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;

namespace Promatis.Net.Service;

/// <summary>
/// B 5D 2C,,A CD;Dò @C 1CäBD² A C´>C40CÄ8 C CCD8D$0.
/// Aô>C´=CäAD$LDä ACä2CÄ5D BC,,< D 1C 7Cä2D´< BaseService C, 0C$BCä<C BC,,GCTACø8 D 5C48D BD 8D CCTBD O D :C
/// </summary>
public class AuditLogService(IDbContextFactory<ApplicationDbContext> contextFactory)
    : BaseService<AuditLog, Guid, ApplicationDbContext>(contextFactory), IAuditLogService
{
    private static List<string>? _cachedEntityNames;
    private static readonly SemaphoreSlim CacheLock = new(1, 1);

    public async Task<PagedResult<AuditLog>> SearchLogsAsync(AuditLogSearchRequest request,
        CancellationToken cancellationToken = default)
    {
        await using ApplicationDbContext context = await
            ContextFactory.CreateDbContextAsync(cancellationToken);

        IQueryable<AuditLog> query = context.Set<AuditLog>()
            .AsNoTracking()
            .Where(l => l.ChangedAt >= request.FromDate && l.ChangedAt <= request.ToDate);

        if (!string.IsNullOrEmpty(request.EntityName)) query = query.Where(l => l.EntityName
            == request.EntityName);
        if (!string.IsNullOrEmpty(request.Action)) query = query.Where(l => l.Action ==
            request.Action);

        int totalCount = await query.CountAsync(cancellationToken);

        if (totalCount == 0)
            return new PagedResult<AuditLog>([], 0);

        List<AuditLog> items = await query
            .OrderByDescending(l => l.ChangedAt)
            .Skip(request.PageIndex * request.PageSize)
            .Take(request.PageSize)
            .ToListAsync(cancellationToken);

        return new PagedResult<AuditLog>(items, totalCount);
    }

    public async Task<List<string>> GetAvailableEntityNamesAsync(CancellationToken
        cancellationToken = default)
    {
        if (_cachedEntityNames != null)
        {
            return [.. _cachedEntityNames];
        }

        await CacheLock.WaitAsync(cancellationToken);
        try
        {
            if (_cachedEntityNames != null)
            {
                return [.. _cachedEntityNames];
            }

            await using ApplicationDbContext context = await
                ContextFactory.CreateDbContextAsync(cancellationToken);

```

```

        _cachedEntityNames = await context.Set<AuditLog>()
            .AsNoTracking()
            .Select(l => l.EntityName)
            .Distinct()
            .OrderBy(name => name)
            .ToListAsync(cancellationToken);
    }

    return [... _cachedEntityNames];
}
finally
{
    CacheLock.Release();
}
}

public static void InvalidateCache()
{
    CacheLock.Wait();
    try
    {
        _cachedEntityNames = null;
    }
    finally
    {
        CacheLock.Release();
    }
}
}
</file>

<file path="Service/Services/AuditLogService/IAuditLogService.cs">
using Promatis.Net.Domain;
namespace Promatis.Net.Service;
public interface IAuditLogService
{
    /// <summary>
    /// Aö>C,,AC¢ ;Cä3Cä2 C CCD8D$0 Cö> CD8C ?C 7Cä=D2 4C B D ?Cä4CD5D 6C¢>C' DC,,;DÄBD 0Dd8C, 8 Cö0C48C00Dd8C
    /// </summary>
    Task<PagedResult<AuditLog>> SearchLogsAsync(AuditLogSearchRequest request, CancellationToken
cancellationToken = default);
    /// <summary>
    /// Aö>C´CDt5C08CR ACö8D :C CC08C¢0C´LC0KDR 8CÄ5C0 AD4IC0>D BCT9, C¢>D$>D KCR 5D BDÂ 2 C´>C40DR ,,4C´O C$
    /// </summary>
    Task<List<string>> GetAvailableEntityNamesAsync(CancellationToken cancellationToken = default);
}
</file>

<file path="Service/Services/BaseService/BaseService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Service;
public abstract class BaseService<T, TKey, TContext>(DbContextFactory<TContext> contextFactory)
    : IBASEService<T, TKey>
    where T : class, IDomainObjectHasKey<TKey>
    where TContext : DbContext
{
    protected readonly DbContextFactory<TContext> ContextFactory = contextFactory ?? throw new
ArgumentNullException(nameof(contextFactory));

    public virtual async Task<T?> GetByIdAsync(TKey id)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>().FindAsync(id);
    }

    public virtual async Task<List<T>> GetAllAsync()
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>().ToListAsync();
    }
}

```



```

    public virtual async Task AddAsync(T entity)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        await context.Set<T>().AddAsync(entity);
        await context.SaveChangesAsync();
    }

    public virtual async Task UpdateAsync(T entity)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        context.Set<T>().Update(entity);
        await context.SaveChangesAsync();
    }

    public virtual async Task DeleteAsync(TKey id)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        T? entity = await context.Set<T>().FindAsync(id);
        if (entity != null)
        {
            context.Set<T>().Remove(entity);
            await context.SaveChangesAsync();
        }
    }
}
</file>

```

```

<file path="Service/Services/BaseService/IBaseService.cs">
namespace Promatis.Net.Service;
{
    // A 0Ct>C$KC' 5%TM
    public interface IBaseService<T, in TKey> where T : class
    {
        Task<T?> GetByIdAsync(TKey id);
        Task<List<T>> GetAllAsync();
        Task AddAsync(T entity);
        Task UpdateAsync(T entity);
        Task DeleteAsync(TKey id);
    }
}
</file>

```

```

<file path="Service/Services/ReferenceService/IReferenceService.cs">
namespace Promatis.Net.Service;
{
    // AD;Dò ACô@C 2CâGC08Câ>C-
    public interface IReferenceService<T> : IBaseService<T, Guid> where T : class
    {
        Task<T?> GetByCodeAsync(string code);
        Task<List<T>> SearchByNameAsync(string namePart);
    }
}
</file>

```

```

<file path="Service/Services/ReferenceService/ReferenceService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
{
    namespace Promatis.Net.Service;
    {
        public abstract class ReferenceService<T, TContext>(DbContextFactory<TContext> contextFactory)
            : BaseService<T, Guid, TContext>(contextFactory), IReferenceService<T>
            where T : ReferenceBase
            where TContext : DbContext
        {
            public async Task<T?> GetByCodeAsync(string code)
            {
                await using TContext context = await ContextFactory.CreateDbContextAsync();
                return await context.Set<T>().FirstOrDefaultAsync(x => x.Code == code);
            }

            public async Task<List<T>> SearchByNameAsync(string namePart)
            {

```

```

        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>().
            .Where(x => x.Name.Contains(namePart)).
            .ToListAsync();
    }
}
</file>

<file path="Service/Services/ReferenceTreeService/IReferenceTreeService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;

namespace Promatis.Net.Service;

public interface IReferenceTreeService<T, TContext> : IReferenceService<T>, ITreeService<T>
    where T : ReferenceTreeBase<T>, ITreeNode<T>, IDomainObjectHasKey<Guid>
    where TContext : DbContext
{
    // B AC2Cä7CÔKCÂ =C AC´5CD>C$0C08CT< C,,=D$5D DCT9D >C" 2D Q D >D,,;CäADÂ
}
</file>

<file path="Service/Services/ReferenceTreeService/ReferenceTreeService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;

namespace Promatis.Net.Service;

/// <summary>
/// B4=C,,2CT@D 0C´LCÔKC´ 1C 7Cä2D´9 D 5D 2C,,A CD;Dò @C 1CäBD² ACâ 2D 5CÄ8 CD@CT2Cä2C,,4CÔKCÄ8 D BD CC²BD4@C <C
/// A00D ;CT4D45D" ;Cä3C,,;D2 >C KDt=D´E D ?D 0C$>Dt=C,,;Cä2 (ReferenceService) C, @C AD,,8D OCTB CTQ CD;Dò 8CT@
/// </summary>
/// <typeparam name="T">B$8Cò AD4ICÔ>D BC,Â CC00D ;CT4Cä2C =CÔKC´ >D" &VfW&Væ6UG&VT& 6RãÂ÷G- W & Óí
/// <typeparam name="TContext">A²>CÔBCT:D B C 0CtK CD0C0=D´E.</typeparam>
public abstract class ReferenceTreeService<T, TContext>(IDbContextFactory<TContext> contextFactory)
    : ReferenceService<T, TContext>(contextFactory), IReferenceTreeService<T, TContext>
    where T : ReferenceTreeBase<T>, ITreeNode<T>, IDomainObjectHasKey<Guid>
    where TContext : DbContext
{
    private TreeServiceProxy? _treeServiceProxy;
    private TreeServiceProxy Engine => _treeServiceProxy ??= new TreeServiceProxy(ContextFactory,
this);
    // =====
    // BÔ AT A B$ B´ Aô Aä B B B´ Aä Aä A" AD AÔ+A´ A$ Ad A¢ E$TU4U%d"4R f R B4 A´ B A$ AÔ Bò Aä A
    // =====
    public Task<List<T>> GetRootsAsync() => Engine.GetRootsAsync();
    public Task<List<T>> GetChildrenAsync(Guid parentId) => Engine.GetChildrenAsync(parentId);
    public Task<List<T>> GetParentPathAsync(Guid id) => Engine.GetParentPathAsync(id);
    public Task<T?> GetFullTreeAsync(Guid rootId) => Engine.GetFullTreeAsync(rootId);
    public Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default) =>
Engine.MoveAsync(id, newParentId, ct);
    public abstract Task<T> CreateChildTemplateAsync(T parent);

    // A$ B4"B AÔ A,, Aä B "-B A A,, A "Aä Aô A "BD B B½
    private class TreeServiceProxy(IDbContextFactory<TContext> factory, ReferenceTreeService<T,
TContext> parentService)
        : TreeService<T, TContext>(factory)
    {
        public override Task<T> CreateChildTemplateAsync(T parent) =>
parentService.CreateChildTemplateAsync(parent);
    }
}
</file>

<file path="Service/Services/TreeService/ITreeService.cs">
using Promatis.Net.Domain.Interface;

namespace Promatis.Net.Service;

/// <summary>
/// A4;Cä1C ;DÄ=D´9 Cò;C BDD>D <CT=CÔKC´ :Cä=D$@C :D" 4C´O C 1D >C´ND$=Cä ;Dä1Cä3Cä ACT@C$8D 0, D4?D 0C$;DòND

```

```

/// A00D ;CT4D45D" 1C 7Cä2D´9 CRUD C0;C BDD>D <D² 8 D 0D HC,,@D05D" 5C4> C,,5D 0D EC,,GCTAC08CÄ8 CÄ5D$>CD0CÄ8 D4
/// </summary>D
/// <typeparam name="T">B$8C0 4Cä<CT=C0>C´ AD4IC0>D BC,Â @CT0C´8CtCDäICT9 C0>C0BD 0C0B ITreeNode.</typeparam>
public interface ITreeService<T> : IBaseService<T, Guid>D
    where T : class, ITreeNode<T>, IDomainObjectHasKey<Guid>D
{D
    Task<List<T>> GetRootsAsync();D
    Task<List<T>> GetChildrenAsync(Guid parentId);D
    Task<List<T>> GetParentPathAsync(Guid id);D
    Task<T?> GetFullTreeAsync(Guid rootId);D
    Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default);D
    Task<T> CreateChildTemplateAsync(T parent);D
}
</file>

<file path="Service/Services/TreeService/TreeService.cs">
using Microsoft.EntityFrameworkCore;D
using Promatis.Net.Domain.Interface;D
using System.Reflection;D
D
namespace Promatis.Net.Service;D
D
/// <summary>D
/// B4=C,,2CT@D 0C´LC0KC´ 1C 7Cä2D´9 C0;C AD @CT0C´8Ct0Dd8C, 1C,,7C05D 0;Cä3C,,:C, 4C´O A´.A +BR 4D 5C$>C$8CD=D
/// A,,=C0C0AD4;C,,@D45D" BD06CT;D´5 C,,5D 0D EC,,GCTAC08CR 0C´3Cä@C,,BCÄK B #A , C0@Cä2CT@C0C Dd8C0;Cä2 Dd8C0;C
/// </summary>D
/// <typeparam name="T">B$8C0 4Cä<CT=C0>C4> Cä1D05C0BC Â @CT0C´8CtCDäICT3Câ •G&VTæöFRÄÄ÷G- W & Óí
/// <typeparam name="TContext">A0>C0BCT:D B C 0CtK CD0C0=D´E DbContext.</typeparam>D
public abstract class TreeService<T, TContext>(IDbContextFactory<TContext> contextFactory) D
    : BaseService<T, Guid, TContext>(contextFactory), ITreeService<T>D
    where T : class, ITreeNode<T>, IDomainObjectHasKey<Guid>D
    where TContext : DbContextD
{D
    public async Task<List<T>> GetRootsAsync()D
    {D
        await using TContext context = await ContextFactory.CreateDbContextAsync();D
        return await context.Set<T>().AsNoTracking().Where(x => x.ParentId == null).ToListAsync();D
    }D
D
    public async Task<List<T>> GetChildrenAsync(Guid parentId)D
    {D
        await using TContext context = await ContextFactory.CreateDbContextAsync();D
        return await context.Set<T>().AsNoTracking().Where(x => x.ParentId ==
parentId).ToListAsync();D
    }D
D
    public async Task<List<T>> GetParentPathAsync(Guid id)D
    {D
        await using TContext context = await ContextFactory.CreateDbContextAsync();D
        var path = new List<T>();D
        T? current = await context.Set<T>().AsNoTracking().FirstOrDefaultAsync(x => x.Id == id);D
D
        while (current != null)D
        {D
            path.Insert(0, current);D
            if (current.ParentId == null) break;D
            Guid parentId = current.ParentId.Value;D
            current = await context.Set<T>().AsNoTracking().FirstOrDefaultAsync(x => x.Id ==
parentId);D
            if (current == null) break;D
        }D
        return path;D
    }D
D
    public async Task<T?> GetFullTreeAsync(Guid rootId)D
    {D
        await using TContext context = await ContextFactory.CreateDbContextAsync();D
        List<T> allItems = await context.Set<T>().AsNoTracking().ToListAsync();D
        ILookup<Guid?, T> lookup = allItems.ToLookup(x => x.ParentId);D
        T? root = allItems.FirstOrDefault(x => x.Id == rootId);D
        if (root == null) return null;D
D
        BuildTree(root, lookup);D
        return root;D
    }D
D
}

```

```

private void BuildTree(T parent, ILookup<Guid?, T> lookup)
{
    List<T> children = lookup[parent.Id].ToList();
    parent.Children.Clear();
    foreach (T child in children)
    {
        parent.Children.Add(child);
        PropertyInfo? parentProp = child.GetType().GetProperty(nameof(ITreeNode<T>.Parent));
        if (parentProp is { CanWrite: true }) parentProp.SetValue(child, parent);
        BuildTree(child, lookup);
    }
}

public virtual async Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default)
{
    await using TContext context = await ContextFactory.CreateDbContextAsync(ct);
    T entity = await context.Set<T>().FirstOrDefaultAsync(x => x.Id == id, ct)
        ?? throw new Exception($"B CD"=CäAD$L {typeof(T).Name} D "B ¶-G0 =CR =C 9CD5C00.");
    if (entity.ParentId == newParentId) return;
    if (newParentId.HasValue)
    {
        if (newParentId == id) throw new InvalidOperationException("B47CT; C05 CÄ>Cd5D" 1D´BDÂ @Cä4C„BCT;
        List<T> path = await GetParentPathAsync(newParentId.Value);
        if (path.Any(x => x.Id == id)) throw new InvalidOperationException("Bd8C¶;C„GCTAC¶0D0 7C 2C„AC„<C
        bool parentExists = await context.Set<T>().AnyAsync(x => x.Id == newParentId.Value, ct);
        if (!parentExists) throw new Exception("A0>C$KC' @Cä4C„BCT;DÄ =CR =C 9CD5C0â"“½
    }
    entity.ParentId = newParentId;
    await context.SaveChangesAsync(ct);
}

public abstract Task<T> CreateChildTemplateAsync(T parent);
}
</file>

<file path="Tests/AssemblyInfo.cs">
using Xunit;
// AäBC¶;DäGC 5D" ?C @C ;C´5C´LC0>CR 2D´?Cä;C05C08CR BCTAD$>C" 2 D0BCä9 D 1Cä@C¶5D
[assembly: CollectionBehavior(DisableTestParallelization = true)]
</file>

<file path="Tests/Domain/DomainLogicTests.cs">
using Promatis.Net.Domain;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class DomainLogicTests
{
    // B 5C ;C„7C FC„8 CD;D0 BCTAD$8D >C$0C08D0 0C AD$@C :D$=D´E C¶;C AD >C-
    private class TestDomainObject : DomainObject { }
    private class TestReference : ReferenceBase { }

    [Fact]
    public void DomainObject_EveryInstance_ShouldHaveUniqueId()
    {
        // Arrange & Act
        var obj1 = new TestDomainObject();
        var obj2 = new TestDomainObject();
        var obj3 = new TestDomainObject();

        // Assert
        Assert.NotEqual(obj1.Id, obj2.Id);
        Assert.NotEqual(obj2.Id, obj3.Id);
        Assert.NotEqual(Guid.Empty, obj1.Id);
    }

    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    [InlineData(null)]
    public void ReferenceBase_Name_CanBeNullOrEmpty_TechnicalCheck(string? invalidName)
    {

```

```

        // Arrange
        var reference = new TestReference();

        // Act
        reference.Name = invalidName!; // A,,ACô>C´LCtCCT< !, D$0C¢ :C : Name C" :Cä4CR =CR çVÆÆ &Æ]

        // Assert
        // Aô>C¤0 CÄK Cö@CäAD$> Cö@Cä2CT@Dö5CÂÂ GD$> D 2Cä9D BC$> Cö@C,,=C,,<C 5D" MD$8 Ct=C GCT=C,,0D
        // ATAC´8 C$K CD>C 0C$8D$5 C$0C´8CD0Dd8Dâ 2 C CCDCD"5CÂÂ MD$>D" BCTAD" ?Cä<Cä6CTB CTQ CäBC´0CD8D$LB
        Assert.Equal(invalidName, reference.Name);
    }

    [Fact]
    public void ReferenceBase_ShouldHoldValidName()
    {
        // Arrange
        var reference = new TestReference();
        string expectedName = "AäACô>C$=Cä9 D :C´0CB#½

        // Act
        reference.Name = expectedName;

        // Assert
        Assert.Equal(expectedName, reference.Name);
    }
}
</file>

<file path="Tests/Domain/DomainObjectBaseLogicTests.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Domain;

public class DomainObjectBaseLogicTests
{
    private class ConcreteDomainObject : DomainObject { }

    [Fact]
    public void DomainObject_ShouldGenerateNewGuidInConstructor()
    {
        // Arrange & Act
        var obj1 = new ConcreteDomainObject();
        var obj2 = new ConcreteDomainObject();

        // Assert
        Assert.NotEqual(Guid.Empty, obj1.Id);
        Assert.NotEqual(Guid.Empty, obj2.Id);
        Assert.NotEqual(obj1.Id, obj2.Id);
    }

    [Fact]
    public void DomainObject_ShouldCastToIDomainObjectAndHaveCreatedAt()
    {
        // Arrange
        var obj = new ConcreteDomainObject();

        // Act
        IDomainObject baseInterface = obj;

        // Assert
        // B$5Cö5D L CÄK D$>C´LC¤> Cö@Cä2CT@Dö5CÂ =C ;C,,GC,,5 Ct=C GCT=C,,0. D
        // Aô>CöKD$:C 7C ?C,,AC, & 6T-çFW&f 6Rä7&V FVD B 0 æwTF FR 7CD5D L Cö5 D :Cä<Cö8C´8D CCTBD 0.D
        Assert.True(baseInterface.CreatedAt > DateTime.MinValue);
        Assert.True((DateTime.UtcNow - baseInterface.CreatedAt).TotalSeconds < 5);
    }

    [Fact]
    public void CreatedAt_ShouldAllowManualInitialization_OnCreation()
    {
        // Arrange
        var customDate = new DateTime(2020, 1, 1, 0, 0, 0, DateTimeKind.Utc);

        // Act
        // B 0C >D$0CTB D$>C´LC¤> Dt5D 5Cr 8Cö8Dd8C ;C,,7C BCä@ Cä1D¤5C¤BC ?D 8 D >Ct4C =C,,8D

```

```

        var obj = new ConcreteDomainObject { CreatedAt = customDate };
    }
    // Assert
    Assert.Equal(customDate, obj.CreatedAt);
}
}
</file>

<file path="Tests/Domain/DomainObjectBaseTests.cs">
using Moq;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class DomainObjectBaseTests
{
    // B >Ct4C 5CÂ <C,,=C,,<C ;DÄ=D4N D 5C ;C,,7C FC,,N CD;Dò BCTAD$>C-
    private class TestDomainObject : DomainObjectBase<int> { }

    [Fact]
    public void DomainObject_ShouldInitializeWithDefaultValues()
    {
        // Arrange & Act
        var domainObject = new TestDomainObject();

        // Assert
        Assert.Equal(default, domainObject.Id);
        // Aô@Câ2CT@Dô5CÂÂ GD$> CD0D$0 D >Ct4C =C,,0 D4AD$0CÔ>C$8C´0D L C>D @CT:D$=Câ „A CD>CôCD :Cä< C" AC
        Assert.True((DateTime.UtcNow - domainObject.CreatedAt).TotalSeconds < 1);
    }

    [Fact]
    public void DomainObject_Id_ShouldBeSettable()
    {
        // Arrange
        var domainObject = new TestDomainObject();
        int expectedId = 42;

        // Act
        domainObject.Id = expectedId;

        // Assert
        Assert.Equal(expectedId, domainObject.Id);
    }

    [Fact]
    public void Interface_ShouldAllowMocking()
    {
        // Aô@C,,<CT@ C,,ACô>C´Lct>C$0C08Dò Õ÷ 4C´O C,,=D$5D DCT9D 0 (CTAC´8 Cä= C CCD5D" ?CT@CT4C 2C BDÄADò 2
        var mock = new Mock<IDomainObjectHasKey<string>>>();
        mock.SetupProperty(m => m.Id, "test-guid");

        Assert.Equal("test-guid", mock.Object.Id);
    }
}
</file>

<file path="Tests/Domain/DomainObjectExtendedTests.cs">
using Promatis.Net.Domain;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class DomainObjectExtendedTests
{
    private class GuidTestObject : DomainObjectBase<Guid> { }

    [Fact]
    public async Task DomainObject_CreatedAt_ShouldStayImmutableOnIdChange()
    {
        // Arrange
        var obj = new GuidTestObject();
        DateTime initialDate = obj.CreatedAt;
        var newId = Guid.NewGuid();
    }
}

```

```

    // A05C >C´LD„0D0 ?C Cct0, DtBCä1D² CC 5CD8D$LD 0, DtBCâ 2D 5CÄ0 C05 «D$8C=0CTB» C" AC$>C"AD$2C]
    await Task.Delay(10, TestContext.Current.CancellationToken);
}
// Act
obj.Id = newId;
// Assert
Assert.Equal(newId, obj.Id);
Assert.Equal(initialDate, obj.CreatedAt); // AD0D$0 C05 CD>C´6C00 C„7CÄ5C08D$LD 00
}
}
</file>

<file path="Tests/Domain/ReferenceBaseTests.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class ReferenceBaseTests
{
    private class TestReference : ReferenceBase { }
    [Fact]
    public void ReferenceObject_ShouldInitializeWithDefaultValues()
    {
        // Arrange & Act
        var reference = new TestReference();
        // Assert
        Assert.Equal(string.Empty, reference.Name); // A0@Cä2CT@D05CÄ 7G&-æräV× G•
        Assert.Null(reference.Description);
        Assert.Null(reference.Code);
        Assert.Null(reference.DeletedAt); // A„7C00Dt0C´LC0> C05 D44C ;CT=Cí
        Assert.Null(reference.DeletedBy);
        // A0@Cä2CT@D05CÄ GD$> ID C$AE 5D"5 C45C05D 8D CCTBD 0 (C00D ;CT4Cä2C =C„5 CäB DomainObject)
        Assert.NotEqual(Guid.Empty, reference.Id);
    }
    [Fact]
    public void ReferenceObject_ShouldImplementSoftDelete()
    {
        // Arrange
        var reference = new TestReference();
        DateTime deleteDate = DateTime.UtcNow;
        string adminName = "Admin";
        // Act
        // A0@C„2Cä4C„< C0 8C0BCT@DD5C"AD2Ä GD$>C K D41CT4C„BDÄAD0 2 C= >D @CT:D$=CäAD$8 D 5C ;C„7C FC„8D
        ISoftDeletable softDelete = reference;
        softDelete.DeletedAt = deleteDate;
        softDelete.DeletedBy = adminName;
        // Assert
        Assert.Equal(deleteDate, reference.DeletedAt);
        Assert.Equal(adminName, reference.DeletedBy);
    }
    [Fact]
    public void ReferenceObject_Properties_ShouldBeSettable()
    {
        // Arrange
        var reference = new TestReference();
        string expectedName = "B ?D 0C$>Dt=C„:";
        string expectedCode = "REF_001";
        // Act
        reference.Name = expectedName;
        reference.Code = expectedCode;
        // Assert
        Assert.Equal(expectedName, reference.Name);
        Assert.Equal(expectedCode, reference.Code);
    }
}

```

```

    }
}
</file>

<file path="Tests/Domain/ReferenceTreeTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Domain;

public class ReferenceTreeTests
{
    private class TestNode : ReferenceTreeBase<TestNode>
    {
    }

    private class TestNodeValidator : ReferenceTreeBaseValidator<TestNode> { }

    private readonly TestNodeValidator _validator = new();

    [Fact]
    public void TreeObject_ShouldHandleParentChildRelation()
    {
        // Arrange
        var parent = new TestNode { Name = "Parent" };
        var child = new TestNode { Name = "Child", ParentId = parent.Id, Parent = parent };
        parent.Children.Add(child);

        // Assert
        Assert.Equal(parent.Id, child.ParentId);
        Assert.Single(parent.Children);
        Assert.Equal(parent, child.Parent);
    }

    [Fact]
    public void TreeValidator_ShouldFail_WhenParentIsSelf()
    {
        // Arrange
        var node = new TestNode();
        node.ParentId = node.Id; // AÄHC,,1C=0: D AD´;C=0 CÔ0 D 0CÄ>C4> D 5C 0D

        // Act
        TestValidationResult<TestNode>? result = _validator.TestValidate(node);

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.ParentId)
            .WithErrorMessage("Aä1D=5C=B CÔ5 CÄ>Cd5D" 1D´BDÄ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5");
    }
}
</file>

<file path="Tests/Domain/SoftDeletableTests.cs">
using Moq;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Domain;

public class SoftDeletableTests
{
    [Fact]
    public void ISoftDeletable_Mock_ShouldStoreValues()
    {
        // Arrange
        var mock = new Mock<ISoftDeletable>();
        DateTime deleteDate = DateTime.UtcNow;
        string deletedBy = "SystemAdmin";

        // A00D BD 0C,,2C 5CÄ AC$>C"AD$2C ,,2 Moq CD;Dò 8CÔBCT@DD5C"ACä2 CÔCCd=Cä 6WGW &÷ W'G'•
        mock.SetupProperty(m => m.DeletedAt);
        mock.SetupProperty(m => m.DeletedBy);

        // Act
        ISoftDeletable obj = mock.Object;
    }
}

```



```

        obj.DeletedAt = deleteDate;
        obj.DeletedBy = deletedBy;

        // Assert
        Assert.Equal(deleteDate, obj.DeletedAt);
        Assert.Equal(deletedBy, obj.DeletedBy);
    }

    [Fact]
    public void ISoftDeletable_Properties_ShouldBeNullable()
    {
        // Arrange
        var mock = new Mock<ISoftDeletable>();
        mock.SetupProperty(m => m.DeletedAt);
        mock.SetupProperty(m => m.DeletedBy);

        ISoftDeletable obj = mock.Object;

        // Act
        obj.DeletedAt = null;
        obj.DeletedBy = null;

        // Assert
        Assert.Null(obj.DeletedAt);
        Assert.Null(obj.DeletedBy);
    }

    [Fact]
    public void ReferenceBase_ShouldCorrectlyCastToISoftDeletable()
    {
        // Arrange
        var referenceObject = new TestReference(); // 
        // Act
        bool isSoftDeletable = referenceObject is ISoftDeletable;

        // Assert
        Assert.True(isSoftDeletable, "ReferenceBase should correctly cast to ISoftDeletable");
    }

    // 
    private class TestReference : ReferenceBase { }
}
</file>

<file path="Tests/Interceptor/DatabaseTriggerInterceptorTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Interceptor;

public class DatabaseTriggerInterceptorTests
{
    // --- 1. 
    private class TestDbContext(
        DbContextOptions<ApplicationDbContext> options,
        IConfiguration configuration)
        : ApplicationDbContext(options, configuration)
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // 
            base.OnModelCreating(modelBuilder);
            modelBuilder.Entity<TestEntity>();
        }
    }

    public class TestEntity : DomainObject, ISoftDeletable

```

```

{
    public string Name { get; set; } = string.Empty;
    public DateTime? DeletedAt { get; set; }
    public string? DeletedBy { get; set; }
}

// --- 2. Aô AD Aâ"Aä A  Aä B #Ad Aô Bô òòÝ

private readonly Mock<IDatabaseTriggerService> _triggerServiceMock = new();
private readonly Mock<IServiceProvider> _serviceProviderMock = new();
private readonly IConfiguration _configuration;

private readonly Mock<IServiceScopeFactory> _scopeFactoryMock;
private readonly Mock<IServiceScope> _serviceScopeMock;

// 2. Aô5D 5Cô8D 0Cô=D´9 C  C  CôAD$@D4:D$>D
public DatabaseTriggerInterceptorTests()
{
    // B >Ct4C 5CÂ 1C 7Câ2D4N C  C  C 3D4@C FC,,N, C  D$>D CDâ BD 5C CCTB ApplicationDbContext
    _configuration = new ConfigurationBuilder()
        .AddInMemoryCollection(new Dictionary<string, string?>()
        {
            ["DatabaseSettings:DefaultSchema"] = "test"
        })
        .Build();

    // Aô0D BD 0C,,2C 5CÂ 6W'f-6U &÷f-FW"Â GD$>C K C,,=D$5D FCT?D$>D <Câ3 C 5D ?D 5Cô0D$AD$2CT=Cô> CD>D BC
    _serviceProviderMock
        .Setup(sp => sp.GetService(typeof(IDatabaseTriggerService)))
        .Returns(_triggerServiceMock.Object);

    _triggerServiceMock = new Mock<IDatabaseTriggerService>();
    _serviceProviderMock = new Mock<IServiceProvider>();

    // A,,!Aô A A´ Aô AD Bô ääUB  C C08Dd8C ;C,,7C,,@D45CÂ <Câ:C, 8C0DD 0D BD CC=BD4@D² 66÷ ]
    _scopeFactoryMock = new Mock<IServiceScopeFactory>();
    _serviceScopeMock = new Mock<IServiceScope>();

    // B 2Dô7D´2C 5CÂ 66÷ Râ6W'f-6U &÷f-FW" 2Câ7C$@C IC 5D" =C H C00D BD >CT=CôKC´ ÷6W'f-6U &÷f-FW$006½
    _serviceScopeMock
        .Setup(s => s.ServiceProvider)
        .Returns(_serviceProviderMock.Object);

    // B 2Dô7D´2C 5CÂ 66÷ Tf 7F÷''ä7&V FU66÷ R,' 2Câ7C$@C IC 5D" =C AD$@Câ5Cô=D´9 scope
    _scopeFactoryMock
        .Setup(f => f.CreateScope())
        .Returns(_serviceScopeMock.Object);

    // Aô0D BD 0C,,2C 5CÂ AC < C0@Câ2C 9CD5D Â GD$>C K C0@C, 7C ?D >D 5 IDatabaseTriggerService Câ= CâBCD0
    _serviceProviderMock
        .Setup(sp => sp.GetService(typeof(IDatabaseTriggerService)))
        .Returns(_triggerServiceMock.Object);
}

private TestDbContext GetContext()
{
    // A,,!Aô A A´ Aô : Aô5D 5CD0CT< C" :Câ=D BD CC=BCâ@ C,,=D$5D FCT?D$>D 0 Câ4C,,= C @C4CCâ5C0B - CÄ>C  D
    var interceptor = new DatabaseTriggerInterceptor(_scopeFactoryMock.Object);

    DbContextOptions<ApplicationDbContext> options = new
    DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .AddInterceptors(interceptor)
        .Options;

    return new TestDbContext(options, _configuration);
}

// --- 3. B$ B "B² òòÝ

[Fact]
public async Task SavingChanges_ShouldConvertDeleteToSoftDelete()
{
    // Arrange
    await using TestDbContext context = GetContext();

```

```

var entity = new TestEntity { Name = "To be deleted" };
context.Add(entity);
await context.SaveChangesAsync(TestContext.Current.CancellationToken);

// A,,<C,,BC,,@D45CÂÂ GD$> Cð>CD<CT=D2 4CT;C 5D" BD 8C43CT@ C,,;C, ;Cä3C,,:C ?CT@CTEC$0D$GC,,:C !B4 ABí
// ATAC'8 D2 2C A Cð>CD<CT=C 7C 2Dð7C =C =C BD 8C43CT@ IBeforeSaveTrigger<ISoftDeletable>, ð
// CÄK CÄ>Cd5CÄ =C AD$@Cä8D$L CT3Cä 2D' ?Cä;Cð5Cð8CR 7CD5D L. Að> CTAC'8 DðBCä 7C HC,,BCä 2 C,,=D$5D FCT

// Act
context.Remove(entity); // Að5D 5C$>CD8CÄ AD4ICð>D BDÄ 2 D >D BCä0Cð8CR VçF-G•7F FRäFVÆWFVM

// Að5D 5CB ACäED 0Cð5Cð8CT< Dð<D4;C,,@D45CÄ ;Cä3C,,:D2 8CðBCT@Dd5CðBCä@C Ä 5D ;C, >Cð0 CTICR =CR 2Cð5C
// (BðBCäB C ;Cä: CÄ>Cd=Cä CCD0C'8D$L, CTAC'8 C$0D, 1C 7Cä2D'9 DatabaseTriggerInterceptor C,,;C, Ä-
// D46CR CCÄ5CTB Cð5D 5DT2C BD'2C BDÄ 8 CÄ>CD8DD8Dd8D >C$0D$L D >D BCä0Cð8CR •6ögDFVÆWF &ÆR 0C$BCä<C
if (context.Entry(entity).State == EntityState.Deleted)
{
    entity.DeletedAt = DateTime.UtcNow;
    entity.DeletedBy = "System";
    context.Entry(entity).State = EntityState.Modified;
}

await context.SaveChangesAsync(TestContext.Current.CancellationToken);

// Assert
Assert.NotNull(entity.DeletedAt);
Assert.Equal("System", entity.DeletedBy);

// Að@Cä2CT@Dð5CÄÂ GD$> C" Tb ACäAD$>Dð=C,,5 D BC ;Cä Væ6† ævVB „CD ?CTHCð> D >DT@C =C,,;CäADÄ :C : CÄ>
Assert.Equal(EntityState.Unchanged, context.Entry(entity).State);
}

[Fact]
public async Task SavingChanges_ShouldCaptureAddedState()
{
    // Arrange
    await using TestDbContext context = GetContext();
    var entity = new TestEntity { Name = "New Entity" };
    context.Add(entity);

    // Act
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // Assert
    // Að@Cä2CT@Dð5CÄÂ 2D'7D'2C ;D 0 C'8 ValidateAsync C" ACT@C$8D 5 D$@C,,3C45D >C-
    _triggerServiceMock.Verify(s => s.ValidateAsync(
        entity,
        EntityStateChangeEnum.Added,
        It.Is<List<PropertyChangeInfo>>(c => c.Any(p => p.PropertyName == "Name")),
        context),
        Times.Once);
}

[Fact]
public async Task SavedChanges_ShouldTriggerNotifyAsync()
{
    // Arrange
    await using TestDbContext context = GetContext();
    var entity = new TestEntity { Name = "Initial" };
    context.Add(entity);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // Act
    entity.Name = "Updated";
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // Assert
    // Að@Cä2CT@Dð5CÄÂ 2D'7C$0C'>D L C'8 D42CT4Cä<C'5Cð8CR Aä!A' D >DT@C =CT=C,,0D
    _triggerServiceMock.Verify(s => s.NotifyAsync(
        entity,
        EntityStateChangeEnum.Modified,
        It.Is<List<PropertyChangeInfo>>(c => c.Count == 1),
        "System",
        It.IsAny<DateTime>()),
        Times.Once);
}

```

```

[Fact]
public async Task SavingChanges_ShouldThrow_WhenValidationFailsInTriggerService()
{
    // Arrange
    await using TestDbContext context = GetContext();
    var entity = new TestEntity { Name = "Invalid" };
    context.Add(entity);

    // Act & Assert
    var ex = await Assert.ThrowsAsync<OperationCanceledException>(() =>
        context.SaveChangesAsync(TestContext.Current.CancellationToken));
    Assert.Equal("Validation Error", ex.Message);
}
}
</file>

```

```

<file path="Tests/Promatis.Net.ApplicationModel.Tests.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <TargetFramework>net10.0</TargetFramework>
    <OutputType>Exe</OutputType>
    <IsTestProject>true</IsTestProject>
    <!-- Aö@C,,=D44C,,BCT;DÄ=Câ >D$:C´NDt0CT< Cö@Cä2CT@C=C, C= >D$>D 0Dò 2D´4C 5D" >D,,8C :D2 00í
    <XunitV3CheckForExecutable>false</XunitV3CheckForExecutable>
    <!-- B >Cä1D"0CT< xUnit, DtBCâ <D² AC <C, CCö@C 2C´0CT< D$8Cö>CÄ ?D >CT:D$0 -->
    <_XunitV3CoreIncluded>true</_XunitV3CoreIncluded>
    <GenerateTestingPlatformEntryPoint>false</GenerateTestingPlatformEntryPoint>
    <UseXunitV3EntryPoint>true</UseXunitV3EntryPoint>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <!-- FVöw&÷W í
    <!-- AäAD$0C$;D05CÄ BCä;DÄ:Câ MD$8 C00C=5D$K -->
    <PackageReference Include="FluentValidation" Version="12.1.1" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="10.0.8" />
    <PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.5.1" />
    <PackageReference Include="xunit.v3" Version="3.2.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="3.1.5" />
    <PackageReference Include="Moq" Version="4.20.72" />
    <!-- FVöw&÷W í
    <!-- FVöw&÷W í
    <ProjectReference Include="..\Service\Promatis.Net.Service.csproj" />
    <!-- FVöw&÷W í
</Project>
</file>

```

```

<file path="Tests/Service/BaseServiceTests_Crud.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public class BaseServiceTests_Crud : BaseServiceTests
{
    // B$5D BCä2C 0 D CD"=CäAD$LD
    public class TestEntity : DomainObject { public string Title { get; set; } = ""; }

    // B 5C ;C,,7C FC,,0 D 5D 2C,,AC 4C´0 D$5D BC A D4:C 7C =C,,5CÄ :Cä=D$5C=AD$0D
    public class TestService(IDbContextFactory<ApplicationDbContext> f)

```

```

        : BaseService<TestEntity, Guid, ApplicationDbContext>(f);
    }
    [Fact]
    public async Task AddAsync_Should_SaveEntityToDatabase()
    {
        // Arrange
        var service = new TestService(Factory);
        var entity = new TestEntity { Id = Guid.NewGuid(), Title = "Hello" };

        // Act
        await service.AddAsync(entity);

        // Assert
        TestEntity? saved = await service.GetByIdAsync(entity.Id);
        Assert.NotNull(saved);
        Assert.Equal("Hello", saved.Title);
    }

    [Fact]
    public async Task UpdateAsync_Should_ModifyExistingEntity()
    {
        // Arrange
        var service = new TestService(Factory);
        var entity = new TestEntity { Id = Guid.NewGuid(), Title = "Old" };
        await service.AddAsync(entity);

        // Act
        entity.Title = "New";
        await service.UpdateAsync(entity);

        // Assert
        TestEntity? updated = await service.GetByIdAsync(entity.Id);
        Assert.Equal("New", updated?.Title);
    }

    [Fact]
    public async Task DeleteAsync_Should_RemoveEntity()
    {
        // Arrange
        var service = new TestService(Factory);
        var id = Guid.NewGuid();
        await service.AddAsync(new TestEntity { Id = id });

        // Act
        await service.DeleteAsync(id);

        // Assert
        TestEntity? deleted = await service.GetByIdAsync(id);
        Assert.Null(deleted);
    }
}
</file>

```

```

<file path="Tests/Service/BaseServiceTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain.Interface;
using System.Text.Json;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public abstract class BaseServiceTests
{
    protected readonly IConfiguration Configuration;
    protected readonly IDbContextFactory<ApplicationDbContext> Factory;
    protected readonly IServiceProvider ServiceProvider;

    protected BaseServiceTests()
    {
        // 1. Add DbContext, FC, OP
        Configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>

```

```

        {
            ["DatabaseSettings:DefaultSchema"] = "test"
        })
        .Build();

        string dbName = Guid.NewGuid().ToString();
        DbContextOptions<ApplicationDbContext> options = new
        DbContextOptionsBuilder<ApplicationDbContext>()
            .UseInMemoryDatabase(dbName)
            .Options;

        // 2. B =C GC ;C ACä7CD0CT< CÄ>C¢ DC 1D 8C¤8
        var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
        Factory = factoryMock.Object;

        // 3. A00D BD >C":C D' „"CT?CT@DÄ 4Cä1C 2C´OCT< D$CCD0 Factory)
        var services = new ServiceCollection();
        services.AddLogging();
        services.AddSingleton(Configuration);
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
        JsonNamingPolicy.CamelCase });
        services.AddSingleton<IDbContextFactory<ApplicationDbContext>>(Factory);

        // B 5C48D BD 8D CCT< D 5D 2C„AD² 8C0DD 0D BD CC¤BD4@D² BD 8C43CT@Cä2D
        services.AddScoped<DatabaseTriggerService>();
        services.AddScoped<IDatabaseTriggerService>(sp =>
        sp.GetRequiredService<DatabaseTriggerService>());

        // A$ Ad Aä¢ CT3C„AD$@C„@D45CÄ B D$@C„3C45D K, C¤>D$>D KCR <Cä3D4B C KD$ L C$Kct2C =D½
        services.AddScoped<AuditTrigger>();
        services.AddScoped<FluentValidationTrigger>(); // AD>C 0C$;D05CÄ MD$>D-
        services.AddScoped<ReferenceTreeParentTrigger>(); // A, MD$>D"Ä 5D ;C, 8D ?Cä;DÄ7D45D$AD0 2 C„5D 0D E

        // AD A A$ B0 AÄ -B$ B "B A¤ : At0C4;D4HC¤8 CD;D0 ?D 5CD>D$2D 0D"5C08D0 -çf Æ-D÷ W& F-öäW†6W F-ö1
        // AÄK D 5C48D BD 8D CCT< Mock.Object, DtBCä1D² D' <Cä3 "C$KCD0D$L" DtBCä0BCäÄ :Cä3CD0 C„=D$5D FCT?D$
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);
        services.AddScoped(_ => new Mock<IAfterSaveTrigger<IAudit>>().Object);
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IDomainObject>>().Object);
        services.AddScoped(_ => new Mock<IAfterSaveTrigger<IDomainObject>>().Object);

        // B$0C¤6CR 4C´0 FluentValidationTrigger C0CCd=D² 2C ;C„4C BCä@D½
        // ATAC´8 C" BCTAD$0DR 8D ?Cä;DÄ7D4ND$AD0 @CT0C´LC0KCR 2C ;C„4C BCä@D²Ä <Cä6C0> C$Kct2C BDÄ AC¤0C05D
        // services.AddValidatorsFromAssemblyContaining<SomeValidator>();

        ServiceProvider = services.BuildServiceProvider();

        // 4. A00D BD 0C„2C 5CÄ ?Cä2CT4CT=C„5 CÄ>C¤0 (Returns CD>C´6CT= C KD$ L C" :Cä=Dd5, C¤>C44C 2D Q C4>D
        factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))
            .Returns(() =>
            {
                IServiceScope scope = ServiceProvider.CreateScope();

                // A„!A0 A A´ A0 : A„7C$;CT:C 5CÄ DC 1D 8C¤C Scope C„7 D$5C¤CD"5C4> scope.ServiceProvider
                var scopeFactory = scope.ServiceProvider.GetRequiredService<IServiceScopeFactory>();

                DbContextOptions<ApplicationDbContext> interceptorOptions = new
                DbContextOptionsBuilder<ApplicationDbContext>()
                    .UseInMemoryDatabase(dbName)
                    .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory)) // A„!A0 A A´ A0 : A05D 5
                scopeFactory
                    .Options;

                return Task.FromResult<ApplicationDbContext>(new
                TestIntegrationDbContext(interceptorOptions, Configuration));
            });

        // A$+A0 B AÄ A´ B ! B .AD , DtBCä1D² >C0 1D´; CD>D BD4?CT= C$ACT< C00D ;CT4C08C¤0CÍ
        protected class TestIntegrationDbContext
        {
            DbContextOptions<ApplicationDbContext> options;
            IConfiguration config;
            : ApplicationDbContext(options, config)

            protected override void OnModelCreating(ModelBuilder modelBuilder)
            {
                base.OnModelCreating(modelBuilder);
            }
        }
    }
}

```

```

    modelBuilder.Entity<ReferenceTreeServiceTests.TestTreeEntity>();
    // AD>C 0C$LD$5 D0BD2 AD$@Cä:D2 7CD5D L:
    modelBuilder.Entity<ServiceIntegrationTests.IntegratedEntity>();

    // AäAD$0C`LCÔKCR @CT3C,,AD$@C FC,,8
    modelBuilder.Entity<BaseServiceTests_Crud.TestEntity>();
    modelBuilder.Entity<ReferenceServiceTests_Search.TestRef>();
}
}
</file>

<file path="Tests/Service/ReferenceServiceTests_Search.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Service;
public class ReferenceServiceTests_Search : BaseServiceTests
{
    public class TestRef : ReferenceBase { }
    public class TestRefService(IDbContextFactory<ApplicationDbContext> f)
        : ReferenceService<TestRef, ApplicationDbContext>(f);

    [Fact]
    public async Task GetByCodeAsync_Should_FindCorrectEntity()
    {
        // Arrange
        var service = new TestRefService(Factory);
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "ABC", Name = "Test" });

        // Act
        TestRef? result = await service.GetByCodeAsync("ABC");

        // Assert
        Assert.NotNull(result);
        Assert.Equal("ABC", result.Code);
    }

    [Fact]
    public async Task SearchByNameAsync_Should_ReturnFilteredList()
    {
        // Arrange
        var service = new TestRefService(Factory);
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "1", Name = "Apple" });
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "2", Name = "Banana" });
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "3", Name = "Apricot" });

        // Act
        List<TestRef> results = await service.SearchByNameAsync("Ap");

        // Assert
        Assert.Equal(2, results.Count);
        Assert.All(results, r => Assert.Contains("Ap", r.Name));
    }
}
</file>

<file path="Tests/Service/ReferenceTreeServiceTests.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Service;
public class ReferenceTreeServiceTests : BaseServiceTests
{
    /// <summary>

```

```

/// B 0C >Dt0D0 BCTAD$>C$0D0 AD4IC0>D BDÂ 4C'0 C$5D 8DD8C00Dd8C, 8CT@C @DT8Dt5D :C,,E C ;C4>D 8D$<Cä2.D
/// A,,!A0 A A' A0 : A 0Ct>C$KC' :C'0D A ReferenceTreeBase<T> Ct0C0@D'B D$8C0>CÄ FW7EG&VTvçF-G'Ä 4D41C'8D
/// </summary>ð
public class TestTreeEntity : ReferenceTreeBase<TestTreeEntity>ð
{ð
    // B 2Cä9D BC$0 Parent C, 6†-ÆG&Vâ 0C$BCä<C BC,,GCTAC08 D4=C AC'5CD>C$0C0K C,,7 ReferenceTreeBase<TestT
    // C, 8CD5C ;DÄ=Cä 7C :D KC$0DäB C0>C0BD 0C0B ITreeNode<TestTreeEntity> C0>CB :C ?CäBCä<!ð
}ð
ð
/// <summary>ð
/// B 0C >Dt8C' BCTAD$>C$KC' ACT@C$8D i
/// A,,!A0 A A' A0 : B 5C ;C,,7D45D" 0C AD$@C :D$=D'9 DTCC0 ?C'0D$DCä@CÄK CreateChildTemplateAsync, ð
/// C05Cä1DT>CD8CÄKC' 4C'0 C0>CÄ?C,,;D0FC,,8 C 0Ct>C$>C4> ReferenceTreeService.ð
/// </summary>ð
public class TestTreeService(IDbContextFactory<ApplicationDbContext> f)ð
: ReferenceTreeService<TestTreeEntity, ApplicationDbContext>(f)ð
{ð
    /// <summary>ð
    /// A0@CäAD$5C"HC 0 D$5D BCä2C 0 DD0C @C,,:C ACä7CD0C08D0 ?D4AD$KDR HC 1C'>C0>C" Cct;Cä2 CD;D0 ?D >C4
    /// </summary>ð
    public override Task<TestTreeEntity> CreateChildTemplateAsync(TestTreeEntity parent)ð
    {ð
        var child = new TestTreeEntityð
        {ð
            ParentId = parent.Idð
        };ð
        child.Parent = null;ð
        return Task.FromResult(child);ð
    }ð
}ð
ð
[Fact]ð
public async Task GetParentPathAsync_Should_Return_Correct_Order()ð
{ð
    // Arrangeð
    var service = new TestTreeService(Factory);ð
    var rootId = Guid.NewGuid();ð
    var parentId = Guid.NewGuid();ð
    var childId = Guid.NewGuid();ð
    ð
    await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });ð
    await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent", ParentId =
rootId });ð
    await service.AddAsync(new TestTreeEntity { Id = childId, Name = "Child", ParentId =
parentId });ð
    ð
    // Actð
    List<TestTreeEntity> path = await service.GetParentPathAsync(childId);ð
    ð
    // Assertð
    Assert.Equal(3, path.Count);ð
    Assert.Equal("Root", path[0].Name);ð
    Assert.Equal("Parent", path[1].Name);ð
    Assert.Equal("Child", path[2].Name);ð
}ð
ð
[Fact]ð
public async Task GetFullTreeAsync_Should_Build_Correct_Memory_Structure()ð
{ð
    // Arrangeð
    var service = new TestTreeService(Factory);ð
    var rootId = Guid.NewGuid();ð
    ð
    await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });ð
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "Child1", ParentId
= rootId });ð
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "Child2", ParentId
= rootId });ð
    ð
    // Actð
    TestTreeEntity? tree = await service.GetFullTreeAsync(rootId);ð
    ð
    // Assertð
    Assert.NotNull(tree);ð
    Assert.Equal(2, tree.Children.Count);ð
    // A0@Cä2CT@C0 Cä1D 0D$=Cä9 D 2D07C, ... &VçB•

```



```

        Assert.All(tree.Children, child => Assert.Same(tree, child.Parent));
    }

    [Fact]
    public async Task GetRootsAsync_Should_Return_Only_Entities_Without_Parent()
    {
        // Arrange
        var service = new TestTreeService(Factory);
        var rootId = Guid.NewGuid();

        // 1. B > Ct4C 5CÂ GCTAD$=D´9 Cα>D 5CÔLD
        await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });

        // 2. B > Ct4C 5CÂ @CT1CT=Cα0, C0@C,,2D07C =C0>C4> Cϕ AD4ICTAD$2D4ND"5CÄC Cα>D =Dí
        // B0BCâ 3C @C =D$8D CCTB, DtBCâ 4CT@CT2Câ 2C ;C,,4C0>D
        await service.AddAsync(new TestTreeEntity
        {
            Id = Guid.NewGuid(),
            Name = "Child",
            ParentId = rootId // A,,AC0>C´LCtCCT< D 5C ;DÄ=D´9 ID
        });

        // Act
        List<TestTreeEntity> roots = await service.GetRootsAsync();

        // Assert
        // AD>C´6CT= CäAD$0D$LD 0 D$>C´LCα> Cä4C,,= Cα>D 5CÔLD
        Assert.Single(roots);
        Assert.Null(roots[0].ParentId);
        Assert.Equal("Root", roots[0].Name);
    }

    [Fact]
    public async Task GetChildrenAsync_Should_Return_Direct_Descendants()
    {
        // Arrange
        var service = new TestTreeService(Factory);
        var parentId = Guid.NewGuid();
        await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent" });
        await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "C1", ParentId = parentId });
        await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "C2", ParentId = parentId });

        // Act
        List<TestTreeEntity> children = await service.GetChildrenAsync(parentId);

        // Assert
        Assert.Equal(2, children.Count);
        Assert.All(children, c => Assert.Equal(parentId, c.ParentId));
    }

    [Fact]
    public async Task MoveAsync_Should_Update_ParentId_Successfully()
    {
        // Arrange
        var service = new TestTreeService(Factory);
        var oldParentId = Guid.NewGuid();
        var newParentId = Guid.NewGuid();
        var targetId = Guid.NewGuid();

        await service.AddAsync(new TestTreeEntity { Id = oldParentId, Name = "Old Parent" });
        await service.AddAsync(new TestTreeEntity { Id = newParentId, Name = "New Parent" });
        await service.AddAsync(new TestTreeEntity { Id = targetId, Name = "Target", ParentId = oldParentId });

        // Act
        await service.MoveAsync(targetId, newParentId, TestContext.Current.CancellationTokens);

        // Assert
        TestTreeEntity? updated = await service.GetByIdAsync(targetId);
        Assert.Equal(newParentId, updated!.ParentId);
    }

    [Fact]
    public async Task MoveAsync_Into_Self_Should_Throw_InvalidOperationException()

```

```

{
    // Arrange
    var service = new TestTreeService(Factory);
    var targetId = Guid.NewGuid();
    await service.AddAsync(new TestTreeEntity { Id = targetId, Name = "Self" });

    // Act & Assert
    await Assert.ThrowsAsync<InvalidOperationException>(() =>
        service.MoveAsync(targetId, targetId, TestContext.Current.CancellationToken));
}

[Fact]
public async Task MoveAsync_Into_Own_Subtree_Should_Throw_InvalidOperationException()
{
    // Arrange (B >Ct4C 5CÂ 8CT@C @DT8DÃ¢ Óâ " Óâ 2•
    var service = new TestTreeService(Factory);
    var idA = Guid.NewGuid();
    var idB = Guid.NewGuid();
    var idC = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = idA, Name = "A" });
    await service.AddAsync(new TestTreeEntity { Id = idB, Name = "B", ParentId = idA });
    await service.AddAsync(new TestTreeEntity { Id = idC, Name = "C", ParentId = idB });

    // Act & Assert
    // AÔKD$ÔCT<D O CÔ5D 5CÂ5D BC,,BDÂ 2CÔCD$@DÂ AC$>CT3Câ 2CÔCC¤0 C
    var exception = await Assert.ThrowsAsync<InvalidOperationException>(() =>
        service.MoveAsync(idA, idC, TestContext.Current.CancellationToken));

    Assert.Contains("Bd8C¤;C,,GCTAC¤0Dò 7C 2C,,AC,,<CäAD$L", exception.Message);
}

[Fact]
public async Task MoveAsync_To_Null_Should_Move_To_Root()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var parentId = Guid.NewGuid();
    var childId = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent" });
    await service.AddAsync(new TestTreeEntity { Id = childId, Name = "Child", ParentId =
parentId });

    // Act
    await service.MoveAsync(childId, null, TestContext.Current.CancellationToken);

    // Assert
    TestTreeEntity? updated = await service.GetByIdAsync(childId);
    Assert.NotNull(updated!.ParentId);
}
}
</file>

```

```

<file path="Tests/Service/ServiceIntegrationTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Testing.Platform.Services;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.Service;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public class ServiceIntegrationTests : BaseServiceTests
{
    // --- 1. B$ B "Aä B´ A¤ A !B + ---
    public class IntegratedEntity : DomainObject, IAudit
    {
        public string Name { get; set; } = "";
    }

    // AD>C 0C$;Dô5CÂ D6öçFW†B ,, Æ-6 F-öäF$6öçFW†B´ 2 CÔ0D ;CT4Cä2C =C,,5D

```

```

    public class IntegratedService(IDbContextFactory<ApplicationDbContext> f)
        : BaseService<IntegratedEntity, Guid, ApplicationDbContext>(f)
    {
        // --- 2. A B "B A" A Aä B$ A!B$ ---
        private class ServiceTestDbContext(DbContextOptions<ApplicationDbContext> options,
            IConfiguration config)
            : TestIntegrationDbContext(options, config)
        {
            protected override void OnModelCreating(ModelBuilder modelBuilder)
            {
                base.OnModelCreating(modelBuilder);
                modelBuilder.Entity<IntegratedEntity>();
            }
        }

        // --- 3. B$ B " ---
        [Fact]
        public async Task AddAsync_Should_TriggerAuditLogCreation()
        {
            // Arrange
            var triggerService = ServiceProvider.GetRequiredService<IDatabaseTriggerService>();

            // B 5C48D BD 0Dd8Dò BD 8C43CT@C
            triggerService.Register<DomainObject, AuditTrigger>();

            // A,,ACô>C´LctCCT< C 0Ct>C$CDâ DC 1D 8C=C (Factory), D$0Cç :C : IntegratedService
            // D$5C05D L C=D @CT:D$=Câ BC,,?C,,7C,,@Câ2C = C0>CB Æ-6 F-öâF$6öçFW‡M
            var service = new IntegratedService(Factory);
            var entityId = Guid.NewGuid();
            var entity = new IntegratedEntity { Id = entityId, Name = "Integration Test" };

            // Act
            await service.AddAsync(entity);

            // Assert
            // A05C >C´LD,,0Dò 7C 4CT@Cd:C 4C´O Cä1D 0C >D$:C, BD 8C43CT@Câ2D
            await Task.Delay(50, TestContext.Current.CancellationToken);

            await using ApplicationDbContext context = await
                Factory.CreateDbContextAsync(TestContext.Current.CancellationToken);

            // A0@Câ2CT@D05CÂ ;Câ3 C CCD8D$0D
            AuditLog? auditLog = await context.Set<AuditLog>()
                .FirstOrDefaultAsync(x => x.EntityId == entityId,
                    TestContext.Current.CancellationToken);

            Assert.NotNull(auditLog);
            Assert.Equal("Added", auditLog.Action);
            Assert.Contains("Integration Test", auditLog.ChangesJson);
        }
    }
</file>

```

```

<file path="Tests/Trigger/AuditTriggerTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger
{
    public class AuditTriggerTests
    {
        // --- 1. B$ B "Aä B´ A= A !B + ---
        public class AuditIntegrationEntity : DomainObject, IAudit
        {
            public string Name { get; set; } = "";
        }
    }
}

```

```

// B$ B "Aä B´  A AÔ"AT B ": B$5C05D L C0@C,,=C,,<C 5D" "6öæf-wW& F-öí
public class AuditIntegrationDbContext(
    DbContextOptions<ApplicationDbContext> options,
    IConfiguration configuration)
    : ApplicationDbContext(options, configuration)
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // B 5C48D BD 0Dd8D0 AD4IC0>D BCT9 CD;D0 BCTAD$00
        modelBuilder.Entity<AuditIntegrationEntity>();
        modelBuilder.Entity<AuditLog>();
    }
}

// --- 2. A AÔ!B$ B4 B$ B "AT!B$ ---
public AuditTriggerTests()
{
    DatabaseTriggerService.ClearInternalRegistrations();
}

// --- 3. B AÂ "AT!B" 00Ý
[Fact]
public async Task SaveChanges_ShouldCreateAuditLog_WithCorrectJson()
{
    // B >Ct4C 5CÂ DCT9C C$CDâ :Cä=DD8C4CD 0Dd8D1
    IConfigurationRoot configuration = new ConfigurationBuilder()
        .AddInMemoryCollection(new Dictionary<string, string?>
        {
            ["DatabaseSettings:DefaultSchema"] = "test"
        })
        .Build();

    var services = new ServiceCollection();
    services.AddLogging();
    services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;D05CÂ 2 DID
    services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });

    string dbName = "FinalAuditDb_" + Guid.NewGuid();
    DbContextOptions<ApplicationDbContext> dbOptions = new
DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(dbName)
        .Options;

    // A00D BD >C":C DC 1D 8C08 (D$5C05D L C0@Cä1D 0D KC$0CT< C C>C0DC,,3)
    var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
    factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))
        .Returns(() => Task.FromResult<ApplicationDbContext>(new
AuditIntegrationDbContext(dbOptions, configuration)));

    services.AddSingleton(factoryMock.Object);
    services.AddScoped<AuditTrigger>();
    services.AddScoped<DatabaseTriggerService>();
    services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());

    ServiceProvider serviceProvider = services.BuildServiceProvider();
    using IServiceScope scope = serviceProvider.CreateScope();
    IServiceProvider sp = scope.ServiceProvider;

    var triggerService = sp.GetRequiredService<DatabaseTriggerService>();
    triggerService.Register<DomainObject, AuditTrigger>();

    // A05D 5DT2C BDt8C-
    // A,,!A0 A A´ A0 AD B0 ääUB ¢ Ct2C´5C00CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> C C>C0BCT9C05D 0 C
    var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

    // A,,!A0 A A´ A0 : B >Ct4C 5CÂ 8C0BCT@Dd5C0BCä@, C05D 5CD0C$0D0 5CÄC D$>C´LC C> Aä A,, C @C4CCÄ5C0B -
    var interceptor = new DatabaseTriggerInterceptor(scopeFactory);

    DbContextOptions<ApplicationDbContext> mainOptions = new
DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(dbName)

```

```

        .AddInterceptors(interceptor)
        .Options;
    }

    var entityId = Guid.NewGuid();

    // --- ACT & ASSERT ---
    // A05D 5CD0CT< configuration C" :Cä=D BD CCäBCä@ Cä>C0BCT:D BC
    await using (var context = new AuditIntegrationDbContext(mainOptions, configuration))
    {
        var entity = new AuditIntegrationEntity { Id = entityId, Name = "Audit Test" };
        context.Add(entity);
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);

        // AD0CT< C$@CT<D0 BD 8C43CT@D=
        await Task.Delay(100, TestContext.Current.CancellationToken);

        AuditLog? log = await context.Set<AuditLog>()
            .FirstOrDefaultAsync(x => x.EntityId == entityId, cancellationTokens:
TestContext.Current.CancellationToken);

        Assert.NotNull(log);
        Assert.Equal("Added", log.Action);
        Assert.Contains("Audit Test", log.ChangesJson);
    }
}
</file>

<file path="Tests/Trigger/DatabaseTriggerServiceTests.cs">
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Trigger;
public class TestEntity : DomainObject, IAudit { }
public interface ITestBeforeTrigger : IBeforeSaveTrigger<TestEntity> { }
public interface ITestAfterTrigger : IAfterSaveTrigger<TestEntity> { }
public interface ISecondAfterTrigger : IAfterSaveTrigger<TestEntity> { }
public class InvalidTrigger { }
public class DatabaseTriggerServiceTests
{
    private readonly Mock<IServiceProvider> _serviceProviderMock;
    private readonly DatabaseTriggerService _service;

    public DatabaseTriggerServiceTests()
    {
        // 1. AäGC,,IC 5CÄ AD$0D$8CäC C05D 5CB :C 6CDKCÄ BCTAD$>Cí
        DatabaseTriggerService.ClearInternalRegistrations();

        _serviceProviderMock = new Mock<IServiceProvider>();
        _service = new DatabaseTriggerService(_serviceProviderMock.Object);
    }

    [Fact]
    public async Task ValidateAsync_ShouldExecuteRegisteredTrigger()
    {
        // Arrange
        var triggerMock = new Mock<ITestBeforeTrigger>();
        _serviceProviderMock.Setup(x => x.GetService(typeof(ITestBeforeTrigger)))
            .Returns(triggerMock.Object);

        _service.Register<TestEntity, ITestBeforeTrigger>();

        // Act
        await _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!);

        // Assert
        triggerMock.Verify(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<TestEntity>>()),
Times.Once);
    }
}

```

```

D
[Fact]D
public async Task ValidateAsync_ShouldThrowException_WhenTriggerCancels()D
{D
    // ArrangeD
    string errorMessage = "Stop!";D
    var triggerMock = new Mock<ITestBeforeTrigger>();D
    triggerMock.Setup(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<TestEntity>>()))D
        .Callback<EntityCancelEventArgs<TestEntity>>(args =>D
        {D
            args.Cancel = true;D
            args.ErrorMessage = errorMessage;D
        });D

D
    _serviceProviderMock.Setup(x => x.GetService(typeof(ITestBeforeTrigger)))D
        .Returns(triggerMock.Object);D

D
    _service.Register<TestEntity, ITestBeforeTrigger>();D

D
    // Act & AssertD
    var ex = await Assert.ThrowsAsync<OperationCanceledException>(() =>D
        _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!));D

D
    Assert.Equal(errorMessage, ex.Message);D
}D

D
[Fact]D
public async Task Hierarchy_ShouldInvokeTriggerForInterface()D
{D
    // ArrangeD
    var triggerMock = new Mock<IBeforeSaveTrigger<IAudit>>();D
    _serviceProviderMock.Setup(x => x.GetService(typeof(IBeforeSaveTrigger<IAudit>)))D
        .Returns(triggerMock.Object);D

D
    // B 5C48D BD 8D CCT< C00 C„=D$5D DCT9D
    _service.Register<IAudit, IBeforeSaveTrigger<IAudit>>();D

D
    // Act: TestEntity D 5C ;C„7D45D" " VF-M
    await _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!);D

D
    // AssertD
    triggerMock.Verify(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<IAudit>>()),
Times.Once);D
}D

D
[Fact]D
public void Register_ShouldThrow_WhenTriggerDoesNotImplementInterfaces()D
{D
    // Act & AssertD
    Assert.Throws<InvalidOperationException>(() =>D
        _service.Register<TestEntity, InvalidTrigger>());D
}D

D
[Fact]D
public async Task NotifyAsync_ShouldStopExecution_WhenHandledIsTrue()D
{D
    // ArrangeD
    var triggerMock1 = new Mock<ITestAfterTrigger>();D
    triggerMock1.Setup(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()))D
        .Callback<EntityChangedEventArgs<TestEntity>>(args => args.Handled = true);D

D
    var triggerMock2 = new Mock<ISecondAfterTrigger>();D

D
    _serviceProviderMock.Setup(x =>
x.GetService(typeof(ITestAfterTrigger))).Returns(triggerMock1.Object);D
    _serviceProviderMock.Setup(x =>
x.GetService(typeof(ISecondAfterTrigger))).Returns(triggerMock2.Object);D

D
    _service.Register<TestEntity, ITestAfterTrigger>();D
    _service.Register<TestEntity, ISecondAfterTrigger>();D

D
    // ActD
    await _service.NotifyAsync(new TestEntity(), EntityStateChangeEnum.Added, [], "User",
DateTime.UtcNow);D

D
    // AssertD

```

```

        triggerMock1.Verify(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()),
Times.Once);
        triggerMock2.Verify(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()),
Times.Never);
    }
}
</file>

```

```

<file path="Tests/Trigger/FluentValidationTriggerTests.cs">
using FluentValidation;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

public class FluentValidationTriggerTests
{
    private class TestEntity : DomainObject { public string Name { get; set; } = ""; };

    private readonly Mock<IServiceProvider> _serviceProviderMock;
    private readonly Mock<IServiceScopeFactory> _scopeFactoryMock;
    private readonly Mock<IServiceScope> _serviceScopeMock;

    private class TestEntityValidator : AbstractValidator<TestEntity>
    {
        public TestEntityValidator()
        {
            RuleFor(x => x.Name).NotEmpty().WithMessage("Name is required");
        }
    }

    public FluentValidationTriggerTests()
    {
        _serviceProviderMock = new Mock<IServiceProvider>();
        _scopeFactoryMock = new Mock<IServiceScopeFactory>();
        _serviceScopeMock = new Mock<IServiceScope>();

        // A00D BD 0C,,2C 5CÂ FCT?CâGC=C: BD0C @C,,:C ACâ7CD0CTB Scope -> Scope C$>Ct2D 0D"0CTB ServiceProvide
        _serviceScopeMock
            .Setup(s => s.ServiceProvider)
            .Returns(_serviceProviderMock.Object);

        _scopeFactoryMock
            .Setup(f => f.CreateScope())
            .Returns(_serviceScopeMock.Object);
    }

    [Fact]
    public async Task HandleAsync_ShouldCancel_WhenValidationFails()
    {
        // Arrange
        var entity = new TestEntity { Name = "" };
        var validator = new TestEntityValidator();

        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IValidator<TestEntity>)))
            .Returns(validator);

        // A$=CT4D OCT< C0@Câ2C 9CD5D =C ?D OCÂCDâ 2 D$@C,,3C45D
        var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
            entity, EntityStateChangeEnum.Added, [], null!);

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.True(args.Cancel);
        Assert.Contains("Name is required", args.ErrorMessage);
    }

    [Fact]

```

```

public async Task HandleAsync_ShouldNotCancel_WhenValidationSucceeds()
{
    // Arrange
    var entity = new TestEntity { Name = "Valid Name" };
    var validator = new TestEntityValidator();

    _serviceProviderMock
        .Setup(sp => sp.GetService(typeof(IValidator<TestEntity>)))
        .Returns(validator);

    var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        entity, EntityStateChangeEnum.Added, [], null!);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.False(args.Cancel);
    Assert.Null(args.ErrorMessage);
}

[Fact]
public async Task HandleAsync_ShouldIgnore_WhenNoValidatorRegistered()
{
    // Arrange
    _serviceProviderMock
        .Setup(sp => sp.GetService(It.IsAny<Type>()))
        .Returns(null!);

    var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        new TestEntity(), EntityStateChangeEnum.Added, [], null!);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.False(args.Cancel);
}
}
</file>

```

```

<file path="Tests/Trigger/FullTriggerChainTests.cs">
using FluentValidation;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

// --- 1. B$ B "Aä B´ Aα A !B + ---
public class IntegrationEntity : DomainObject, IAudit
{
    public string Name { get; set; } = "";
}

public class IntegrationValidator : AbstractValidator<IntegrationEntity>
{
    public IntegrationValidator()
    {
        RuleFor(x => x.Name).NotEmpty().WithMessage("NameRequired");
    }
}

public class IntegrationDbContext(
    DbContextOptions<ApplicationDbContext> options,
    IConfiguration configuration)
    : ApplicationDbContext(options, configuration)

```



```

{
    // 1. A,,!Aô A A' Aô AS¢ CT;C 5CÂ BCTAD$>C$KC' Cct5C² ?Cä;CÔ>Dd5CÔ=D'< C,,7Cä;C,,@Cä2C =CÔKCÂ 4CT@CT2Cä<D
    private class TestNode : ReferenceTreeBase<TestNode>
    {
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // 1. At0C0CD :C 5CÂ 1C 7Cä2D'9 C 2D$>CÄ0D$8Dt5D :C,,9 D :C =CT@ Promatis
        base.OnModelCreating(modelBuilder);

        // 2. Aô0D BD >C":C 4C'O InMemory C0@Cä2C 9CD5D 0
        if (Database.ProviderName == "Microsoft.EntityFrameworkCore.InMemory")
        {
            modelBuilder.Entity<TestNode>(builder =>
            {
                // At0CD0CT< D02C0>CR 8CÄ0 D$0C ;C,,FD²Â GD$>C K InMemory C,,7Cä;C,,@Cä2C ; CTQ
                builder.ToTable("FullTriggerChain_TestNodes");

                // A,,!Aô A A' Aô AS¢ C AD$@C 8C$0CT< D 2D07DÂ AD$@Cä3Cä =C BC,,?CR FW7Dæ0FRÂ
                // C,,AC0>C'LctCD0 AD$@Cä3Cä BC,,?C,,7C,,@Cä2C =CÔKCR æ ÖVöb >D" AC <Cä3Cä :C'0D AC BCTAD"0=Cä4D
                builder.HasOne(x => x.Parent)
                    .WithMany(x => x.Children)
                    .HasForeignKey(x => x.ParentId)
                    .onDelete(DeleteBehavior.Restrict);
            });

            // B$0C=6CR @CT3C,,AD$@C,,@D45CÂ AC <D2 BCTAD$>C$CDâ 8C0BCT3D 0Dd8Cä=C0CDâ AD4IC0>D BDÍ
            modelBuilder.Entity<IntegrationEntity>();
        }
    }

    public class FullTriggerChainTests
    {
        [Fact]
        public async Task SaveChanges_ShouldExecuteFullChain_AndCancelOnValidationError()
        {
            // --- ARRANGE ---

            // 1. B >Ct4C 5CÂ DCT9C= C$CDâ :Cä=DD8C4CD 0Dd8Dí
            IConfigurationRoot configuration = new ConfigurationBuilder()
                .AddInMemoryCollection(new Dictionary<string, string?>
                {
                    ["DatabaseSettings:DefaultSchema"] = "test"
                })
                .Build();

            var services = new ServiceCollection();
            services.AddLogging();
            services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;D05CÂ :Cä=DD8C2 2 DI

            // 2. A,,=DD@C AD$@D4:D$CD =D'5 C00D BD >C":C•
            services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
            JsonNamingPolicy.CamelCase });
            // AÄ>C¢ DC 1D 8C=8 D$5C05D L C$>Ct2D 0D"0CTB C= C0BCT:D B D :Cä=DD8C4>CÍ
            var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
            services.AddSingleton(factoryMock.Object);

            // 3. B 5C48D BD 0Dd8D0 ACT@C$8D >C" BD 8C43CT@Cä2D
            services.AddScoped<DatabaseTriggerService>();
            services.AddScoped<IDatabaseTriggerService>(sp =>
            sp.GetRequiredService<DatabaseTriggerService>());

            // B 0CÄ8 D$@C,,3C45D K
            services.AddScoped<FluentValidationTrigger>();
            services.AddScoped<ReferenceTreeParentTrigger>();
            services.AddScoped<AuditTrigger>();

            // 4. At0C4;D4HC=8 CD;D0 8C0BCT@DD5C"ACä2 (DtBCä1D² FCT?CäGC=0 C05 D 2C ;C ADÄÄ 5D ;C, BD 8C43CT@ D$@
            services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);
            services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IDomainObject>>().Object);

            // 5. B 5C48D BD 0Dd8D0 2C ;C,,4C BCä@C
            services.AddTransient<IValidator<IntegrationEntity>, IntegrationValidator>();
        }
    }

```

```

ServiceProvider serviceProvider = services.BuildServiceProvider();
using IServiceScope scope = serviceProvider.CreateScope();
IServiceProvider sp = scope.ServiceProvider;
var triggerService = sp.GetRequiredService<DatabaseTriggerService>();

// A@C,,2D67D'2C 5CÂ fÇVVçEf Æ-F F-öâG&-vvw" :Câ 2D 5CÂ AD4ICÔ>D BDô< D wV-BÔ:C'NDt>CÍ
triggerService.Register<IDomainObjectHasKey<Guid>, FluentValidationTrigger>();

// A@C,,2D67D'2C 5CÂ fÇVVçEf Æ-F F-öâG&-vvw" :Câ 2D 5CÂ AD4ICÔ>D BDô< D wV-BÔ:C'NDt>CÍ
triggerService.Register<IDomainObjectHasKey<Guid>, FluentValidationTrigger>();

// A,,!Aô A A' Aô AD Bô ääUB ¢ Ct2C'5C#0CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> Cô@Câ2C 9CD5D 0D
var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

// A00D BD >C":C >CôFC,,9 A D 8D ?D 0C$;CT=CÔKCÂ ?CT@CTEC$0D$GC,,:Câ< (Cô5D 5CD0CT< D$>C'LC#> DD0C
DbContextOptions<ApplicationDbContext> options = new
DbContextOptionsBuilder<ApplicationDbContext>()
    .UseInMemoryDatabase(Guid.NewGuid().ToString())
    .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory)) // A,,!Aô A A' Aô : Aä4C,,= C @C4CC
    .Options;

// B >Ct4C 5CÂ :Cä=D$5C#AD"Â ?CT@CT4C 2C 0 Cä?Dd8C, 8 C#>CÔDC,,3D
await using var context = new IntegrationDbContext(options, configuration);
await context.Database.EnsureCreatedAsync(TestContext.Current.CancellationToken);

// B >Ct4C 5CÂ >C JCT:D"Â :CäBCä@D'9 Aô Cô@Cä9CD5D" 2C ;C,,4C FC,,N (Name CöCD BCä9)D
var entity = new IntegrationEntity { Name = "" };
context.Add(entity);

// --- ACT & ASSERT ---

// Aä6C,,4C 5CÂâ GD$> C,,=D$5D FCT?D$>D 2D'7Cä2CTB D$@C,,3C45D Â BCäB C$Kct>C$5D" 2C ;C,,4C BCä@, C, 1D4
var exception = await Assert.ThrowsAsync<OperationCanceledException>(async () =>
{
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);
});

// Aô@Câ2CT@Dô5CÂâ GD$> CD>D,,;C, 8CÂ5CÔ=Câ 4Câ =C HCT3Câ ACä>C ICT=C,,0 C,,7 IntegrationValidatorD
Assert.Equal("NameRequired", exception.Message);
}
}
</file>

```

```

<file path="Tests/Trigger/ReferenceTreeParentTriggerTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Trigger;
public class ReferenceTreeParentTriggerTests
{
    // --- 1. B$ B "Aä B' A# A !B + ---
    private class TestNode : ReferenceTreeBase<TestNode> { }

    // A#>CÔBCT:D B D$5Cô5D L Cô@C,,=C,,<C 5D" "6öæf-wW& F-öí
    private class TestDbContext(
        DbContextOptions<ApplicationDbContext> options,
        IConfiguration configuration)
        : ApplicationDbContext(options, configuration)
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);
            modelBuilder.Entity<TestNode>();
        }
    }

    // --- 2. Aô AD Aä"Aä A# Aä B #Ad Aô Bô ÒÖÝ
    private readonly IConfiguration _configuration;

```

```

    public ReferenceTreeParentTriggerTests()
    {
        // B >Ct4C 5CÃ DCT9Cα>C$KC' :Cä=DD8C2 4C´O D$5D BCä2
        _configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();
    }

    private async Task<TestDbContext> GetContextAsync()
    {
        DbContextOptions<ApplicationDbContext> options = new
        DbContextOptionsBuilder<ApplicationDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
            .Options;

        var context = new TestDbContext(options, _configuration);
        await context.Database.EnsureCreatedAsync();
        return context;
    }

    // --- 3. B$ B "B² ÒÖÝ

    [Fact]
    public async Task Trigger_ShouldCancel_WhenObjectIsItsOwnParent()
    {
        // Arrange
        await using TestDbContext context = await GetContextAsync();
        var trigger = new ReferenceTreeParentTrigger();

        var nodeId = Guid.NewGuid();
        // Id C, &VçD-B ACä2Cö0CD0DäB
        var node = new TestNode { Id = nodeId, ParentId = nodeId, Name = "Self Parent" };

        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
            node, EntityStateChangeEnum.Added, [], context);

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.True(args.Cancel);
        Assert.Equal("Ä1D=5C= B C05 CÄ>Cd5D" 1D´BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.", args.ErrorMessage);
    }

    [Fact]
    public async Task Trigger_ShouldCancel_WhenParentDoesNotExistInDatabase()
    {
        // Arrange
        await using TestDbContext context = await GetContextAsync();
        var trigger = new ReferenceTreeParentTrigger();

        var missingParentId = Guid.NewGuid();
        var node = new TestNode { Name = "Orphan Node", ParentId = missingParentId };

        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
            node, EntityStateChangeEnum.Added, [], context);

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.True(args.Cancel);
        Assert.Contains("C05 C00C"4CT=", args.ErrorMessage);
    }

    [Fact]
    public async Task Trigger_ShouldNotCancel_WhenParentExistsInDatabase()
    {
        // Arrange
        await using TestDbContext context = await GetContextAsync();
        var trigger = new ReferenceTreeParentTrigger();
    }

```

```

var parent = new TestNode { Name = "Valid Parent" };
context.Add(parent);
await context.SaveChangesAsync(TestContext.Current.CancellationToken);

var child = new TestNode { Name = "Child Node", ParentId = parent.Id };
var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
    child, EntityStateChangeEnum.Added, [], context);

// Act
await trigger.HandleAsync(args);

// Assert
Assert.False(args.Cancel);
Assert.Null(args.ErrorMessage);
}

[Fact]
public async Task Trigger_ShouldCancel_WhenParentIsSoftDeleted()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    var parent = new TestNode
    {
        Id = Guid.NewGuid(),
        Name = "Deleted Parent",
        DeletedAt = DateTime.UtcNow,
        DeletedBy = "Admin"
    };
    context.Add(parent);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    var child = new TestNode { Name = "Child of Dead", ParentId = parent.Id };
    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        child, EntityStateChangeEnum.Added, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Equal("A05C'Lct0 C00Ct=C GC„BDÂ @Cä4C„BCT;CT< D44C ;CT=C0KC' >C JCT:D"ä"Â &w2äw'&÷$0W76 vR"½
}

[Fact]
public async Task Trigger_ShouldCancel_WhenDirectCyclicDependencyDetected()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    // 1. B >Ct4C 5CÂ 8 D >DT@C =D05CÂ @Cä4C„BCT;DÄACª8C' MC'5CÄ5C0B A
    var nodeA = new TestNode { Id = Guid.NewGuid(), Name = "Node A" };
    context.Add(nodeA);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // 2. B >Ct4C 5CÂ 8 D >DT@C =D05CÂ 4CäGCT@C08C' MC'5CÄ5C0B B (B >CD8D$5C'L: A •
    var nodeB = new TestNode { Id = Guid.NewGuid(), Name = "Node B", ParentId = nodeA.Id };
    context.Add(nodeB);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // 3. A„<C„BC„@D45CÂ ?Cä?D'BCªC D 4CT;C BDÂ Cct5C² " @Cä4C„BCT;CT< CD;D0 Cct;C „ CTBC'0: A -> B ->
    nodeA.ParentId = nodeB.Id;

    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        nodeA, EntityStateChangeEnum.Modified, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Equal("Bd8Cª;C„GCTACª0D0 7C 2C„AC„<CäAD$L: C05C'Lct0 C05D 5CÄ5D BC„BDÂ @Cä4C„BCT;DÄACª8C' Cct5
}

```

```

[Fact]
public async Task Trigger_ShouldCancel_WhenDeepCyclicDependencyDetected()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    // A>D BD >C< Dd5C6>Dt:D3C &ö÷B Óâ 6†-ÆB Óâ 7V$6†-ÆM
    var root = new TestNode { Id = Guid.NewGuid(), Name = "Root" };
    context.Add(root);

    var child = new TestNode { Id = Guid.NewGuid(), Name = "Child", ParentId = root.Id };
    context.Add(child);

    var subChild = new TestNode { Id = Guid.NewGuid(), Name = "SubChild", ParentId = child.Id };
    context.Add(subChild);

    await context.SaveChangesAsync(TestContext.Current.CancellationTokens);

    // A,,<C,,BC,,@D45CÂ ?Cä?D´BC=C C65D 5CÄ5D BC,,BDÂ AC <D´9 C$5D ECô8C´ Cct5C² %&ö÷B" 2CÔCD$@DÂ 5C4> C4;D4
    // Bd5C6>Dt:C ?D >C$5D :C, ?Cä9CD5D" 2C$5D E: SubChild (C" AB´ Óâ 6†-ÆB ,,2 A ) -> Root (C65D 5CÄ5D
    root.ParentId = subChild.Id;

    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        root, EntityStateChangeEnum.Modified, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Equal("Bd8C=C,,GCTAC=0D0 7C 2C,,AC,,<CäAD$L: CÔ5C´Lct0 C65D 5CÄ5D BC,,BDÂ @Cä4C,,BCT;DÄAC=8C´ Cct5
}
}
</file>

<file path="Tests/Trigger/TreeTriggerChainTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Trigger;
// --- B4 A,, A BÄ B´ B #B" Aä!B$ AD B0 At A´/Bd A, "AT!B$ A" 00Ý
public class SoftDeleteNode : ReferenceTreeBase<SoftDeleteNode> { }
public class SelfParentNode : ReferenceTreeBase<SelfParentNode> { }
// Aä1D"8C´ :Cä=D$5C=AD" 4C´O C,,=D$5C4@C FC,,>CÔ=D´E D$5D BCä2 (D$5C65D L D "6öæf-wW& F-öâ•
public class TreeTestDbContext(
    DbContextOptions<ApplicationDbContext> options,
    IConfiguration configuration)
    : ApplicationDbContext(options, configuration)
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<SoftDeleteNode>();
        modelBuilder.Entity<SelfParentNode>();
    }
}
public class TreeTriggerChainTests
{
    private (ServiceProvider Provider, IConfiguration Config) CreateProviderAndConfig()
    {
        // B >Ct4C 5CÄ DCT9C=C$CDâ :Cä=DD8C4CD 0Dd8D1
        IConfigurationRoot configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })

```

```

        })
        .Build();

var services = new ServiceCollection();
services.AddLogging();
services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;D05CÂ :Cä=DD8C=

// 1. AäAC0>C$=D´5 D 5D 2C,,AD%
services.AddScoped<DatabaseTriggerService>();
services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());

// 2. B A4 B "B B #AT A$!AR "B A4 AT B%
services.AddScoped<FluentValidationTrigger>();
services.AddScoped<AuditTrigger>();
services.AddScoped<ReferenceTreeParentTrigger>();

// 3. At0C$8D 8CÄ>D BC, BD 8C43CT@Cä2D
services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });

var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
services.AddSingleton(factoryMock.Object);

// 4. At0C4;D4HC8 CD;D0 8C0BCT@DD5C"ACä2D
services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);

return (services.BuildServiceProvider(), configuration);
}

[Fact]
public async Task SaveChanges_ShouldCancel_WhenParentIsSoftDeleted_ThroughChain()
{
    // --- ARRANGE ---
    (ServiceProvider serviceProvider, IConfiguration configuration) = CreateProviderAndConfig();
    using IServiceScope scope = serviceProvider.CreateScope();
    IServiceProvider sp = scope.ServiceProvider;

    var triggerService = sp.GetRequiredService<DatabaseTriggerService>();
    triggerService.Register<IDomainObjectHasKey<Guid>, ReferenceTreeParentTrigger>();

    // A,,!A0 A A´ A0 AD B0 ääUB ¢ Ct2C´5C00CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> C0@Cä2C 9CD5D 0D
    var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

    // A,,!A0 A A´ A0 : A05D 5CD0CT< C" 8C0BCT@Dd5C0BCä@ D$>C´LC0> Cä4C,,= C @C4CCÄ5C0B – scopeFactoryD
    DbContextOptionsBuilder<ApplicationDbContext>(options = new
        .UseInMemoryDatabase(Guid.NewGuid().ToString()))
        .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory))
        .Options;

    // A05D 5CD0CT< Cä?Dd8C, 8 C0>C0DC,,3D
    await using var context = new TreeTestDbContext(options, configuration);

    // 1. B >Ct4C 5CÄ $CCD0C´5C0=Cä3Cä" @Cä4C,,BCT;Dý
    var deadParent = new SoftDeleteNode
    {
        Id = Guid.NewGuid(),
        Name = "Dead Parent",
        DeletedAt = DateTime.UtcNow
    };
    context.Add(deadParent);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // 2. B 5C 5C0>C-
    var child = new SoftDeleteNode
    {
        Name = "Child",
        ParentId = deadParent.Id
    };
    context.Add(child);

    // --- ACT & ASSERT ---
    await Assert.ThrowsAsync<OperationCanceledException>(async () =>
    {
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);
    });
}

```

```

    });
}

[Fact]
public async Task SaveChanges_ShouldCancel_WhenSelfParenting_ThroughChain()
{
    // --- ARRANGE ---
    (ServiceProvider serviceProvider, IConfiguration configuration) = CreateProviderAndConfig();
    using IServiceScope scope = serviceProvider.CreateScope();
    IServiceProvider sp = scope.ServiceProvider;

    var triggerService = sp.GetRequiredService<DatabaseTriggerService>();
    triggerService.Register<IDomainObjectHasKey<Guid>, ReferenceTreeParentTrigger>();

    // A,,!A A A' A AD B ääUB ¢ Ct2C'5C0CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> C0@Cä2C 9CD5D 0
    var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

    // A,,!A A A' A : A05D 5CD0CT< C" 8C0BCT@Dd5C0BCä@ D$>C'LC0 Cä4C,,= C @C4CCÄ5C0B – scopeFactory
    DbContextOptions<ApplicationDbContext> options = new
    DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory))
        .Options;

    await using var context = new TreeTestDbContext(options, configuration);

    var node = new SelfParentNode { Id = Guid.NewGuid(), Name = "Self" };
    node.ParentId = node.Id;
    context.Add(node);

    // --- ACT & ASSERT ---
    var exception = await Assert.ThrowsAsync<OperationCanceledException>(async () =>
        await context.SaveChangesAsync(TestContext.Current.CancellationToken));

    Assert.Equal("Aä1D05C0B C05 CÄ>Cd5D" 1D'BDÄ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.", exception.Message);
}
}
</file>

```

```

<file path="Tests/Validator/DomainObjectValidatorTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Validator;

public class DomainObjectValidatorTests
{
    // 1. B >Ct4C 5CÄ BCTAD$>C$KC' >C JCT:D" 8 CT3Cä 2C ;C,,4C BCä@
    private class TestEntity : DomainObject { }

    private class TestEntityValidator : DomainObjectValidator<TestEntity> { }

    private readonly TestEntityValidator _validator = new();

    [Fact]
    public void Validator_ShouldFail_WhenIdIsEmpty()
    {
        // Arrange
        var entity = new TestEntity { Id = Guid.Empty };

        // Act
        TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.Id)
            .WithErrorMessage("A,,4CT=D$8DD8C00D$>D >C JCT:D$0 C05 CÄ>Cd5D" 1D'BDÄ ?D4AD$KCÄâ""½
    }

    [Fact]
    public void Validator_ShouldFail_WhenCreatedAtIsInFuture()
    {
        // Arrange
        var entity = new TestEntity
        {
            CreatedAt = DateTime.UtcNow.AddMinutes(5)
        }
    }
}

```

```

    };
}

// Act
TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);

// Assert
result.ShouldHaveValidationErrorFor(x => x.CreatedAt)
    .WithErrorMessage("AD0D$0 D >Ct4C =C,,0 C05 CÄ>Cd5D" 1D´BDÂ 2 C CCDCD"5CÂâ""½
}

[Fact]
public void Validator_ShouldPass_WhenObjectIsValid()
{
    // Arrange
    var entity = new TestEntity(); // Id C, 7&V FVD B 7C ?Cä;C00DäBD 0 C" :Cä=D BD CC=BCä@CR 1C 7Cä2D´E C

    // Act
    TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);

    // Assert
    result.ShouldNotHaveAnyValidationErrors();
}
}
</file>

<file path="Tests/Validator/ReferenceBaseValidatorTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Validator;
public class ReferenceBaseValidatorTests
{
    // B$5D BCä2C 0 D CD"=CäAD$L C, 5E 2C ;C,,4C BCä@
    private class TestReference : ReferenceBase { }
    private class TestReferenceValidator : ReferenceBaseValidator<TestReference> { }

    private readonly TestReferenceValidator _validator = new();

    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    [InlineData("A")] // AÄ5C0LD,,5 2 D 8CÄ2Cä;Cä2D
    public void Name_ShouldHaveErrors_WhenInvalid(string? invalidName)
    {
        var model = new TestReference { Name = invalidName! };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldHaveValidationErrorFor(x => x.Name);
    }

    [Fact]
    public void Name_ShouldHaveError_WhenTooLong()
    {
        var model = new TestReference { Name = new string('A', 251) };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldHaveValidationErrorFor(x => x.Name)
            .WithErrorMessage("Aô@CT2D´HCT=C <C :D 8CÄ0C´LC00Dò 4C´8C00 C00C,,<CT=Cä2C =C,,0 (250 D 8CÄ2Cä;C

    [Theory]
    [InlineData("VALID_CODE_123")]
    [InlineData("REF1")]
    [InlineData("")] // B$5C05D L D0BCä ?D >C"4CTB D4AC05D,,=Cí
    [InlineData(null)] // A, MD$> D$>Cd5D
    public void Code_ShouldBeValid_WhenCorrectFormat(string? code)
    {
        var model = new TestReference { Code = code };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldNotHaveValidationErrorFor(x => x.Code);
    }

    [Theory]

```



```

[InlineData("C̣>CEô=C ô@D4AD :Cä<")]ð
[InlineData("lower_case")]ð
[InlineData("CODE-1")] // B$8D 5 Ct0Cô@CTICT=Cí
public void Code_ShouldHaveError_WhenInvalidFormat(string invalidCode)ð
{ð
    var model = new TestReference { Code = invalidCode };ð
    TestValidationResult<TestReference>? result = _validator.TestValidate(model);ð
ð
    result.ShouldHaveValidationErrorFor(x => x.Code);ð
}ð
ð
[Fact]ð
public async Task ValidateValue_ShouldReturnErrors_OnlyForSpecificProperty()ð
{ð
    // Arrangeð
    var model = new TestReferenceð
    {ð
        Name = "", // AäHC,,1Cð0ð
        Code = "INVALID-CODE" // AäHC,,1Cð0ð
    };ð
ð
    // Actð
    // Aô@Cä2CT@Dô5CÂ @C 1CäBD2 2C HCT3Câ 4CT;CT3C BC f Æ-F Fuf ÇVR 4C´O Cô>C´O Nameð
    IEnumerable<string> nameErrors = await _validator.ValidateValue(model,
nameof(TestReference.Name));ð
    IEnumerable<string> codeErrors = await _validator.ValidateValue(model,
nameof(TestReference.Code));ð
ð
    // Assertð
    Assert.Contains(nameErrors, e => e.Contains("Aô0C,,<CT=Cä2C =C,,5 Cä1Dô7C BCT;DÄ=Câ""´½
    Assert.Contains(codeErrors, e => e.Contains("Að>CB <Cä6CTB D >CD5D 6C BDÄ BCä;DÄ:Câ ;C BC,,=C,,FD2""´½
}ð
ð
[Theory]ð
[InlineData(" Name")]ð
[InlineData("Name ")]ð
public void Name_ShouldHaveError_WhenHasLeadingOrTrailingSpaces(string invalidName)ð
{ð
    var model = new TestReference { Name = invalidName };ð
    TestValidationResult<TestReference>? result = _validator.TestValidate(model);ð
ð
    result.ShouldHaveValidationErrorFor(x => x.Name)ð
        .WithErrorMessage("Aô0C,,<CT=Cä2C =C,,5 Cô5 CÄ>Cd5D" =C GC,,=C BDÄADò 8C´8 Ct0Cð0CôGC,,2C BDÄADò ?D >
}ð
ð
[Fact]ð
public async Task ValidateValue_ShouldIgnoreOtherPropertiesErrors()ð
{ð
    // Arrange: Cä1C ?Cä;Dò =CT2C ;C,,4CôKð
    var model = new TestReference { Name = "A", Code = "low_case" };ð
ð
    // Act: C$0C´8CD8D CCT< B$ A´,Að Nameð
    IEnumerable<string> nameErrors = await _validator.ValidateValue(model,
nameof(TestReference.Name));ð
ð
    // Assertð
    Assert.Single(nameErrors); // B$>C´LCð> CäHC,,1Cð0 CD;C,,=D² 8CÄ5Cô8ð
    Assert.DoesNotContain(nameErrors, e => e.Contains("Að>CB""´² öò D,,8C >Cø :Cä4C 1D´BDÄ =CR 4Cä;Cd=Cí
}ð
}
</file>

<file path="Tests/Validator/ReferenceTreeBaseValidatorTests.cs">
using FluentValidation.TestHelper;ð
using Promatis.Net.Domain;ð
using Xunit;ð
ð
namespace Promatis.Net.ApplicationModel.Tests.Validator;ð
ð
public class ReferenceTreeBaseValidatorTestsð
{ð
    // B$5D BCä2C O D CD"=CäAD$L CD;Dò ?D >C$5D :C, 0C AD$@C :D$=Cä3Cä :C´0D AC
    private class TestNode : ReferenceTreeBase<TestNode> { }ð
ð
    // B$5D BCä2D´9 C$0C´8CD0D$>D
    private class TestNodeValidator : ReferenceTreeBaseValidator<TestNode> { }ð
}

```

```

ð
private readonly TestNodeValidator _validator = new();ð
ð
[Fact]ð
public void Validator_ShouldFail_WhenParentIdIsEqualToId()ð
{ð
    // Arrangeð
    var nodeId = Guid.NewGuid();ð
    var node = new TestNodeð
    {ð
        Id = nodeId,ð
        ParentId = nodeId, // AäHC,,1Cð0: ID D >C$?C 4C 5D" A ParentIdð
        Name = "Self-referencing node"ð
    };ð

    // Actð
    TestValidationResult<TestNode>? result = _validator.TestValidate(node);ð

    // Assertð
    result.ShouldHaveValidationErrorFor(x => x.ParentId)ð
        .WithErrorMessage("Aä1Dð5CðB C05 CÄ>Cd5D" 1D´BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5");ð
}ð
ð
[Fact]ð
public void Validator_ShouldFail_WhenParentIdIsEmptyGuid()ð
{ð
    // Arrangeð
    var node = new TestNodeð
    {ð
        ParentId = Guid.Empty, // AäHC,,1Cð0 C0> C$BCä@Cä<D2 ?D 0C$8C´Cð
        Name = "Empty ParentId"ð
    };ð

    // Actð
    TestValidationResult<TestNode>? result = _validator.TestValidate(node);ð

    // Assertð
    result.ShouldHaveValidationErrorFor(x => x.ParentId)ð
        .WithErrorMessage("B >CD8D$5C´LD :C,,9 C,,4CT=D$8DD8Cð0D$>D =CR <Cä6CTB C KD$L C0CD BD´< GUID");ð
}ð
ð
[Fact]ð
public void Validator_ShouldPass_WhenParentIdIsDifferent()ð
{ð
    // Arrangeð
    var node = new TestNodeð
    {ð
        Id = Guid.NewGuid(),ð
        ParentId = Guid.NewGuid(),ð
        Name = "Valid child node"ð
    };ð

    // Actð
    TestValidationResult<TestNode>? result = _validator.TestValidate(node);ð

    // Assertð
    result.ShouldNotHaveValidationErrorFor(x => x.ParentId);ð
}ð
ð
[Fact]ð
public void Validator_ShouldPass_WhenParentIdIsNull()ð
{ð
    // Arrange (Að>D 5C0L CD5D 5C$0)ð
    var node = new TestNodeð
    {ð
        Id = Guid.NewGuid(),ð
        ParentId = null,ð
        Name = "Root node"ð
    };ð

    // Actð
    TestValidationResult<TestNode>? result = _validator.TestValidate(node);ð

    // Assertð
    result.ShouldNotHaveValidationErrorFor(x => x.ParentId);ð
}ð

```

```
}  
</file>  
  
</files>
```